

COMPLETE TEXTBOOK SERIES

• MODERN C++17 •

Data Structures and Algorithms

in C++

From Zero to Advanced: a complete, self-study
reference with code, dry runs, practice problems
and interview preparation.

34 chapters | 5 modules of revision | 6 appendices
150+ practice problems with full solutions | 200+ worked examples

Version 1.0
C++17 Edition

Front Matter

This textbook is a complete, self-paced course on Data Structures and Algorithms using modern C++ (C++17). It is organised into modules and chapters, each containing theory, analogies, diagrams, complete commented code, line-by-line dry runs, complexity analysis and graded practice problems with full solutions. Every few chapters you will find revision sheets, formula sheets, cheat sheets and common interview patterns. Six appendices close the book with a full STL reference, a Big-O cheat sheet, frequently asked interview questions, a study roadmap, a curated problem list and a glossary.

Table of Contents

FOUNDATIONS OF C++ FOR DSA

1. C++ Fundamentals for Data Structures ..	4
1.1 Welcome and How to Use This Book ..	4
1.2 Variables, Types and I/O ..	4
1.3 Functions, References and Pointers ..	6
1.4 Classes and Structures ..	6
1.5 Templates: Write Once, Use with Any Type ..	7
1.6 The Standard Template Library in One Page ..	7
1.7 Complexity Preview ..	7
1.8 Interview Tips ..	8
1.9 Summary ..	8
Practice Set: C++ Fundamentals ..	9
Beginner Problems ..	9
Intermediate Problems ..	9
Advanced Problems ..	9
Interview-Style Problems ..	9
Competitive Programming Problems ..	9
Challenge Exercises ..	9
Solutions: C++ Fundamentals ..	10
Beginner Solutions ..	10
Intermediate Solutions ..	10
Advanced Solutions ..	10
Interview-Style Solutions ..	11
Competitive Programming Solutions ..	11
Challenge Solutions ..	11
2. STL Essentials: Containers, Iterators, Algorithms ..	12
2.1 Introduction ..	12
2.2 Vector - the Workhorse Dynamic Array ..	12
2.3 List, Deque, and the Container Adapters ..	14
2.4 set, map, multiset, multimap ..	15
2.5 pair, tuple and structured bindings ..	16
2.6 Algorithms ..	16
2.7 Lambdas ..	17
2.8 Custom structs in containers ..	18
2.9 Complexity Cheat Sheet for Common Operations ..	18
2.10 Common Mistakes ..	19
2.11 Interview Tips ..	19

2.12 Summary ..	19
Practice Set: STL Essentials ..	20
Beginner Problems ..	20
Intermediate Problems ..	20
Advanced Problems ..	20
Interview-Style Problems ..	20
Competitive Programming Problems ..	20
Challenge Exercises ..	20
Solutions: STL Essentials ..	22
Beginner Solutions ..	22
Intermediate Solutions ..	22
Advanced Solutions ..	22
Interview-Style Solutions ..	23
Competitive Programming Solutions ..	23
Challenge Solutions ..	23
COMPLEXITY ANALYSIS	
3. Time and Space Complexity ..	25
3.1 Introduction ..	25
3.2 The Formal Zoo: O, Omega, Theta ..	25
3.3 Growth Rates ..	25
3.4 How to Count Steps ..	26
3.5 Space Complexity ..	26
3.6 Common Complexity Patterns in One Picture ..	27
3.7 The Master Theorem ..	27
3.8 Amortized Analysis ..	28
3.9 Common Mistakes ..	29
3.10 Applications ..	29
3.11 Interview Tips ..	29
3.12 Summary ..	29
Practice Set: Complexity Analysis ..	30
Beginner Problems ..	30
Intermediate Problems ..	30
Advanced Problems ..	30
Interview-Style Problems ..	30
Competitive Programming Problems ..	30
Challenge Exercises ..	30
Solutions: Complexity Analysis ..	31
Beginner Solutions ..	31
Intermediate Solutions ..	31
Advanced Solutions ..	31

Interview-Style Solutions ..	31
Competitive Programming Solutions ..	31
Challenge Solutions ..	31
ARRAYS, STRINGS AND SEARCHING	
4. Arrays ..	33
4.1 Introduction ..	33
4.2 Theory: Why Random Access is Fast ..	33
4.3 Real-World Analogy ..	33
4.4 Static vs Dynamic Arrays in C++ ..	33
4.5 Operations on Arrays ..	34
4.6 Two Pointers ..	34
4.7 Sliding Window ..	35
4.8 Dry Run: Two Pointers with a = [1,3,5,7,9], K = 10 ..	35
4.9 Applications ..	35
4.10 Common Mistakes ..	35
4.11 Interview Tips ..	36
4.12 Summary ..	36
Practice Set: Arrays ..	37
Beginner Problems ..	37
Intermediate Problems ..	37
Advanced Problems ..	37
Interview-Style Problems ..	37
Competitive Programming Problems ..	37
Challenge Exercises ..	37
Solutions: Arrays ..	38
Beginner Solutions ..	38
Intermediate Solutions ..	38
Advanced Solutions ..	38
Interview-Style Solutions ..	39
Competitive Programming Solutions ..	39
5. Strings ..	40
5.1 Introduction ..	40
5.2 Common Operations and Their Cost ..	40
5.3 Theory: String == Array of Char ..	41
5.4 Real-World Analogy ..	41
5.5 Worked Examples ..	41
5.6 Dry Run: isAnagram("listen", "silent") ..	41
5.7 String Algorithms Preview ..	42
5.8 Common Mistakes ..	42
5.9 Applications ..	42

5.10 Interview Tips ..	42
5.11 Summary ..	42
Practice Set: Strings ..	44
Beginner Problems ..	44
Intermediate Problems ..	44
Advanced Problems ..	44
Interview-Style Problems ..	44
Competitive Programming Problems ..	44
Challenge Exercises ..	44
Solutions: Strings ..	45
Beginner Solutions ..	45
Intermediate Solutions ..	45
Advanced Solutions ..	45
Interview-Style Solutions ..	45
Competitive Programming Solutions ..	46
Challenge Solutions ..	46
6. Searching Algorithms ..	47
6.1 Introduction ..	47
6.2 Linear Search - $O(n)$..	47
6.3 Binary Search - $O(\log n)$..	47
6.4 Binary Search Variants - the real interview value ..	48
6.5 Ternary Search - $O(\log n)$ on unimodal functions ..	49
6.6 Interpolation Search - $O(\log \log n)$ average ..	50
6.7 Complexity Comparison ..	50
6.8 Dry Run: binary search for $x = 3$ in $[1,2,3,4,5]$..	50
6.9 Common Mistakes ..	50
6.10 Applications ..	51
6.11 Interview Tips ..	51
6.12 Summary ..	51
Practice Set: Searching ..	52
Beginner Problems ..	52
Intermediate Problems ..	52
Advanced Problems ..	52
Interview-Style Problems ..	52
Competitive Programming Problems ..	52
Challenge Exercises ..	52
Solutions: Searching ..	53
Beginner Solutions ..	53
Intermediate Solutions ..	53
Advanced Solutions ..	53
Interview-Style Solutions ..	54

Competitive Programming Solutions ..	54
Challenge Solutions ..	54
SORTING	
7. Sorting Algorithms ..	56
7.1 Introduction ..	56
7.2 The $O(n^2)$ Sorts: when simplicity wins ..	56
7.3 Merge Sort - $O(n \log n)$, stable, guaranteed ..	57
7.4 Quick Sort - $O(n \log n)$ average, $O(n^2)$ worst ..	57
7.5 Heap Sort - $O(n \log n)$, in place, not stable ..	58
7.6 Linear-Time (Non-Comparison) Sorts ..	58
7.7 Sorting Complex Objects ..	59
7.8 Common Mistakes ..	59
7.9 Applications ..	59
7.10 Interview Tips ..	60
7.11 Summary ..	60
Practice Set: Sorting ..	61
Beginner Problems ..	61
Intermediate Problems ..	61
Advanced Problems ..	61
Interview-Style Problems ..	61
Competitive Programming Problems ..	61
Challenge Exercises ..	61
Solutions: Sorting ..	62
Beginner Solutions ..	62
Intermediate Solutions ..	62
Advanced Solutions ..	62
Interview-Style Solutions ..	63
Competitive Programming Solutions ..	63
Challenge Solutions ..	63
RECURSION AND BACKTRACKING	
8. Recursion and Backtracking ..	65
8.1 Introduction ..	65
8.2 The Three Laws of Recursion ..	65
8.3 Factorial and Fibonacci - the hello world of recursion ..	65
8.4 The Call Stack - how recursion uses memory ..	65
8.5 Classic Recursive Patterns ..	66
8.6 Backtracking - recursion with undo ..	66
8.7 Analogy: Backtracking is a maze ..	67
8.8 Complexity of Backtracking ..	67
8.9 Common Mistakes ..	68

8.10 Applications ..	68
8.11 Interview Tips ..	68
8.12 Summary ..	68
Practice Set: Recursion and Backtracking ..	69
Beginner Problems ..	69
Intermediate Problems ..	69
Advanced Problems ..	69
Interview-Style Problems ..	69
Competitive Programming Problems ..	69
Challenge Exercises ..	69
Solutions: Recursion and Backtracking ..	70
Beginner Solutions ..	70
Intermediate Solutions ..	70
Advanced Solutions ..	70
Interview-Style Solutions ..	70
Competitive Programming Solutions ..	71
Challenge Solutions ..	71
 LINEAR DATA STRUCTURES	
9. Linked Lists ..	73
9.1 Introduction ..	73
9.2 Theory: The Cost Structure ..	73
9.3 Real-World Analogy ..	74
9.4 Implementing a Singly Linked List (C++17) ..	74
9.5 Reversal - the classic interview problem ..	75
9.6 Two Pointers on Linked Lists ..	76
9.7 Sorting and Merging Lists ..	76
9.8 Common Mistakes ..	77
9.9 Applications ..	77
9.10 Interview Tips ..	77
9.11 Summary ..	77
Practice Set: Linked Lists ..	78
Beginner Problems ..	78
Intermediate Problems ..	78
Advanced Problems ..	78
Interview-Style Problems ..	78
Competitive Programming Problems ..	78
Challenge Exercises ..	78
Solutions: Linked Lists ..	79
Beginner Solutions ..	79
Intermediate Solutions ..	79

Advanced Solutions ..	79
Interview-Style Solutions ..	80
Competitive Programming Solutions ..	80
Challenge Solutions ..	80
10. Stacks ..	81
10.1 Introduction ..	81
10.2 Theory: LIFO discipline ..	81
10.3 Real-World Analogy ..	81
10.4 Implementation on top of vector ..	81
10.5 Classic Problems with Stacks ..	82
10.6 Dry Run: balanced parentheses on "{[()]}" ..	83
10.7 Complexity ..	83
10.8 Common Mistakes ..	83
10.9 Applications ..	84
10.10 Interview Tips ..	84
10.11 Summary ..	84
Practice Set: Stacks ..	85
Beginner Problems ..	85
Intermediate Problems ..	85
Advanced Problems ..	85
Interview-Style Problems ..	85
Competitive Programming Problems ..	85
Challenge Exercises ..	85
Solutions: Stacks ..	86
Beginner Solutions ..	86
Intermediate Solutions ..	86
Advanced Solutions ..	86
Interview-Style Solutions ..	87
Competitive Programming Solutions ..	87
Challenge Solutions ..	87
11. Queues and Deques ..	88
11.1 Introduction ..	88
11.2 Theory: FIFO discipline ..	88
11.3 Real-World Analogy ..	88
11.4 Implementations ..	88
11.5 Breadth-First Search (BFS) - the queue's star algorithm ..	89
11.6 Sliding Window Maximum with a Deque - $O(n)$..	90
11.7 Dry Run: maxSlidingWindow on [1,3,-1,-3,5,3,6,7], $k = 3$..	90
11.8 Priority: What about priority_queue? ..	91
11.9 Complexity Summary ..	91
11.10 Common Mistakes ..	91

11.11 Applications ..	91
11.12 Interview Tips ..	91
11.13 Summary ..	91
Practice Set: Queues and Deques ..	92
Beginner Problems ..	92
Intermediate Problems ..	92
Advanced Problems ..	92
Interview-Style Problems ..	92
Competitive Programming Problems ..	92
Challenge Exercises ..	92
Solutions: Queues and Deques ..	93
Beginner Solutions ..	93
Intermediate Solutions ..	93
Advanced Solutions ..	94
Interview-Style Solutions ..	94
Competitive Programming Solutions ..	94
Challenge Solutions ..	94
HASH TABLES	
12. Hash Tables ..	96
12.1 Introduction ..	96
12.2 Theory: From Key to Address ..	96
12.3 Real-World Analogy ..	96
12.4 Collision Resolution ..	96
12.5 Load Factor and Rehashing ..	97
12.6 Hash Functions in C++ ..	97
12.7 Complexity Analysis ..	98
12.8 Applications ..	98
12.9 Common Mistakes ..	98
12.10 Interview Tips ..	99
12.11 Summary ..	99
Practice Set: Hash Tables ..	100
Beginner Problems ..	100
Intermediate Problems ..	100
Advanced Problems ..	100
Interview-Style Problems ..	100
Competitive Programming Problems ..	100
Challenge Exercises ..	100
Solutions: Hash Tables ..	101
Beginner Solutions ..	101
Intermediate Solutions ..	101

Advanced Solutions ..	101
Interview-Style Solutions ..	102
Competitive Programming Solutions ..	102
Challenge Solutions ..	102
TREES AND BINARY TREES	
13. Trees and Binary Trees ..	104
13.1 Introduction ..	104
13.2 Theory: Properties of Trees ..	104
13.3 Real-World Analogy ..	104
13.4 Representing Binary Trees in C++ ..	105
13.5 Traversals - visit every node exactly once ..	105
13.6 Level-by-Level Traversal (with depth) ..	106
13.7 Core Recursive Algorithms ..	106
13.8 Dry Run: inorder traversal of the tree above ..	107
13.9 Complexity ..	107
13.10 Applications ..	107
13.11 Common Mistakes ..	107
13.12 Interview Tips ..	107
13.13 Summary ..	107
Practice Set: Trees ..	109
Beginner Problems ..	109
Intermediate Problems ..	109
Advanced Problems ..	109
Interview-Style Problems ..	109
Competitive Programming Problems ..	109
Challenge Exercises ..	109
Solutions: Trees ..	110
Beginner Solutions ..	110
Intermediate Solutions ..	110
Advanced Solutions ..	110
Interview-Style Solutions ..	111
Competitive Programming Solutions ..	111
Challenge Solutions ..	111
14. Binary Search Trees ..	112
14.1 Introduction ..	112
14.2 Theory: Why Search Becomes $O(h)$..	112
14.3 Real-World Analogy ..	112
14.4 Implementation (C++17) ..	112
14.5 Operations Step by Step ..	113
14.6 Complexity Analysis ..	114

14.7 BST vs Other Structures ..	114
14.8 Applications ..	114
14.9 Common Mistakes ..	114
14.10 Interview Tips ..	115
14.11 Summary ..	115
Practice Set: Binary Search Trees ..	116
Beginner Problems ..	116
Intermediate Problems ..	116
Advanced Problems ..	116
Interview-Style Problems ..	116
Competitive Programming Problems ..	116
Challenge Exercises ..	116
Solutions: Binary Search Trees ..	117
Beginner Solutions ..	117
Intermediate Solutions ..	117
Interview-Style Solutions ..	117
Competitive Programming Solutions ..	118
Challenge Solutions ..	118
15. AVL Trees ..	119
15.1 Introduction ..	119
15.2 Theory: Height Bound ..	119
15.3 Real-World Analogy ..	119
15.4 Rotations - the two basic moves ..	119
15.5 Full Implementation (C++17) ..	120
15.6 Dry Run: insert 1,2,3,4 into an empty AVL tree ..	122
15.7 Complexity Analysis ..	122
15.8 Advantages and Disadvantages ..	122
15.9 Applications ..	123
15.10 Common Mistakes ..	123
15.11 Interview Tips ..	123
15.12 Summary ..	123
Practice Set: AVL Trees ..	124
Beginner Problems ..	124
Intermediate Problems ..	124
Advanced Problems ..	124
Interview-Style Problems ..	124
Competitive Programming Problems ..	124
Challenge Exercises ..	124
Solutions: AVL Trees ..	125
Beginner Solutions ..	125
Intermediate Solutions ..	125

Advanced Solutions ..	125
Interview-Style Solutions ..	126
Competitive Programming Solutions ..	126
Challenge Solutions ..	126
16. Heaps and Priority Queues ..	127
16.1 Introduction ..	127
16.2 Theory: Array Representation ..	127
16.3 Real-World Analogy ..	127
16.4 The Two Core Operations ..	128
16.5 Complete Min-Heap Implementation (C++17) ..	128
16.6 Building a Heap in $O(n)$..	129
16.7 Dry Run: push 4, 8, 2 into a min-heap ..	129
16.8 <code>std::priority_queue</code> ..	130
16.9 Complexity Analysis ..	130
16.10 Classic Heap Problems ..	130
16.11 Applications ..	131
16.12 Common Mistakes ..	131
16.13 Interview Tips ..	131
16.14 Summary ..	131
Practice Set: Heaps ..	133
Beginner Problems ..	133
Intermediate Problems ..	133
Advanced Problems ..	133
Interview-Style Problems ..	133
Competitive Programming Problems ..	133
Challenge Exercises ..	133
Solutions: Heaps ..	134
Beginner Solutions ..	134
Intermediate Solutions ..	134
Advanced Solutions ..	134
Interview-Style Solutions ..	135
Competitive Programming Solutions ..	135
Challenge Solutions ..	135
17. Tries ..	136
17.1 Introduction ..	136
17.2 Theory: Sharing Prefixes ..	136
17.3 Real-World Analogy ..	136
17.4 Implementation (C++17) ..	136
17.5 Operations and Complexity ..	137
17.6 Dry Run: insert "tea", then search "ted" in a trie ..	137
17.7 Trie Applications ..	138

17.8 Common Mistakes ..	138
17.9 Trie vs Alternatives ..	138
17.10 Interview Tips ..	138
17.11 Summary ..	138
Practice Set: Tries ..	139
Beginner Problems ..	139
Intermediate Problems ..	139
Advanced Problems ..	139
Interview-Style Problems ..	139
Competitive Programming Problems ..	139
Challenge Exercises ..	139
Solutions: Tries ..	140
Beginner Solutions ..	140
Intermediate Solutions ..	140
Advanced Solutions ..	140
Interview-Style Solutions ..	140
Competitive Programming Solutions ..	140
Challenge Solutions ..	141
REVISION 1	
Revision 1: C++ and Complexity Through Sorting ..	143
R1.1 One-Page Revision Sheet ..	143
R1.2 Important Formulas ..	143
R1.3 Cheat Sheet: Complexity of Common Operations ..	143
R1.4 Common Interview Patterns So Far ..	144
R1.5 Multiple-Choice Questions ..	144
R1.6 True / False Questions ..	144
R1.7 Short-Answer Questions ..	144
R1.8 Debugging Exercises ..	145
R1.9 Code Tracing Exercises ..	145
R1.10 Answers to Short/Debug/Trace ..	145
R1.11 What to Revise Before Moving On ..	146
R1.12 Progress Check ..	146
GRAPHS I	
18. Graphs: Representations and Fundamentals ..	148
18.1 Introduction ..	148
18.2 Theory: Representations ..	148
18.3 Real-World Analogy ..	149
18.4 Why the representation matters ..	149
18.5 Common Mistakes ..	149
18.6 Applications ..	149

18.7 Interview Tips ..	149
18.8 Summary ..	150
19. Graph Traversal: BFS and DFS ..	151
19.1 Introduction ..	151
19.2 DFS - recursion (or explicit stack) ..	151
19.3 BFS - the queue ..	151
19.4 Connected Components ..	152
19.5 Cycle Detection ..	152
19.6 Bipartite Graph Check ..	152
19.7 Flood Fill and Grid BFS ..	153
19.8 Complexity ..	153
19.9 Common Mistakes ..	153
19.10 Applications ..	153
19.11 Interview Tips ..	153
19.12 Summary ..	153
Practice Set: Graphs, BFS and DFS ..	155
Beginner Problems ..	155
Intermediate Problems ..	155
Advanced Problems ..	155
Interview-Style Problems ..	155
Competitive Programming Problems ..	155
Challenge Exercises ..	155
Solutions: Graphs, BFS and DFS ..	156
Beginner Solutions ..	156
Intermediate Solutions ..	156
Advanced Solutions ..	156
Interview-Style Solutions ..	156
Competitive Programming Solutions ..	156
Challenge Solutions ..	156
20. Shortest Path Algorithms ..	158
20.1 Introduction ..	158
20.2 Dijkstra's Algorithm - $O((V+E) \log V)$..	158
20.3 Dry Run: Dijkstra on the graph below from A ..	158
20.4 Bellman-Ford - $O(V \cdot E)$, handles negatives ..	159
20.5 Floyd-Warshall - all pairs, $O(V^3)$..	159
20.6 Shortest Path in a DAG - $O(V + E)$ via topological order ..	160
20.7 Which algorithm when? ..	160
20.8 Real-World Analogy ..	160
20.9 Common Mistakes ..	160
20.10 Applications ..	160
20.11 Interview Tips ..	160

20.12 Summary ..	161
Practice Set: Shortest Paths ..	162
Beginner Problems ..	162
Intermediate Problems ..	162
Advanced Problems ..	162
Interview-Style Problems ..	162
Competitive Programming Problems ..	162
Challenge Exercises ..	162
Solutions: Shortest Paths ..	163
Beginner Solutions ..	163
Intermediate Solutions ..	163
Advanced Solutions ..	163
Interview-Style Solutions ..	163
Competitive Programming Solutions ..	163
Challenge Solutions ..	164
21. Minimum Spanning Trees and Union-Find ..	165
21.1 Introduction ..	165
21.2 Union-Find (Disjoint Set Union) ..	165
21.3 Dry Run: DSU ..	165
21.4 Kruskal's Algorithm - $O(E \log E)$..	165
21.5 Prim's Algorithm - $O((V+E) \log V)$..	166
21.6 MST Properties ..	166
21.7 Real-World Analogy ..	167
21.8 Applications ..	167
21.9 Common Mistakes ..	167
20.? Interview Tips ..	167
21.10 Summary ..	167
Practice Set: MST and Union-Find ..	168
Beginner Problems ..	168
Intermediate Problems ..	168
Advanced Problems ..	168
Interview-Style Problems ..	168
Competitive Programming Problems ..	168
Challenge Exercises ..	168
Solutions: MST and Union-Find ..	169
Beginner Solutions ..	169
Intermediate Solutions ..	169
Advanced Solutions ..	169
Interview-Style Solutions ..	169
Competitive Programming Solutions ..	169
Challenge Solutions ..	169

22. Topological Sorting ..	171
22.1 Introduction ..	171
22.2 Theory: When is it possible? ..	171
22.3 Kahn's Algorithm - $O(V + E)$..	171
22.4 Dry Run: Kahn's ..	171
22.5 DFS-based Topological Sort ..	172
22.6 Longest Path in a DAG (via topo order) ..	172
22.7 Real-World Analogy ..	172
22.8 Applications ..	172
22.9 Common Mistakes ..	173
22.10 Interview Tips ..	173
22.11 Summary ..	173
Practice Set: Topological Sorting ..	174
Beginner Problems ..	174
Intermediate Problems ..	174
Advanced Problems ..	174
Interview-Style Problems ..	174
Competitive Programming Problems ..	174
Challenge Exercises ..	174
Solutions: Topological Sorting ..	175
Beginner Solutions ..	175
Intermediate Solutions ..	175
Advanced Solutions ..	175
Interview-Style Solutions ..	175
Competitive Programming Solutions ..	175
Challenge Solutions ..	176
23. Advanced Graph Algorithms ..	177
23.1 Introduction ..	177
23.2 Strongly Connected Components - Kosaraju's Algorithm ..	177
23.3 Bridges and Articulation Points - Tarjan's Algorithm ..	178
23.4 Dry Run: Bridges ..	178
23.5 Euler Paths and Circuits ..	178
23.6 Real-World Analogy ..	179
23.7 Applications ..	180
23.8 Common Mistakes ..	180
23.9 Interview Tips ..	180
23.10 Summary ..	180
Practice Set: Advanced Graph Algorithms ..	181
Beginner Problems ..	181
Intermediate Problems ..	181
Advanced Problems ..	181

Interview-Style Problems ..	181
Competitive Programming Problems ..	181
Challenge Exercises ..	181
Solutions: Advanced Graph Algorithms ..	182
Beginner Solutions ..	182
Intermediate Solutions ..	182
Advanced Solutions ..	182
Interview-Style Solutions ..	182
Competitive Programming Solutions ..	182
Challenge Solutions ..	182
ALGORITHMS II	
24. Greedy Algorithms ..	185
24.1 Introduction ..	185
24.2 Theory: Exchange Argument ..	185
24.3 Classic Greedy Problems ..	185
24.4 Dry Run: Activity Selection ..	186
24.5 When Greedy Fails - and what to do ..	186
24.6 Real-World Analogy ..	186
24.7 Applications ..	186
24.8 Common Mistakes ..	187
24.9 Interview Tips ..	187
24.10 Summary ..	187
Practice Set: Greedy Algorithms ..	188
Beginner Problems ..	188
Intermediate Problems ..	188
Advanced Problems ..	188
Interview-Style Problems ..	188
Competitive Programming Problems ..	188
Challenge Exercises ..	188
Solutions: Greedy Algorithms ..	189
Beginner Solutions ..	189
Intermediate Solutions ..	189
Advanced Solutions ..	189
Interview-Style Solutions ..	190
Competitive Programming Solutions ..	190
Challenge Solutions ..	190
25. Divide and Conquer ..	191
25.1 Introduction ..	191
25.2 The D&C template ..	191
25.3 Merge Sort - $O(n \log n)$..	191

25.4 Quicksort - $O(n \log n)$ average, $O(n^2)$ worst ..	191
25.5 Counting Inversions (with merge sort) ..	192
25.6 Maximum Subarray Sum - $O(n)$ via D&C or Kadane ..	192
25.7 Other D&C classics ..	193
25.8 Real-World Analogy ..	193
25.9 Common Mistakes ..	193
25.10 Applications ..	193
25.11 Interview Tips ..	193
25.12 Summary ..	193
Practice Set: Divide and Conquer ..	195
Beginner Problems ..	195
Intermediate Problems ..	195
Advanced Problems ..	195
Interview-Style Problems ..	195
Competitive Programming Problems ..	195
Challenge Exercises ..	195
Solutions: Divide and Conquer ..	196
Beginner Solutions ..	196
Intermediate Solutions ..	196
Advanced Solutions ..	196
Interview-Style Solutions ..	196
Competitive Programming Solutions ..	196
Challenge Solutions ..	197
26. Dynamic Programming ..	198
26.1 Introduction ..	198
26.2 The 5-step DP recipe ..	198
26.3 Fibonacci - the canonical example ..	198
26.4 1D DP examples ..	199
26.5 2D DP examples ..	199
26.6 Memoisation: the same language ..	200
26.7 State-space explosions and when to stop ..	200
26.8 Common DP patterns worth knowing ..	201
26.9 Real-World Analogy ..	201
26.10 Common Mistakes ..	201
26.11 Applications ..	201
26.12 Interview Tips ..	201
26.13 Summary ..	202
Practice Set: Dynamic Programming ..	203
Beginner Problems ..	203
Intermediate Problems ..	203
Advanced Problems ..	203

Interview-Style Problems ..	203
Competitive Programming Problems ..	203
Challenge Exercises ..	203
Solutions: Dynamic Programming ..	204
Beginner Solutions ..	204
Intermediate Solutions ..	204
Advanced Solutions ..	204
Interview-Style Solutions ..	204
Competitive Programming Solutions ..	204
Challenge Solutions ..	205
27. Bit Manipulation ..	206
27.1 Introduction ..	206
27.2 The operators ..	206
27.3 The essential trick sheet ..	206
27.4 Classic problems ..	206
27.5 The `__builtin` toolbox (gcc/clang) ..	208
27.6 Real-World Analogy ..	208
27.7 Applications ..	208
27.8 Common Mistakes ..	208
27.9 Interview Tips ..	208
27.10 Summary ..	209
Practice Set: Bit Manipulation ..	210
Beginner Problems ..	210
Intermediate Problems ..	210
Advanced Problems ..	210
Interview-Style Problems ..	210
Competitive Programming Problems ..	210
Challenge Exercises ..	210
Solutions: Bit Manipulation ..	211
Beginner Solutions ..	211
Intermediate Solutions ..	211
Advanced Solutions ..	211
Interview-Style Solutions ..	211
Competitive Programming Solutions ..	212
Challenge Solutions ..	212
ADVANCED DATA STRUCTURES	
28. Segment Tree ..	214
28.1 Introduction ..	214
28.2 Theory: The structure ..	214
28.3 Implementation (iterative - simpler and fast) ..	214

28.4 Recursive segment tree with lazy propagation ..	215
28.5 Variants ..	216
28.6 Dry Run: iterative query [1, 4) on the tree above ..	216
28.7 Real-World Analogy ..	216
28.8 Applications ..	216
28.9 Common Mistakes ..	216
28.10 Interview Tips ..	217
28.11 Summary ..	217
29. Fenwick Tree (Binary Indexed Tree) ..	218
29.1 Introduction ..	218
29.2 Theory: The key idea ..	218
29.3 Implementation ..	218
29.4 Dry Run: sum(7) ..	218
29.5 Fenwick vs Segment Tree ..	218
29.6 Range update + range query (difference trick) ..	219
29.7 Real-World Analogy ..	219
29.8 Common Mistakes ..	219
29.9 Applications ..	219
29.10 Interview Tips ..	219
29.11 Summary ..	219
30. Sparse Table ..	220
30.1 Introduction ..	220
30.2 Theory: The structure ..	220
30.3 Dry Run: RMQ on [1, 3] of [9, 2, 5, 4] ..	220
30.4 Comparisons ..	220
30.5 Real-World Analogy ..	221
30.6 Common Mistakes ..	221
30.7 Applications ..	221
30.8 Interview Tips ..	221
30.9 Summary ..	221
Practice Set: Segment Tree, Fenwick, Sparse Table ..	222
Beginner Problems ..	222
Intermediate Problems ..	222
Advanced Problems ..	222
Interview-Style Problems ..	222
Competitive Programming Problems ..	222
Challenge Exercises ..	222
Solutions: Segment Tree, Fenwick, Sparse Table ..	223
Beginner Solutions ..	223
Intermediate Solutions ..	223
Advanced Solutions ..	223

Interview-Style Solutions ..	223
Competitive Programming Solutions ..	223
Challenge Solutions ..	224
STRING ALGORITHMS	
31. String Searching: KMP, Rabin-Karp, Z-algorithm ..	226
31.1 Introduction ..	226
31.2 Theory: the failure function (KMP) ..	226
31.3 Dry Run: prefix function of "ababc" ..	226
31.4 Rabin-Karp - rolling hash ..	227
31.5 Z-algorithm ..	227
31.6 Dry Run: Z of "aaaaa" ..	228
31.7 What to use when ..	228
31.8 Real-World Analogy ..	228
31.9 Common Mistakes ..	228
31.10 Applications ..	228
31.11 Interview Tips ..	229
31.12 Summary ..	229
32. Advanced String Techniques ..	230
32.1 Manacher's Algorithm - palindromes in $O(n)$..	230
32.2 Suffix array and LCP (brief) ..	230
32.3 String hashing for substring equality ..	230
32.4 Trie applications recap (Chapter 17) ..	230
32.5 Aho-Corasick (multi-pattern, brief) ..	230
32.6 Real-World Analogy ..	230
32.7 Common Mistakes ..	231
32.8 Applications ..	231
32.9 Interview Tips ..	231
32.10 Summary ..	231
Practice Set: String Algorithms ..	232
Beginner Problems ..	232
Intermediate Problems ..	232
Advanced Problems ..	232
Interview-Style Problems ..	232
Competitive Programming Problems ..	232
Challenge Exercises ..	232
Solutions: String Algorithms ..	233
Beginner Solutions ..	233
Intermediate Solutions ..	233
Advanced Solutions ..	233
Interview-Style Solutions ..	233

Competitive Programming Solutions ..	233
Challenge Solutions ..	233
GEOMETRY AND ADVANCED TECHNIQUES	
33. Computational Geometry Basics ..	236
33.1 Introduction ..	236
33.2 Points and vectors ..	236
33.3 The cross product - the workhorse ..	236
33.4 Convex hull - Andrew's monotone chain, $O(n \log n)$..	236
33.5 Polygon area (shoelace formula) ..	237
33.6 Segment intersection ..	237
33.7 Closest pair of points - $O(n \log n)$..	237
33.8 Real-World Analogy ..	237
33.9 Common Mistakes ..	238
33.10 Applications ..	238
33.11 Interview Tips ..	238
33.12 Summary ..	238
34. Advanced Techniques and Optimization ..	239
34.1 Introduction ..	239
34.2 Coordinate compression ..	239
34.3 Binary search on the answer ..	239
34.4 Mo's algorithm - offline range queries ..	239
34.5 Meet in the middle ..	240
34.6 The sliding window formalised ..	240
34.7 Offline processing and event sweep ..	240
34.8 Real-World Analogy ..	240
34.9 Common Mistakes ..	240
34.10 Applications ..	241
34.11 Interview Tips ..	241
34.12 Summary ..	241
Practice Set: Geometry and Advanced Techniques ..	242
Beginner Problems ..	242
Intermediate Problems ..	242
Advanced Problems ..	242
Interview-Style Problems ..	242
Competitive Programming Problems ..	242
Challenge Exercises ..	242
Solutions: Geometry and Advanced Techniques ..	243
Beginner Solutions ..	243
Intermediate Solutions ..	243
Advanced Solutions ..	243

Interview-Style Solutions ..	243
Competitive Programming Solutions ..	243
Challenge Solutions ..	244
REVISION 2	
Revision 2: Trees, Graphs, and Algorithm Design ..	246
R2.1 One-Page Revision Sheet ..	246
R2.2 Complexity Cheat Sheet ..	246
R2.3 Key Patterns in Algorithm Design ..	247
R2.4 Multiple-Choice Questions ..	247
R2.5 True / False Questions ..	247
R2.6 Short-Answer Questions ..	248
R2.7 Debugging Exercises ..	248
R2.8 Code Tracing Exercises ..	248
R2.9 Answers to Short/Debug/Trace ..	248
R2.10 What to Revise Before Moving On ..	249
R2.11 Progress Check ..	249
BACK MATTER	
Appendix A: Complete STL Reference ..	251
A.1 Containers at a Glance ..	251
A.2 Algorithms (all in <algorithm>) ..	251
A.3 Numerics and Utilities ..	251
A.4 String API ..	252
A.5 Custom Comparators ..	252
A.6 I/O Speedups ..	252
A.7 Common Pitfalls ..	252
Appendix B: Big-O Cheat Sheet ..	253
B.1 Growth Rates (slow to fast) ..	253
B.2 Operations Summary ..	253
B.3 Space Summary ..	254
B.4 Time-limit mental math ..	254
Appendix C: Frequently Asked Questions ..	255
C.1 Which is faster: unordered_map or map? ..	255
C.2 When is BFS better than DFS? ..	255
C.3 What is the difference between top-down and bottom-up DP? ..	255
C.4 How do I know if greedy is safe? ..	255
C.5 Why is the knapsack inner loop descending? ..	255
C.6 What are "idempotent" operations for sparse tables? ..	255
C.7 Should I use recursion or iteration for tree traversals? ..	255
C.8 How do I handle 1-based vs 0-based indexing? ..	255
C.9 What does "O(n log n)" mean practically for sorting? ..	256

C.10 How do I get faster at these problems? ..	256
Appendix D: The DSA Roadmap ..	257
D.1 Phase 1 - Foundations (weeks 1-2) ..	257
D.2 Phase 2 - Core structures (weeks 3-5) ..	257
D.3 Phase 3 - Trees and advanced structures (weeks 6-8) ..	257
D.4 Phase 4 - Graphs (weeks 9-11) ..	257
D.5 Phase 5 - Algorithm design (weeks 12-15) ..	257
D.6 Phase 6 - Specialized topics (weeks 16-18) ..	257
D.7 Phase 7 - Interview prep (weeks 19+) ..	257
D.8 Golden advice ..	257
Appendix E: Recommended Problem List ..	258
E.1 Must-do (in order) ..	258
E.2 Next tier (interviews) ..	259
E.3 Competitive programming tier ..	259
E.4 How to use this list ..	259
Appendix F: Glossary ..	260
Closing Words ..	261

MODULE

Foundations of C++ for DSA

C++ syntax, memory model and the Standard Template Library

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

1. C++ Fundamentals for Data Structures

1.1 Welcome and How to Use This Book

Welcome! This book will take you from a beginner who knows nothing about data structures and algorithms to a level where you can solve hard coding problems confidently. We use **C++17** throughout, because C++ is the language of choice for competitive programming and most coding interviews.

Every chapter follows the same structure:

- **Introduction** and **Theory** with beginner-friendly explanations.
- A **real-world analogy** to make the idea intuitive.
- **Visual explanations** with tables and ASCII diagrams.
- **Operations** explained step by step.
- **Complexity analysis** (time and space).
- **Advantages, disadvantages, applications** and **common mistakes**.
- Complete **C++17 code** with a line-by-line **dry run**.
- **Interview tips** and a **summary**.
- A **practice set**: 5 beginner + 5 intermediate + 5 advanced + 2 interview + 2 competitive-programming problems + challenge exercises.
- **Full solutions** at the end of the chapter.

Study workflow

1. Read the theory and the code.
2. Trace the dry run yourself on paper.
3. Try the beginner problems **before** reading the solutions.
4. Re-read the summary and move on; come back for revision sheets later.

1.2 Variables, Types and I/O

Fundamental Types

C++ provides a set of built-in types. The most important ones for DSA are:

Type	Size (typical)	Range / Use
`int`	4 bytes	roughly -2×10^9 .. 2×10^9 (use `long long` for larger)
`long long`	8 bytes	roughly -9×10^{18} .. 9×10^{18}
`double`	8 bytes	floating point, ~15 significant digits
`bool`	1 byte	`true` / `false`
`char`	1 byte	a single ASCII character
`size_t`	8 bytes	unsigned, used for indices and sizes

Integer overflow

int maxes out near 2.1 billion. Multiplying two 10-digit numbers overflows. In DSA and competitive programming, when in doubt, use `long long`.

Input / Output

Fast I/O for competitive programming

Add these two lines at the start of `main()` to make `cin/cout` much faster:

Do not mix C-style `printf/scanf` with `cin/cout` when these are enabled.

1.3 Functions, References and Pointers

Pass by Value, Reference and Pointer

Pointers vs references

A **pointer** is a variable that stores the address of another variable. It can be `nullptr` and can be reassigned. A **reference** is an alias (another name) for an existing variable; it can never be null and cannot be "reseated". In modern C++ you should prefer references and smart pointers over raw pointers.

`const` and `auto`

1.4 Classes and Structures

A struct/class groups data and the functions that operate on it.

Member initializer list

Always initialise members in the *initializer list* : $x(a)$, $y(b)$ rather than assigning inside the constructor body. It is faster and works for const members.

1.5 Templates: Write Once, Use with Any Type

1.6 The Standard Template Library in One Page

The STL provides ready-made containers and algorithms. You will use these in every single chapter of this book.

Container	Ordering	Typical operations
<code>`vector<T>`</code>	dynamic array	push/pop back, random access
<code>`list<T>`</code>	doubly linked list	insert/erase anywhere
<code>`deque<T>`</code>	double-ended queue	push/pop both ends
<code>`stack<T>`</code>	LIFO	push/pop/top
<code>`queue<T>`</code>	FIFO	push/pop/front
<code>`priority_queue<T>`</code>	max-heap	push/pop/top
<code>`set<T>`</code> , <code>`map<K,V>`</code>	sorted tree	log n insert/find
<code>`unordered_set`</code> , <code>`unordered_map`</code>	hash table	O(1) average

1.7 Complexity Preview

We measure how fast an algorithm is using **Big-O notation**:

- $O(1)$ constant, $O(\log n)$ logarithmic, $O(n)$ linear, $O(n \log n)$, $O(n^2)$ quadratic.

Growth order

$1 < \log n < n < n \log n < n^2 < 2^n < n!$ Faster on the left, slower on the right. If $n = 10^6$, $O(n \log n)$ is fine but $O(n^2)$ is hopeless.

1.8 Interview Tips

- Always ask about the **input size** before choosing an algorithm; the input size dictates the acceptable complexity.
- Prefer `long long` for any arithmetic that might overflow.
- Use `const` and references aggressively; it prevents bugs and improves speed.
- Write clean, idiomatic C++ using the STL; interviewers read style, not just logic.

1.9 Summary

- C++ gives you low-level power (pointers, memory control) and high-level safety (STL).
 - Master `vector`, `map/set`, `stack`, `queue` and `priority_queue` first.
 - Always track complexity; know your growth orders.
 - Practice the workflow: theory -> code -> dry run -> problems.
-

Practice Set: C++ Fundamentals

Beginner Problems

1. Read N integers and print their average with two decimal places.
2. Given a Point struct with x, y, write a function returning the Euclidean distance.
3. Write a template function `repeatPrint(const T&, int k)` that prints a value k times.
4. Read a string and print its reverse.
5. Swap two integers using references (do not write your own swap).

Intermediate Problems

1. Write a function that returns the largest and second largest of N numbers using references.
2. Convert a decimal number to binary and count the number of 1 bits.
3. Given an array, print every pair (i, j) with $i < j$ and $a[i] + a[j] == K$.
4. Implement a small struct `Rational` with add and simplify methods.
5. Read a matrix and print its transpose.

Advanced Problems

1. Implement a generic function `filter(const vector<T>&, bool(*pred)(const T&))`.
2. Write a class `Counter` with overloaded operator()(int) that counts calls per value.
3. Implement `my_swap` using XOR for integers and explain when it fails.
4. Write a recursive template that computes the sum of a `vector<T>`.
5. Given n points, find the pair with minimum distance using brute force ($O(n^2)$).

Interview-Style Problems

1. Implement a thread-safe counter using `std::atomic` (or `mutex`) and explain the cost.
2. Given two large numbers as strings, add them without using a big-int library.

Competitive Programming Problems

1. Codeforces-style: read T test cases, each with two integers a, b; print a+b.
2. Print all primes up to N using the sieve (use a `vector<bool>`).

Challenge Exercises

- Write a variadic template function `sumOf(...)` that accepts any number of arguments of any numeric types and returns their sum.
 - Benchmark `vector::push_back` vs manual dynamic array growth; explain reallocation.
-

Solutions: C++ Fundamentals

Beginner Solutions

1. Read N, accumulate with `long long`, print `sum / (double)N` with `cout << fixed << setprecision(2)`.
2. `sqrt(dx*dx + dy*dy)`; note `dx` must be `double` or cast.
3. Loop `k` times; the template deduces the type from the argument.
4. Print characters from `s.size() - 1` down to 0 (mind the unsigned underflow: use `int i = (int)s.size() - 1`).
5. Pass by reference; the change is visible in the caller.

Intermediate Solutions

1. Track `mx` and `second`; update carefully when `a[i] > mx`.
2. Loop while `n > 0`, `ans += n % 2`; `n /= 2`; or use `__builtin_popcountll`.
3. Double loop $O(n^2)$ checking `a[i] + a[j] == K`.
4. Store numerator and denominator; simplify by dividing by `gcd` (use `std::gcd` from `<numeric>`).
5. Output `a[j][i]` for all `i, j` ($O(nm)$).

Advanced Solutions

1. Loop and call the predicate; return a new vector.
2. Store `unordered_map<int, int> cnt` and return the count.
3. XOR swap: `a^=b`; `b^=a`; `a^=b`; fails for objects and is slower than a temp; for integers it is correct but pointless in practice.
4. Specialise for size 0/1 or recurse on a subrange; a variadic template is cleaner.
5. Two nested loops keeping the best distance.

Interview-Style Solutions

1. Use `std::atomic<int>` with `fetch_add`; the cost is a hardware lock-free op (~ 1 ns).
2. Add digit by digit from the right, carrying a carry; $O(\max(n,m))$.

Competitive Programming Solutions

1. Straight loop; read `t`, loop, print each `a+b`.
2. Sieve of Eratosthenes in $O(n \log \log n)$.

Challenge Solutions

- Variadic: `template<typename T> T sum0f(T a){return a;} template<typename T, typename...Rest> auto sum0f(T a, Rest...r){return a+sum0f(r...);}`
- `vector::push_back` grows capacity by a factor (commonly 2), so `push_back` is amortized $O(1)$; a naive `realloc` that grows by 1 is $O(n^2)$ overall.

2. STL Essentials: Containers, Iterators, Algorithms

2.1 Introduction

The Standard Template Library (STL) is the toolbox every C++ programmer reaches for. It gives you ready-made, battle-tested implementations of the most common data structures (containers), ways to walk through them (iterators), generic algorithms (sort, find, lower_bound, ...) and functional tools (lambdas, pair, tuple). In DSA interviews and competitive programming, you will write STL code on every single problem, so investing a chapter here pays off immediately.

Why the STL matters for DSA

- You avoid re-implementing and re-debugging standard structures.
- STL implementations are extremely fast (optimised by compiler vendors for years).
- Learning the *interfaces* first makes the hand-written data structures in later chapters much easier to understand - you already know the API contract.

2.2 Vector - the Workhorse Dynamic Array

`vector<T>` is a dynamically sized array stored contiguously in memory. Random access is $O(1)$; adding at the end is amortized $O(1)$.

Common operations

Index type

`v.size()` returns `size_t` (unsigned). Writing `for (int i = 0; i < v.size(); i++)` compares `int` with `size_t` and warns. Prefer `for (int i = 0; i < (int)v.size(); ++i)` or use `size_t i`. Never write `i >= 0` with `size_t` - it wraps around!

Capacity and reallocation

A vector grows by allocating a *larger* buffer and copying. Growth is geometric (typically doubling), so `push_back` is $O(1)$ amortized.

Operation	Time	Notes
<code>push_back</code>	$O(1)$ amortized	may reallocate
<code>pop_back</code>	$O(1)$	
<code>operator[]</code>	$O(1)$	no bounds check
<code>at()</code>	$O(1)$	throws <code>std::out_of_range</code>
insert/erase middle	$O(n)$	shifts elements
size/capacity	$O(1)$	

2.3 List, Deque, and the Container Adapters

list (doubly linked list)

`list<T>` allows $O(1)$ insert/erase *anywhere* once you have an iterator, but has no random access and terrible cache locality. Use it rarely; vector is usually faster even when you "need" insertion.

deque (double-ended queue)

`deque<T>` gives $O(1)$ push/pop at *both* ends plus $O(1)$ random access. It is the default backing container for stack and queue.

stack, queue, priority_queue

These are **adapters**: they wrap an underlying container and expose only a restricted interface.

priority_queue gotchas

- `priority_queue<T>` is a **max-heap** by default. For a min-heap use `greater<T>` or push negatives.
- There is **no way to iterate** a `priority_queue` or `stack`.
- `pop()` returns `void`; you must read `top()` before popping.

2.4 set, map, multiset, multimap

The ordered associative containers are balanced binary search trees (red-black trees) with $O(\log n)$ insert/find/erase.

map vs unordered_map

- map/set: sorted, $O(\log n)$, needs only `<` comparator.
- unordered_map/unordered_set: hash based, $O(1)$ average, needs `std::hash`.
- Use unordered when order does not matter; use ordered when you need sorted iteration, `lower_bound`, or when a good hash is unavailable.

2.5 pair, tuple and structured bindings

2.6 Algorithms

`<algorithm>` contains the most used generic functions.

lower_bound / upper_bound

For a **sorted** range: - `lower_bound(first, last, x)` returns the first position where `x` could be inserted, i.e. first element $\geq x$. - `upper_bound(first, last, x)` returns the first element $> x$. Together they find the range of equal elements: `[lower_bound, upper_bound)`.

2.7 Lambdas

A lambda is an anonymous function object. They are essential for custom comparators.

Comparator contract

A comparator `comp(a, b)` must be a **strict weak ordering**: it returns true if `a` should come *before* `b`. For sorting descending use `a > b`; for ascending `a < b`. The default `less<T>` sorts ascending.

2.8 Custom structs in containers

To put your own type into `set/map/priority_queue/sort`, provide `operator<` (for ordered) or a custom comparator/hash (for unordered).

2.9 Complexity Cheat Sheet for Common Operations

Operation	vector	list	deque	set/map	unordered	priority_queue
push front	-	O(1)	O(1)	-	-	-
push back	O(1)*	O(1)	O(1)	-	-	-
pop front	-	O(1)	O(1)	-	-	-
access first/last	O(1)	O(1)	O(1)	-	-	-

Operation	vector	list	deque	set/map	unordered	priority_queue
access index	O(1)	O(n)	O(1)	-	-	-
insert	O(n)	O(1)	O(n)	O(log n)	O(1)*	O(log n)
erase	O(n)	O(1)	O(n)	O(log n)	O(1)*	O(log n)
find	O(n)	O(n)	O(n)	O(log n)	O(1)*	O(n)
top/min/max	-	-	-	O(1)	-	O(1)

- amortized / average case.

2.10 Common Mistakes

- erase while iterating: `for (auto it = s.begin(); it != s.end(); ) { if (*it==x) it = s.erase(it); else ++it; }` (in C++20 use `erase_if`).
- `ms.erase(x)` erases **all** copies in a multiset; erase find result to remove one.
- Forgetting `greater<int>` makes `priority_queue` a max-heap, not min-heap.
- `binary_search` on an *unordered* range - undefined behaviour.
- Modifying a set key via `*it` is UB; erase + reinsert instead.
- `it++` vs `it++` invalidation: after `vector::push_back` a reallocation invalidates iterators.

2.11 Interview Tips

- Say the complexity of every STL call you make out loud; interviewers listen for it.
- Prefer `std::array/vector` over raw arrays + `new`.
- Use auto and structured bindings - idiomatic and safe.
- Know when *not* to use the STL: if the interviewer wants a manual implementation of a linked list or a hash map, write it from scratch.

2.12 Summary

- vector is the default container; learn its capacity behaviour.
- set/map are balanced BSTs; `unordered_*` are hash tables.
- stack/queue/priority_queue are adapters with minimal interfaces.
- Lambdas + `<algorithm>` make generic programming concise.
- Always state the complexity of the operations you use.

Practice Set: STL Essentials

Beginner Problems

1. Read N numbers, print them sorted ascending and then descending.
2. Count the frequency of each word in a sentence using `map<string,int>`.
3. Read N numbers; print the second largest distinct value using a `set`.
4. Use a stack to check that parentheses in a string are balanced.
5. Merge two sorted `vector<int>` into one sorted vector (no STL sort).

Intermediate Problems

1. Given an array, find the count of distinct elements using a `unordered_set`.
2. Print the top K most frequent numbers in $O(n \log n)$ using a `map` + `vector` of pairs.
3. Use `lower_bound` to answer "how many elements are $\leq X$ " for many queries.
4. Implement a simple LRU cache using `unordered_map` + `list`.
5. Sort an array of strings by length, ties broken alphabetically.

Advanced Problems

1. Implement a MedianFinder using two `priority_queues` (min/max heap).
2. Use `next_permutation` to print all permutations of a string in lexicographic order.
3. Given queries of form (L,R), answer range sums with a prefix-sum array.
4. Use a `deque` to implement a sliding-window maximum in $O(n)$.
5. Write a generic `my_sort` that works on any random-access container using iterators and a comparator.

Interview-Style Problems

1. Find the K largest elements in an array without sorting everything (use a min-heap of size K).
2. "Design a data structure that supports insert, delete, search, and getRandom in $O(1)$."

Competitive Programming Problems

1. For each element in an array, print the index of the *next greater element* (use a stack) - classic NGE problem.
2. Given a stream of integers, keep the running median using two heaps.

Challenge Exercises

- Implement `std::accumulate`, `std::transform` and `std::remove_if` yourself.
 - Write a compile-time (`constexpr`) function that returns the sum of a `std::array` at compile time.
-

Solutions: STL Essentials

Beginner Solutions

1. `sort(v.begin(), v.end());` then `sort(v.rbegin(), v.rend());`;
2. `for (auto& w : words) freq[w]++;`
3. Insert all into a set, take the last two elements (or store in a vector and sort).
4. On (, push; on), if stack empty -> false else pop; at end check empty.

1. Two-pointer merge: advance the pointer with the smaller value, append to result.

Intermediate Solutions

1. `unordered_set<int> d(a.begin(), a.end()); return d.size();`
2. Count with a map, copy to `vector<pair<int, int>>`, sort by count descending, take first K.
3. Sort the array once; each query X is answered with `upper_bound(a.begin(), a.end(), X) - a.begin()` in $O(\log n)$.
4. On get: if present, move to front of list; on put: insert at front, evict from back.

1. `sort(v.begin(), v.end(), [](const string&a, const string&b){ return a.size() != b.size() ? a.size() < b.size() : a < b; });`

Advanced Solutions

1. Max-heap holds the smaller half, min-heap the larger half; rebalance so sizes differ by at most 1. Median is the top of the larger heap (or average of tops).
2. `do { print; } while (next_permutation(s.begin(), s.end()));` - starts from the lexicographically smallest permutation, so sort first.
3. `pref[i] = pref[i-1] + a[i-1]; answer = pref[R] - pref[L-1].`

4. Maintain a deque of *candidate indices* in decreasing value order; push from back while `a[dq.back()] <= a[i]`; pop front while out of window.

1. Use template `<typename RanIt, typename Cmp>` and `*it` swaps inside a loop.

Interview-Style Solutions

1. Keep a min-heap of size K: for each `x`, if `heap.size() < K` push, else if `x > heap.top()` pop+push. After the scan, the heap holds the K largest; `top()` is the K-th largest.
2. vector for values + `unordered_map<value, index>`; `getRandom = v[rand()%v.size()]`; swap-with-last on erase to keep $O(1)$.

Competitive Programming Solutions

1. NGE with a stack: iterate left-to-right, while stack top value `< a[i]`, the answer for stack top is `i`; pop and record; push `i`. Remaining elements in the stack have no greater element.
2. Running median = two heaps (see #11) but updated as numbers arrive.

Challenge Solutions

- `accumulate`: fold with `init + *it`. `transform`: `*out = f(*in)`. `remove_if`: partition non-matching elements forward and return the new end.
- Mark a `constexpr` function and call it with a `constexpr` array; the compiler computes it at compile time.

MODULE

Complexity Analysis

Measuring algorithms: Big-O, Omega, Theta, master theorem and amortized analysis

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

3. Time and Space Complexity

3.1 Introduction

Before we study a single algorithm, we need a language for describing how fast it runs. Complexity analysis answers the question: **"If the input is n times bigger, how many more steps does the algorithm take?"** It lets us compare algorithms without running them, predict behaviour on huge inputs, and decide - before writing any code - whether a solution will finish in time.

Big-O notation

Big-O describes the *upper bound* of growth. We say an algorithm is **$O(f(n))$** if the number of steps is at most $c * f(n)$ for some constant c and all sufficiently large n (beyond some n_0). We drop constant factors and lower-order terms: $3n^2 + 5n + 7$ is $O(n^2)$.

3.2 The Formal Zoo: O, Omega, Theta

Notation	Meaning	Intuition
$O(f(n))$	grows no faster than $f(n)$	upper bound
$\Omega(f(n))$	grows no slower than $f(n)$	lower bound
$\Theta(f(n))$	grows exactly like $f(n)$	tight bound
$o(f(n))$	strictly slower than $f(n)$	strict upper
$\omega(f(n))$	strictly faster than $f(n)$	strict lower

For most interview discussions you only need O . A precise statement like "quicksort is $O(n^2)$ worst case and $\Theta(n \log n)$ average case" is the kind of precision interviewers reward.

3.3 Growth Rates

Common growth orders

$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

What each can handle (roughly, in a 1-second budget on a modern CPU): - $O(n)$: $\sim 10^8$ - $O(n \log n)$: $\sim 10^7$ - $O(n^2)$: $\sim 10^4$ - $O(n^3)$: ~ 500 - $O(2^n)$: ~ 25

The $n \cdot \log n$ threshold

In competitive programming, $n = 10^5$ or 10^6 is typical. That forces $O(n)$ or $O(n \log n)$ solutions. An $O(n^2)$ solution with $n = 10^5$ means 10^{10} operations - about 30 seconds even in C++.

3.4 How to Count Steps

A single primitive operation (comparison, addition, dereference) costs $O(1)$. Loops multiply. Sequential blocks add. Function calls are analysed by analysing the called function.

Loop-growth rules of thumb

- A loop over n items, each step $O(1)$: **$O(n)$** .
- Two nested loops over n : **$O(n^2)$** .
- Halving/doubling a counter each step (binary search style): **$O(\log n)$** .
- Splitting n into halves recursively (merge sort style): **$O(n \log n)$** .

3.5 Space Complexity

Space complexity counts the *extra* memory an algorithm uses beyond the input. An in-place sort uses $O(1)$ extra space. Merge sort needs an $O(n)$ auxiliary array. Recursion uses stack space: a recursive function that recurses to depth d uses $O(d)$ space.

3.6 Common Complexity Patterns in One Picture

Code shape	Complexity
constant work	$O(1)$
loop over n	$O(n)$
loop over n , inner constant	$O(n)$
loop over n , inner loop over n	$O(n^2)$
loop over n , inner loop over i	$O(n^2)$
halving loop	$O(\log n)$
divide and conquer (two halves, $O(n)$ merge)	$O(n \log n)$
recurrence $T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
recurrence $T(n) = 2T(n/2) + O(1)$	$O(n)$
recurrence $T(n) = T(n-1) + O(n)$	$O(n^2)$
recurrence $T(n) = 2T(n-1) + O(1)$	$O(2^n)$
nested loop up to n with backtracking	$O(n!)$

3.7 The Master Theorem

For recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$ ($a \geq 1$, $b > 1$):

Master theorem

- If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$: **$T(n) = \Theta(n^{\log_b a})$**
- If $f(n) = \Theta(n^{\log_b a})$: **$T(n) = \Theta(n^{\log_b a} \cdot \log n)$**
- If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $a \cdot f(n/b) \leq c \cdot f(n)$: **$T(n) = \Theta(f(n))$**

Example: merge sort $T(n) = 2T(n/2) + O(n)$. Here $a=2$, $b=2$, $\log_b a = 1$, $f(n) = \Theta(n) = \Theta(n^1)$
 $\rightarrow$ case 2 $\rightarrow \Theta(n \log n)$. ✓

3.8 Amortized Analysis

Sometimes a single operation is expensive, but averaged over a sequence the cost is low. `vector::push_back` is the canonical example: one call may copy the whole array ($O(n)$), but over k calls the *total* cost is $O(k)$, so the *amortized* cost is $O(1)$ per call.

Amortized cost

The amortized cost of an operation is the total cost of a sequence of operations divided by the number of operations. It describes the *average* cost per operation over the worst-case sequence.

The accountant / potential method for vector growth

- Each `push_back` "pays" 3 units: 1 for the insert, 2 stored as credit.
- When a reallocation of size n needs to copy n elements, it spends the n units of credit accumulated since the last reallocation.
- Because the array doubles, each element is copied at most twice total (once into each larger array), so total work stays $O(n)$ for n inserts.

Other amortized examples

- `std::string/vector push_back` growth: amortized $O(1)$.
- `splay tree` operations: amortized $O(\log n)$.
- Union-Find with path compression + union by rank: amortized $O(\alpha(n)) \approx O(1)$.
- `std::deque push_back / push_front`: amortized $O(1)$.

3.9 Common Mistakes

- Saying " $O(2n)$ " or " $O(n/2)$ " - constants are dropped, it is just $O(n)$.
- Forgetting space complexity of recursion (stack depth).
- Assuming all sorts are $O(n \log n)$ worst case (quicksort is $O(n^2)$ worst case).
- Treating $O(1)$ *average* as $O(1)$ *worst* (hash tables).
- Not asking for the input size before choosing an algorithm.
- Mixing up `lower_bound` ($\log n$) on a sorted vector with linear `find`.

3.10 Applications

- Choosing between two candidate solutions before coding.
- Setting limits in competitive problems (each judge has a time limit).
- Designing systems: "what happens when users grow 10x?" - complexity analysis predicts it before it happens.
- Database query planning, search engines, and distributed algorithms all lean on asymptotic reasoning.

3.11 Interview Tips

- State complexity *after* you finish coding, as a habit: "This is $O(n \log n)$ time, $O(n)$ space."
- If asked "can you do better?", look for hidden loops first (nested loops are the usual culprit) and redundant copies.
- Interviewers love the trade-off question: "faster time vs less space?"
- For recursion, say the recurrence explicitly and solve it.

3.12 Summary

- Big-O = asymptotic upper bound; Ω = lower; Θ = tight.
 - $1 < \log n < n < n \log n < n^2 < 2^n < n!$.
 - Space includes recursion stack depth.
 - Master theorem solves $T(n) = aT(n/b) + f(n)$.
 - Amortized analysis explains why `push_back` is " $O(1)$ ".
-

Practice Set: Complexity Analysis

Beginner Problems

1. Find the complexity of a single loop from 1..n that calls an $O(1)$ function.
2. Find the complexity of a loop where i doubles.
3. Find the complexity of three sequential $O(n)$ loops.
4. Find the complexity of a nested loop where both bounds are n .
5. What is the space complexity of storing n integers in a vector?

Intermediate Problems

1. Determine complexity of `for i in 1..n: for j in 1..i: f()`.
2. Determine complexity of `while (n > 0) { n /= 3; }`.
3. Given two loops of n and m respectively (independent), state the complexity.
4. What is the complexity of binary search on a sorted array of size n ?
5. A function calls itself twice with $n-1$. What is its complexity?

Advanced Problems

1. Solve $T(n) = 3T(n/2) + O(n)$ using the master theorem.
2. Solve $T(n) = 2T(n/2) + O(n^2)$ using the master theorem.
3. Show that `std::vector::push_back` amortized cost is $O(1)$ formally.
4. Find the complexity of matrix multiplication by the schoolbook method ($n \times n$).
5. Compute the complexity of computing the n th Fibonacci number by naive recursion.

Interview-Style Problems

1. Given a sorted array, design $O(\log n)$ search. What would $O(1)$ even mean here?
2. Two nested loops where the inner loop runs until $ij < n$. *Find complexity.*

Competitive Programming Problems

1. $n = 10^5$, time limit 2 s. Which complexities are feasible? (State the reasoning.)
2. Estimate the largest n for which an $O(n^3)$ algorithm runs in 1 second.

Challenge Exercises

- Prove that $3n^3 + 100n^2 + 5$ is $\Theta(n^3)$.
 - Prove that a binary tree with n nodes has height at least $\log_2(n+1) - 1$, and use it to justify why BST operations are $O(h)$.
-

Solutions: Complexity Analysis

Beginner Solutions

1. $O(n)$. 2. $O(\log n)$. 3. $O(n)$ (constants and separate loops add but stay linear).
2. $O(n^2)$. 5. $O(n)$.

Intermediate Solutions

1. $O(n^2)$: sum of $1..n = n(n+1)/2 \sim n^2/2$.
2. $O(\log_3 n) = O(\log n)$ since n shrinks by a factor of 3 each step.
3. $O(n + m)$ - independent loops add, they do not multiply.
4. $O(\log n)$ - halving the search range each step.
5. $O(2^n)$ - the recursion tree doubles at every level.

Advanced Solutions

1. $a=3, b=2, \log_b a \approx 1.58, f(n)=O(n) = O(n^{(1.58-\epsilon)}) \rightarrow \text{case 1} \rightarrow \Theta(n^{1.58})$.
2. $a=2, b=2, \log_b a=1, f(n)=O(n^2)=\Omega(n^{(1+\epsilon)})$ and $2 \cdot (n/2)^2 \leq c \cdot n^2$ holds $\rightarrow \text{case 3} \rightarrow \Theta(n^2)$.
3. Use the potential method: charge 3 per push_back; the array doubles so the total copying cost is $O(n)$ over n inserts $\rightarrow$ amortized $O(1)$.
4. $O(n^3)$ - three nested loops over the dimensions.
5. $O(2^n)$ - naive recursion recomputes every subproblem exponentially many times.

Interview-Style Solutions

1. Binary search is $O(\log n)$; " $O(1)$ " for search would require a perfect hash / index which is generally impossible for sorted-array search.
2. The inner loop runs while $i*j < n$, i.e. about n/i iterations for fixed i . Sum over i of $n/i = n \cdot H(n) = O(n \log n)$.

Competitive Programming Solutions

1. $O(n)$ and $O(n \log n)$ are feasible (10^5 ops $\approx$ microseconds); $O(n^2) = 10^{10}$ is not.
2. Since $O(n^3)$ budget is $\sim 10^8$ ops, $n \approx (10^8)^{1/3} \approx 464$.

Challenge Solutions

- For n^3 : $3n^3 \leq 3n^3 + 100n^2 + 5 \leq 4n^3$ for $n \geq 200$, so the function is $\Theta(n^3)$.
- A binary tree of height h has at most $2^{(h+1)} - 1$ nodes; $n \leq 2^{(h+1)} - 1 \rightarrow h \geq \log_2(n+1) - 1$. BST operations follow a root-to-leaf path, hence $O(h)$.



MODULE

Arrays, Strings and Searching

The building blocks: contiguous storage, text processing and search algorithms

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

4. Arrays

4.1 Introduction

An **array** is a contiguous block of memory holding elements of one type. It is the most fundamental data structure in computer science. Everything else - lists, trees, graphs, heaps - is, at the bottom, an array (or a tree built on pointers). Most interview problems are array problems in disguise, so this chapter builds the mental model you will use for the rest of the book.

Array

An array is a fixed-capacity sequence $a[0], a[1], \dots, a[n-1]$ stored in consecutive memory. Element $a[i]$ lives at address $\text{base} + i * \text{sizeof}(T)$, so any element is reachable in **$O(1)$** .

4.2 Theory: Why Random Access is Fast

Because elements sit next to each other in memory, the CPU can fetch $a[i]$ directly using address arithmetic - no pointer chasing. This single property (cache locality + $O(1)$ index) is why arrays beat linked lists in almost every measurement.

4.3 Real-World Analogy

Think of an array as a **numbered locker wall**. Each locker has a fixed address (location). You know exactly which locker is the 27th one - you just walk to the 27th spot; you do not have to open lockers 1 through 26 to find it. Inserting a locker in the middle is painful (everything after it must be shifted), but "what is in locker i ?" is instant.

4.4 Static vs Dynamic Arrays in C++

Use `std::vector`

In 99% of code you want `std::vector`. It is a dynamic array: same $O(1)$ indexing, automatic resizing, no memory leaks. Use raw arrays only for ultra-performant hot paths or C-interop.

4.5 Operations on Arrays

Operation	Time	Notes
read <code>a[i]</code> / write <code>a[i]</code>	$O(1)$	direct address arithmetic
find a value	$O(n)$	must scan unless sorted
insert at end	$O(1)^*$	amortized for vector
insert in middle	$O(n)$	shift elements
delete from middle	$O(n)$	shift elements
min / max	$O(n)$	single scan
sort	$O(n \log n)$	<code>std::sort</code>
prefix sums	$O(n)$ build, $O(1)$ per range query	

Prefix sums - the single most useful trick

Prefix sums

$\text{pref}[i] = a[0] + a[1] + \dots + a[i-1]$ range $\text{sum}(L, R) = \text{pref}[R+1] - \text{pref}[L]$ Build: $O(n)$. Each query: $O(1)$.

4.6 Two Pointers

Many $O(n^2)$ problems become $O(n)$ with two pointers on a sorted array.

Two pointers

Maintain two indices and move them inward/outward according to the answer. Because the array is sorted, each pointer moves $O(n)$ times total, giving $O(n)$.

4.7 Sliding Window

A window of fixed or variable length slides over the array; each step adds one element and removes one.

4.8 Dry Run: Two Pointers with $a = [1,3,5,7,9]$, $K = 10$

4.9 Applications

- Adjacency lists in graph representations (arrays of vectors).
- Hash tables with open addressing.
- Heap implementations (array-indexed complete binary tree).
- DP tables (2D arrays of subproblem answers).
- Cache design, frame buffers, and every database table.

4.10 Common Mistakes

- Index out of bounds ($a[n]$ when size is n) - undefined behaviour.
- Off-by-one in prefix sums ($\text{pref}[R+1] - \text{pref}[L]$, not $\text{pref}[R] - \text{pref}[L]$).

- `size_t` underflow: `for (int i = v.size()-1; i>=0; --i)` with `size_t` loops forever.
- Confusing capacity with size in vector.
- Shifting elements inside a loop giving accidental $O(n^2)$.

4.11 Interview Tips

- "Can you do it in $O(n)$?" usually hints at two pointers, sliding window, prefix sums, or a hash map.
- For range queries, say "prefix sums, $O(n)$ build, $O(1)$ query".
- For any subarray-sum-/product problem, think of prefix arrays first.
- Draw the array with index labels; interviewers love seeing you trace.

4.12 Summary

- Arrays give $O(1)$ random access at the cost of $O(n)$ middle insert/delete.
 - Prefix sums turn range-sum queries into $O(1)$.
 - Two pointers turn $O(n^2)$ pair problems into $O(n)$ on sorted data.
 - Sliding window turns $O(n \cdot k)$ subarray problems into $O(n)$.
-

Practice Set: Arrays

Beginner Problems

1. Find the largest element and its index.
2. Reverse an array in place.
3. Print all elements at even indices.
4. Count how many numbers are greater than the average.
5. Check if an array is sorted ascending.

Intermediate Problems

1. Given a sorted array, remove duplicates in place and return the new length.
2. Rotate an array right by k positions ($O(n)$, $O(1)$ extra space).
3. Move all zeros to the end while preserving relative order of non-zeros.
4. Given an array and target K , return indices of the two numbers summing to K ($O(n)$).
5. Find the subarray (contiguous) with the largest sum - Kadane's algorithm.

Advanced Problems

1. Find the longest subarray with a sum equal to K (negative numbers allowed).
2. Trapping rain water: compute how much water can be trapped between bars.
3. Find the smallest window that sorts the whole array.
4. Product of array except self, without division, $O(n)$.
5. Maximum area between two vertical lines (Container With Most Water).

Interview-Style Problems

1. "Merge two sorted arrays in place" (the classic two-pointer from the end).
2. "Find the missing number in an array of $1..n$ with one missing" (XOR / sum).

Competitive Programming Problems

1. Given n and k , find the subarray whose sum is exactly divisible by k (uses prefix sums mod k).
2. Sliding window median with a window of size k (two heaps + lazy deletion).

Challenge Exercises

- Implement Kadane's algorithm that also reports the subarray boundaries.
 - Solve the "mountain array" problem: find the maximum element in a bitonic array in $O(\log n)$.
 - Implement a sparse-array "difference array" technique for range add updates.
-

Solutions: Arrays

Beginner Solutions

1. Loop tracking max and its index. 2. Swap ends moving inward.
2. Loop $i += 2$. 4. Two passes: compute avg, then count.
3. `for i in 1..n-1: if (a[i] < a[i-1]) return false; return true;`

Intermediate Solutions

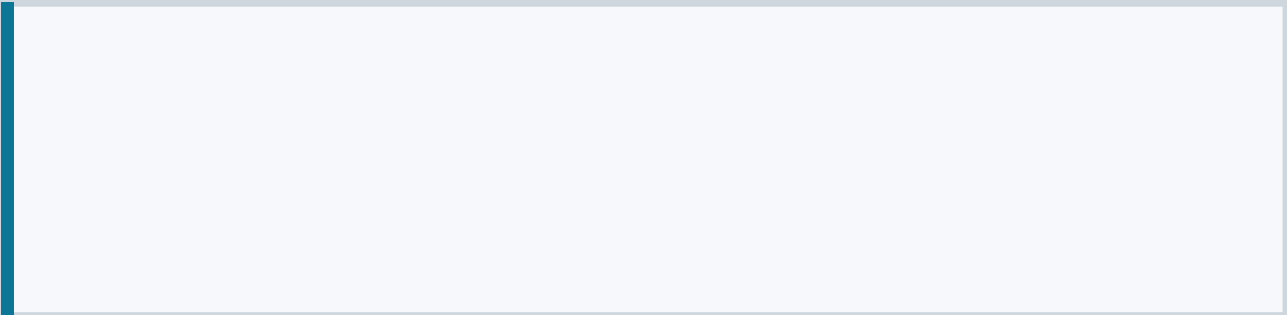
1. Slow/fast pointer: write index advances only when a new value appears.

1. Reverse the whole array, then reverse $[0, k-1]$ and $[k, n-1]$ (with $k \bmod n$).
2. Zero-pointer: move non-zeros forward; fill the rest with 0.

1. Hash map: store value->index; for each element check if $K - a[i]$ was seen.
2. Kadane: running sum, reset to 0 when negative; keep max.

Advanced Solutions

1. Prefix sums + map of first occurrence of each prefix value; a subarray sums to K when $\text{pref}[R] - \text{pref}[L-1] == K$.
2. Two pointers / stack: for each bar, $\text{water} += \min(\text{leftMax}, \text{rightMax}) - \text{height}$.
3. Find first and last index where the array is not sorted, then the min and max of that segment; the answer is $[\text{firstUnsorted}, \text{lastUnsorted}]$ extended until the min/max fit.
4. $\text{left}[i] = \text{product of all left elements}$; right product multiplied in a second pass.

- 
1. Two pointers from both ends; keep the taller line and move the shorter one.

Interview-Style Solutions

1. Start from the ends of both arrays (largest elements) and fill the merged result from the back; $O(n+m)$ in place.
2. $\text{expectedSum} - \text{actualSum}$ or XOR all numbers with $1..n$.

Competitive Programming Solutions

1. Store $\text{pref} \% k$ in a map; when the same remainder appears twice, the segment between them is divisible by k .
2. Maintain a median structure (two heaps) while sliding a window; remove old elements lazily (mark removed, clean the heap top when stale).

5. Strings

5.1 Introduction

Strings are arrays of characters with the vocabulary of text processing on top. In C++17, `std::string` is a dynamic, null-safe character buffer. String problems are a category of their own in interviews: palindromes, anagrams, substrings, and pattern matching appear constantly. Understanding how to manipulate strings efficiently (and what is $O(n)$ vs $O(n^2)$) is essential.

`std::string`

`std::string` stores a sequence of `char` in contiguous memory. It is mutable, resizable, and plays nicely with the STL algorithms. In C++17 you can store UTF-8 as `std::string` and use `std::string_view` for non-owning reads.

5.2 Common Operations and Their Cost

substr copies

`s.substr(i, len)` allocates a new string. In hot loops use `std::string_view` (a borrowed view, $O(1)$) instead:

5.3 Theory: String == Array of Char

Everything you know about arrays applies to strings, plus a few text-specific ideas:

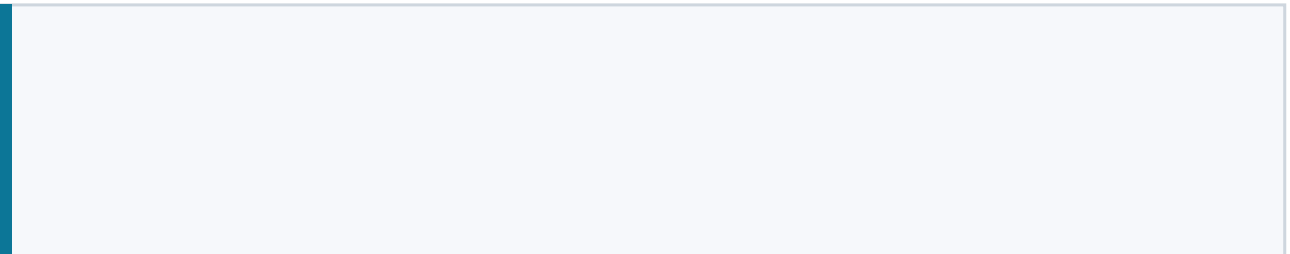
- **Anagram check:** sort both strings and compare, or count character frequencies.
- **Palindrome check:** two pointers from both ends.
- **Substring search:** pattern of n in text of m costs $O(n \cdot m)$ naive, $O(n+m)$ with KMP (Chapter 32).
- **Hashing strings:** compare substrings in $O(1)$ with polynomial hashes.

5.4 Real-World Analogy

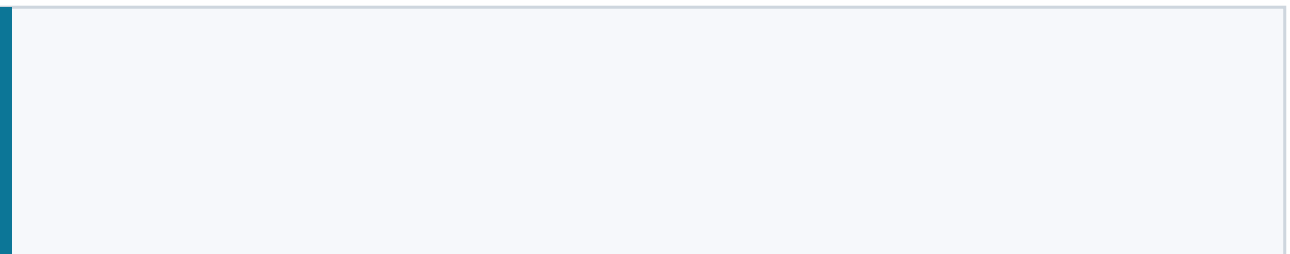
A string is a **row of lettered tiles**. You can look at any tile instantly ($O(1)$); you can flip/swap tiles; but "find the word 'the' in this row" requires scanning, and gluing two rows together requires physically moving tiles.

5.5 Worked Examples

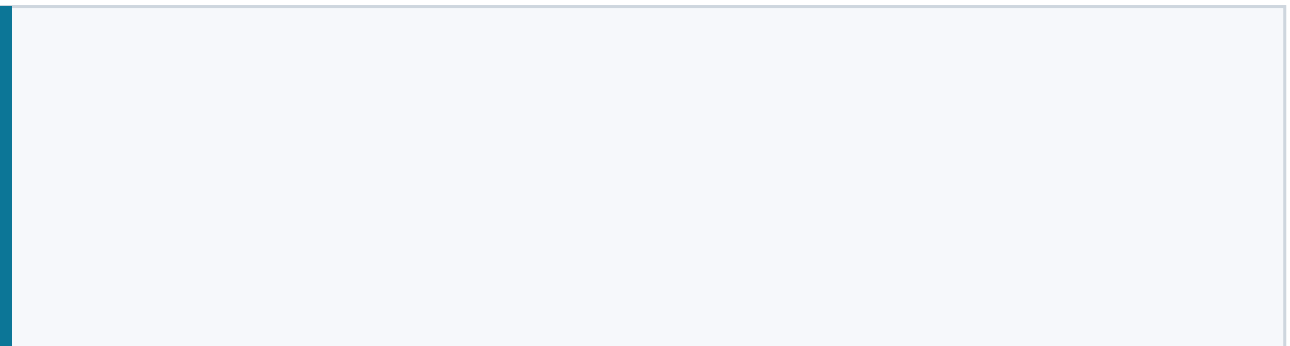
5.5.1 Anagram check - $O(n)$ with frequency table



5.5.2 Palindrome check - two pointers



5.5.3 Longest palindromic substring - expand around centers, $O(n^2)$



5.6 Dry Run: isAnagram("listen", "silent")

5.7 String Algorithms Preview

Technique	Use	Complexity
frequency array	anagrams, first-unique, duplicates	$O(n)$
two pointers	palindrome, reverse words	$O(n)$
sliding window	longest substring without repeats	$O(n)$
prefix hashing	compare substrings, Rabin-Karp	$O(1)/\text{substring}$
KMP / Z	pattern matching in text	$O(n+m)$
trie	dictionary/prefix queries	$O(L)$ per op

5.8 Common Mistakes

- Comparing strings with `==` inside a loop - fine, but remember it is $O(n)$ each.
- Using `s.size()` as `int` and mixing with negative indexes.
- `s[i] = c` on an empty string - out of bounds.
- Assuming `substr` is $O(1)$.
- Forgetting to sort before `binary_search` when using string collections.

5.9 Applications

- Compilers (tokenizing, symbol tables), search engines (inverted index, pattern match), DNA sequence analysis, plagiarism detection (n-gram hashing), text editors (autocomplete via trie), data compression.

5.10 Interview Tips

- For "longest substring without repeating characters" remember the sliding window with a `lastSeen[256]` array.
- For "group anagrams", the key is `sortedWord` or a 26-count fingerprint.
- Mention `string_view` when you want to avoid copies; interviewers like it.
- Always handle the empty string and single-character inputs.

5.11 Summary

- A string is an array of chars with $O(1)$ indexing.
- Frequency arrays solve anagram/hash-class problems in $O(n)$.
- Palindromes: expand around centers or two pointers.

- Watch out for hidden $O(n^2)$ from `substr`, `insert`, repeated `+=` in a loop.
-

Practice Set: Strings

Beginner Problems

1. Count the vowels in a string.
2. Check whether a string is a palindrome ignoring case.
3. Convert a string to lowercase.
4. Print the frequency of every character.
5. Reverse each word in a sentence, preserving word order.

Intermediate Problems

1. Longest substring without repeating characters ($O(n)$).
2. Check if two strings are one edit away (insert/delete/replace).
3. Given two strings, find the minimum window containing all chars of the other.
4. String to integer (atoi) with overflow handling.
5. Given a string with 0,1,?, replace '?' to make it a palindrome if possible.

Advanced Problems

1. Longest palindromic substring in $O(n)$ (Manacher's algorithm).
2. Find all starting indices of anagrams of pattern p in text s .
3. Implement wildcard matching " and '?' (DP or greedy).
4. Decode a string like "3[a2[bc]]" (nested).
5. Check if s can be formed by taking a prefix of a string repeated (rotation).

Interview-Style Problems

1. "Compare two version strings" (e.g. "1.2.3" vs "1.10.0").
2. "Group all anagrams from a list of strings."

Competitive Programming Problems

1. Given a dictionary and a string, break the string into dictionary words (word break).
2. Count distinct substrings of a string using a suffix array or rolling hash ($O(n^2)$ acceptable for small n).

Challenge Exercises

- Implement `strstr` in $O(n+m)$ using the Z-algorithm by hand.
 - Write a function to generate all rotations of a string and find the lexicographically smallest one (Booth's algorithm).
 - Implement a full "big integer" using `std::string` digits (add, multiply).
-

Solutions: Strings

Beginner Solutions

1. Loop counting a, e, i, o, u. 2. Lowercase then two pointers.
2. Use `tolower` or add 32 for ASCII letters.
3. `int freq[26]={0}; freq[s[i] - 'a']++;`
4. Split on spaces, reverse each token.

Intermediate Solutions

1. Sliding window with `last[256]` tracking the last index of each char; when a char repeats, move the window start past its previous occurrence.

1. Two pointers; count mismatches; allow one skip on either side.
2. Sliding window with a frequency array of the pattern; track how many requirements are satisfied to decide when to shrink.

1. Skip spaces/sign; accumulate with overflow clamp to `INT_MAX/MIN`.
2. Two pointers; for each mismatch pair if one is '?' set both to that letter, else impossible.

Advanced Solutions

1. Manacher: maintain center `c` and right bound `r`; reuse mirrors to compute radii in $O(n)$.
2. Sliding window anagrams: for each window of length `|p|`, compare the frequency fingerprint (a 26-vector); collect indices where they match.
3. DP on (i, j) or greedy with backtracking on "*position; greedy: match s with pattern left-to-right, on "* save state and consume more chars when stuck.
4. Stack of (prefix, repeat); process digits, `[`, `]` and letters.
5. Check `(s + s).find(rotation) != npos` and correct length (or use KMP).

Interview-Style Solutions

1. Split on '.', compare numerically part by part (ignore trailing zeros).

2. Map fingerprint -> group; the fingerprint can be the sorted string or a 26-count encoded as a string/tuple.

Competitive Programming Solutions

1. Word break: DP with $dp[i]$ = can prefix $[0..i)$ be segmented; check each dictionary word ending at i using set lookup $O(|w|)$.
2. Insert all substrings into an `unordered_set<string>` and return size (or use rolling hash to avoid copying).

Challenge Solutions

- Build Z array of `pattern + '#' + text`; every position where $Z[i] == |pattern|$ is a match.
- Booth's algorithm: minimal rotation via KMP-style failure on the doubled string.
- Big integer: represent as digit vector, add/multiply like in elementary school.

6. Searching Algorithms

6.1 Introduction

Searching is the most frequently executed operation in computing: finding a record, an element, a minimum, a threshold. This chapter covers linear search, binary search (and its many high-level variants), ternary search, and interpolation search. Binary search deserves special attention: getting it right - including the trickier "binary search on the answer" form - is a core interview skill.

6.2 Linear Search - $O(n)$

Walk the array from left to right until you find the target.

When to use linear search

Use linear search when the array is unsorted, tiny, or you must find all occurrences. It is also the baseline you compare binary search against.

6.3 Binary Search - $O(\log n)$

On a **sorted** array, compare the middle element with the target and discard half the search space.

The invariant

At every step, if x exists it lies in $[lo, hi]$. When the loop ends ($lo > hi$), lo is the first index whose value is $\geq x$ - this is exactly what `std::lower_bound` returns. Keep $lo = mid + 1 / hi = mid - 1$ (never $lo = mid$) or you risk an infinite loop.

6.4 Binary Search Variants - the real interview value

6.4.1 lower_bound / upper_bound

Invariant for lowerBound

$[0, lo)$ contains values $< x$; $[hi, n)$ contains values $\geq x$. At the end $lo == hi$ is the split point. This "boundary binary search" pattern is the one you will reuse constantly.

6.4.2 Binary search on the answer (parametric search)

Instead of searching for a value, search for the *answer* to an optimization problem using a monotone predicate $ok(mid)$.

Monotone predicate

A boolean function $ok(x)$ is monotone if there exists a boundary b such that $ok(x)$ is false for $x < b$ and true for $x \geq b$ (or the reverse). Binary search on the answer finds b in $O(\log \text{range})$ evaluations of ok .

Examples you will meet later: minimum ship capacity to ship all packages in D days; split an array into k subarrays minimising the largest sum; the k th element of two sorted arrays; "guess the median" problems.

6.5 Ternary Search - $O(\log n)$ on unimodal functions

For a function that strictly increases then strictly decreases (unimodal), pick two mid points; if $f(m_1) < f(m_2)$ the maximum is to the right of m_1 .

6.6 Interpolation Search - $O(\log \log n)$ average

When values are uniformly distributed, guess the position proportionally to the value rather than always the middle.

When interpolation search wins

On huge, uniformly distributed, sorted arrays (e.g. dictionary lookups) it can beat binary search. Worst case (bad distribution) degrades to $O(n)$; binary search is the safe default.

6.7 Complexity Comparison

Algorithm	Best	Average	Worst	Space	Sorted needed
Linear	$O(1)$	$O(n)$	$O(n)$	$O(1)$	no
Binary	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	yes
Ternary	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	unimodal fn
Interpolation	$O(1)$	$O(\log \log n)$	$O(n)$	$O(1)$	yes, uniform

6.8 Dry Run: binary search for $x = 3$ in $[1,2,3,4,5]$

6.9 Common Mistakes

- Forgetting to sort before binary search.

- $(lo + hi) / 2$ overflowing when $lo+hi$ exceeds int range - use $lo + (hi-lo)/2$.
- Infinite loop from $lo = mid$ instead of $mid + 1$.
- Using $\leq$ vs $<$ inconsistently between `lower_bound` and `upper_bound` variants.
- Off-by-one when the answer is "insert position" vs "index of value".

6.10 Applications

- Dictionary lookups, phone-book search, database B-tree searches.
- Debugging: "bisection" finds the first bad commit in $O(\log n)$ test runs.
- Numeric root finding; physics simulation (binary search on time steps).
- Recommendation systems (threshold tuning), compilers (search tables).

6.11 Interview Tips

- If the interviewer says "sorted array" you should think $\log n$ immediately.
- For "find the first / last occurrence", remember boundary binary search.
- For "minimise the maximum" or "maximise the minimum", say: "binary search on the answer with a greedy feasibility check".
- State the invariant out loud; it proves you know what you are doing.

6.12 Summary

- Linear search: $O(n)$, any order.
 - Binary search: $O(\log n)$, sorted only; master the boundary variants.
 - Binary search on the answer turns optimization into feasibility.
 - Ternary search finds maxima of unimodal functions.
 - Interpolation search: faster on uniform data, risky otherwise.
-

Practice Set: Searching

Beginner Problems

1. Implement linear search and count comparisons for a given array.
2. Implement binary search iteratively and check it on 10 test arrays.
3. Find the first occurrence of x in a sorted array.
4. Find the last occurrence of x in a sorted array.
5. Count the total occurrences of x in a sorted array.

Intermediate Problems

1. Find the index where a sorted rotated array's minimum is located ($O(\log n)$).
2. Search for a target in a rotated sorted array ($O(\log n)$, duplicates allowed variant too).
3. Find the smallest element in a sorted array that is $\geq x$ (lower bound).
4. Given an array and K , find the K -th smallest element (can use quickselect $O(n)$).
5. Find the peak element in an array where $a[i] \neq a[i+1]$ ($O(\log n)$).

Advanced Problems

1. Find the k th element in the union of two sorted arrays in $O(\log(\min(n,m)))$.
2. Given a mountain array (strictly inc then dec), find the peak and then search a target.
3. Minimise the maximum sum of k subarrays (binary search on answer).
4. Find the missing number in a sorted array of consecutive ints with one gap ($O(\log n)$).
5. Split array largest sum (LeetCode 410) - the interview-favourite.

Interview-Style Problems

1. "Implement $\text{sqrt}(x)$ without using math functions" (binary search / Newton).
2. "Find the median of two sorted arrays in $O(\log(\min(n,m)))$ ".

Competitive Programming Problems

1. Given n dishes and m guests with eating speeds, find the minimum time to serve all food (binary search on time).
2. Find the first bad version in an array of good/bad (binary search on version).

Challenge Exercises

- Implement `lower_bound` recursively and prove it terminates.
 - Write a correct binary search over floating point with 10^{-9} precision.
 - Given a predicate with a "valley" shape, adapt binary search to find the valley minimum.
-

Solutions: Searching

Beginner Solutions

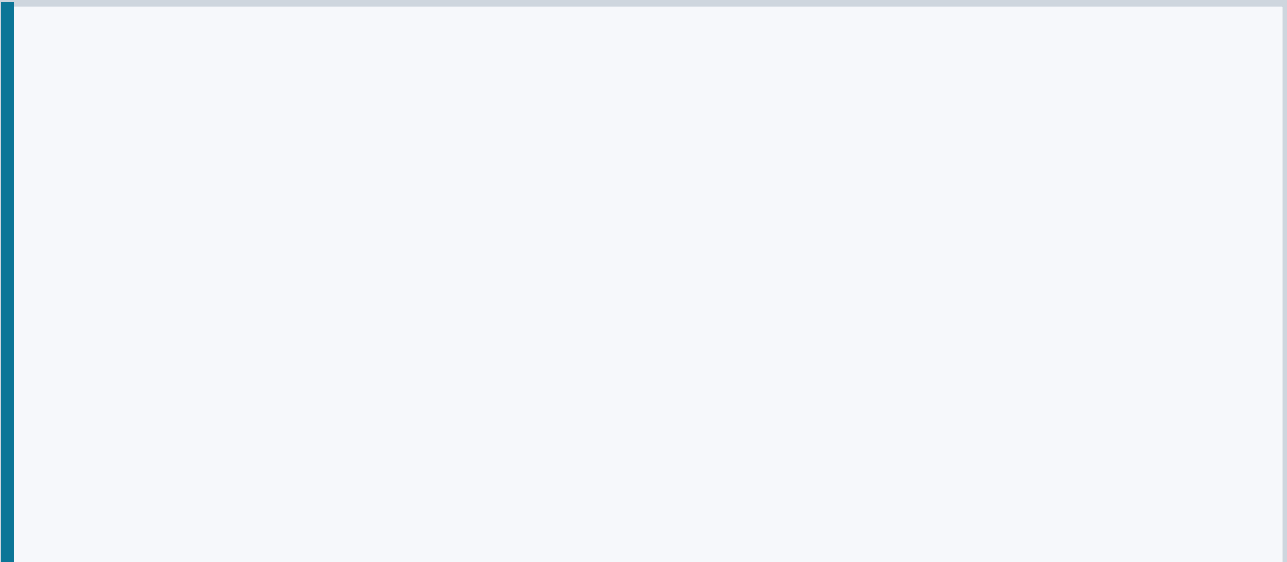
1. Return index and count each comparison.
 2. Standard implementation (Section 6.3); test on sorted arrays.
- 3-5. Boundary binary search:

Intermediate Solutions

1. Rotated-min: compare $a[mid]$ with $a[hi]$; if $a[mid] > a[hi]$ the min is in the right half, else in the left half (with care for equals).
2. Standard rotated search: one half is always sorted; check which side target belongs to. With duplicates, shrink $lo++/hi--$ when $a[lo]==a[mid]==a[hi]$.
3. `lower_bound` as in Section 6.4.1.
4. Quickselect: partition like quicksort, recurse only into the side containing index k ; expected $O(n)$.
5. Peak: if $a[mid] > a[mid+1]$ peak is at mid or left; else peak is right. (Works because $a[i] \neq a[i+1]$ guarantees a local peak exists.)

Advanced Solutions

1. Binary search on the smaller array's partition point; adjust so that the left partitions of both arrays contain the same total count and are correctly ordered (classic LeetCode 4).

- 
1. Find peak by ternary/binary, then binary search on each side.
 2. Same canSplit trick: binary search the largest sum; greedily pack subarrays.
 3. Compare $a[i] - a[0] == i$ to decide which side the gap is on.
 4. Shown above - the canonical "minimise max" binary search.

Interview-Style Solutions

1. Integer sqrt via binary search on $[0, x]$ with $mid * mid \geq x$ predicate; Newton's method converges faster for doubles.
2. LeetCode 4: binary search the smaller array partition (see #11).

Competitive Programming Solutions

1. Binary search on total time t ; check if all dishes can be served within t .
2. `isBadVersion(mid)` is the predicate; find first true.

Challenge Solutions

- Recursive lower_bound: same invariant, recurse on $[lo, mid)$ or $(mid, hi]$.
- Float binary search: loop 100 times (fixed iterations) instead of $lo < hi$.
- Valley minimum: if $a[mid] > a[mid+1]$ go right, else go left.

MODULE

Sorting

From $O(n^2)$ classics to $O(n \log n)$ and linear-time sorts

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

7. Sorting Algorithms

7.1 Introduction

Sorting is the most studied problem in computer science. Almost every system sorts something, and almost every advanced data structure (BST, heaps, segment trees) relies on order. Understanding sorting teaches you recursion, divide and conquer, divide-and-conquer analysis, and comparison vs non-comparison trade-offs - all at once.

Comparison sort

A sort that only ever compares two elements (with $<$) and permutes them is a comparison sort. Any comparison sort needs at least $\Omega(n \log n)$ comparisons in the worst case (there are $n!$ permutations but only 2^k comparison outcomes, so $k \geq \log_2(n!) \approx n \log n$).

7.2 The $O(n^2)$ Sorts: when simplicity wins

Bubble sort

Repeatedly swap adjacent out-of-order pairs; the largest element "bubbles" to the end each pass.

Selection sort

Find the minimum of the unsorted part and place it at the front.

Insertion sort

Take each element and insert it into the correct position of the already-sorted prefix. Excellent for nearly-sorted data and small arrays.

Insertion sort in practice

`std::sort` uses insertion sort for tiny subarrays (size $< \sim 16$) inside its introsort implementation - the constant factor beats divide and conquer at that size.

7.3 Merge Sort - $O(n \log n)$, stable, guaranteed

Divide, sort each half recursively, merge two sorted halves.

7.4 Quick Sort - $O(n \log n)$ average, $O(n^2)$ worst

Choose a pivot, partition into elements smaller/larger, recurse.

Worst case

If the pivot is always the min/max (sorted input + naive pivot), partition is unbalanced and quicksort degrades to $O(n^2)$. Fixes: random pivot, median-of-three, or introsort (used by `std::sort`) which falls back to heapsort. Quicksort is also **unstable** - equal elements may swap order.

7.5 Heap Sort - $O(n \log n)$, in place, not stable

Build a max-heap, repeatedly swap the max to the end and sift down. The heap data structure is the subject of Chapter 16; here it powers a sort.

7.6 Linear-Time (Non-Comparison) Sorts

When the keys are small integers or short strings, we can sort without comparing.

Counting sort - $O(n + k)$

Count occurrences, then place elements by position.

Radix sort - $O(d \cdot (n + k))$ for d-digit numbers

Sort digit by digit (least significant first) using counting sort per digit.

Bucket sort - $O(n)$ average

Distribute n elements into n buckets by value ranges, sort each bucket, concatenate.

Sort	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	no
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)^*$	no
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	no
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	yes
Radix	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	yes

- quicksort stack depth.

7.7 Sorting Complex Objects

Sorting structs with `std::sort` and a custom comparator (see Chapter 2). Stability matters when sorting by multiple keys: sort by the least significant key first.

7.8 Common Mistakes

- Using bubble sort for large inputs in production code.
- Unstable sorts where stability was required.
- Quicksort with a bad pivot on already-sorted input.
- Counting sort with negative values (shift by min).
- Forgetting that `std::sort` is not stable - use `std::stable_sort`.

7.9 Applications

- Databases (ORDER BY), search result ranking, spell-checker suggestion order.
- Binary search requires a sorted array.
- Duplicate detection and merging (sort then scan).
- Median/quantile computation; frequency histograms.

7.10 Interview Tips

- Know merge sort by heart: it is stable, $O(n \log n)$, and the backbone of "count inversions", "sort linked list", "merge k sorted lists".
- Quicksort's partition is reused in **quickselect** for k-th smallest.
- For "sort a string with only 3 letters", counting sort in $O(n)$.
- For nearly-sorted input, say insertion sort is $O(n)$ - interviewers love this.

7.11 Summary

- Comparison sorts: $\Omega(n \log n)$; non-comparison sorts beat it for small key ranges.
 - Merge: stable, $O(n)$ space. Quick: in place, $O(n^2)$ worst. Heap: in place, $O(n \log n)$ guaranteed, not stable.
 - `std::sort` = introsort (quick + heap + insertion fallbacks).
 - Linear sorts need a small known key domain.
-

Practice Set: Sorting

Beginner Problems

1. Implement selection sort and trace it on [4,2,7,1].
2. Implement bubble sort with the early-exit flag.
3. Sort an array of 3 distinct values (0,1,2) without extra space (Dutch national flag).
4. Given an array and index k, put a[k] in its correct sorted position (partition once).
5. Check whether an array is sorted; if not, find the first inversion index.

Intermediate Problems

1. Merge two sorted arrays in place (classic two-pointer).
2. Count inversions in an array using merge sort ($O(n \log n)$).
3. Sort an array so that all negatives come before positives (stable variant).
4. Given a list of intervals, sort by start then merge overlapping ones.
5. Sort a linked list in $O(n \log n)$ (merge sort on lists).

Advanced Problems

1. Sort an almost-sorted array where each element is at most k positions away ($O(n \log k)$).
2. Find the kth largest element with quickselect in expected $O(n)$.
3. Stable sort an array of strings by their second character.
4. Radix sort an array of integers.
5. Sort the elements of a matrix diagonally (matrix + sort).

Interview-Style Problems

1. "Given a stream of numbers, maintain the sorted order efficiently" (insertion with binary search / balanced tree).
2. "Sort colors" (Dutch national flag, $O(n)$, one pass).

Competitive Programming Problems

1. Given two sorted arrays, find the median of the combined arrays ($O(\log(\min(n,m)))$).
2. Sort an array with many duplicate values faster than $O(n \log n)$ (three-way quicksort / counting).

Challenge Exercises

- Implement introsort: start with quicksort, switch to heapsort on depth, use insertion sort for small subarrays.
 - Prove that any comparison sort requires $\Omega(n \log n)$ comparisons.
 - Implement a stable in-place $O(n \log n)$ sort (very hard) or explain why it is hard.
-

Solutions: Sorting

Beginner Solutions

1-2. Straightforward loop implementations (Section 7.2). 3. Dutch national flag: three pointers `lo`, `mid`, `hi`; on 0 swap with `lo++`; on 1 advance `mid`; on 2 swap with `hi--`.

1. Lomuto partition on the whole array with pivot = `a[k]`.
2. Scan for the first `i` with `a[i] > a[i+1]`.

Intermediate Solutions

1. Two pointers from the end (see Chapter 4 solutions).
2. In the merge step, when `L[i] > R[j]`, every remaining element of `L` is an inversion with `R[j]`: add `(L.size() - i)`.

1. Stable partition: use extra space (copy negatives then positives) or a list.
2. Sort by start; merge overlapping intervals in one scan.
3. Split list at mid, recursively sort halves, merge (works because lists have no random access - merge sort is the natural $O(n \log n)$ choice).

Advanced Solutions

1. Min-heap of size `k+1`: slide window, pop the smallest each time.

1. Quickselect: partition; if pivot index == `k-1` return; recurse into the side containing the target index. Expected $O(n)$, worst $O(n^2)$.

1. Sort by `s[1]` with a stable sort (`std::stable_sort` or merge sort).
2. Radix: counting sort per digit from LSD to MSD; be careful with negative numbers (shift) or use bucket sort per digit.

1. Collect each diagonal into a vector, sort, and write back.

Interview-Style Solutions

1. A balanced BST (`std::multiset`) supports ordered insert in $O(\log n)$; or keep sorted vector + binary search insert ($O(n)$ shift but simple).
2. Dutch national flag (solution 3).

Competitive Programming Solutions

1. Binary search the partition of the smaller array (Chapter 6, problem 17).
2. Three-way quicksort (Bentley-McIlroy) partitions into `< pivot`, `== pivot`, `> pivot`; duplicates cluster in the middle and are not re-sorted.

Challenge Solutions

- Introsort: track recursion depth; if $\text{depth} > 2 \cdot \log_2(n)$ switch to heapsort on that subarray; for $n < 16$ use insertion sort.
- Lower bound: the decision tree has $n!$ leaves and $\text{depth} \geq \log_2(n!) = n \log n$.
- In-place stable $O(n \log n)$ merge is possible (Block sort/grail sort) but very complex; `std::stable_sort` uses an $O(n)$ buffer when available.

MODULE

Recursion and Backtracking

Divide a problem into itself, then undo and retry

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

8. Recursion and Backtracking

8.1 Introduction

Recursion is the idea of a function calling itself to solve smaller copies of the same problem. **Backtracking** is recursion with undo: it tries a candidate, and if the choice leads nowhere, it *un-chooses* and tries the next one. Recursion is the engine behind trees, divide and conquer, dynamic programming, and nearly every hard problem in this book - so we build strong intuition here.

Recursion

A recursive function has two parts: 1. **Base case** - stops the recursion (smallest input, returns directly). 2. **Recursive case** - solves a smaller subproblem and combines the result.

8.2 The Three Laws of Recursion

1. **Base case**: there must be a case that does not recurse.
2. **Progress**: every recursive call must move toward a base case.
3. **Trust**: assume the recursive call solves the subproblem correctly.

8.3 Factorial and Fibonacci - the hello world of recursion

Exponential fib

`fib(n)` without memoisation recomputes subtrees: the recursion tree has $\sim 2^n$ calls. `fib(45)` already takes seconds. Memoisation (Chapter 27) reduces it to $O(n)$.

8.4 The Call Stack - how recursion uses memory

Every call pushes a stack frame (parameters + return address). Deep recursion = deep stack.

Stack overflow

Each frame costs stack memory. Recursion depth $\sim 10^6$ may overflow the default stack. For very deep recursion in C++, either increase stack size, convert to iteration, or (for some problems) use an explicit stack.

8.5 Classic Recursive Patterns

8.5.1 Tower of Hanoi

8.5.2 GCD via Euclidean algorithm

8.5.3 Generate all subsets - the "include / exclude" pattern

8.6 Backtracking - recursion with undo

Backtracking is the systematic search of all possibilities while pruning dead ends. The template is:

Backtracking template

8.6.1 N-Queens

Place n queens on an $n \times n$ board so no two attack each other.

8.6.2 Sudoku solver

Fill empty cells with digits 1-9, checking row/column/box constraints, undoing on failure.

8.6.3 Word search (grid + backtracking)

8.7 Analogy: Backtracking is a maze

Think of backtracking as exploring a maze with chalk. Walk forward while the path is open; when you hit a wall, **walk back to the last fork** (undoing your chalk marks along the way) and try the next corridor. Recursion gives you the "walk back" for free: each call's stack remembers the fork.

8.8 Complexity of Backtracking

Usually exponential: n-Queens is $O(n!)$ with pruning; subsets $O(2^n)$; permutations $O(n!)$. Backtracking is only feasible for small n ($n \leq 20$ -ish) unless heavy pruning or branch-and-bound is used.

Problem	Choices per node	Depth	Worst case
subsets	2	n	$O(2^n)$
permutations	n	n	$O(n!)$
n-Queens	n	n	$O(n!)$ with pruning
Sudoku	9	empty cells	$O(9^e)$
combinations	n	k	$O(C(n,k))$

8.9 Common Mistakes

- Forgetting the base case -> infinite recursion / stack overflow.
- Forgetting to **undo** after trying - the board stays modified.
- Passing large arrays by value (copies) -> exponential blow-up; pass by reference.
- Not pruning: exploring obviously invalid branches wastes time.
- Using global mutable state without restoring it.

8.10 Applications

- Expression evaluation and parsing, directory traversal, game AIs (minimax), constraint solving (Sudoku, crossword), permutation/combinatorial search in chemistry and logistics, and the foundation of dynamic programming.

8.11 Interview Tips

- Always identify base case, recursive case, and what changes each call.
- For backtracking, say "explore, then un-explore" - it signals you know the undo step.
- State the complexity: " $O(n!)$ in the worst case but pruned heavily in practice".
- Convert recursion to iteration with an explicit stack when asked.
- Practise the classic set: subsets, permutations, combinations, n-Queens, phone-letter combinations, word search.

8.12 Summary

- Recursion = base case + smaller subproblem + trust.
- Every call uses stack memory: watch depth.
- Backtracking = recursion + undo; prune invalid branches.
- Complexity is usually exponential; constraints dictate feasibility.

Practice Set: Recursion and Backtracking

Beginner Problems

1. Sum the first n natural numbers recursively.
2. Reverse a string using recursion.
3. Check whether a string is a palindrome recursively.
4. Print numbers $1..n$ then $n..1$ using recursion.
5. Compute a^b using fast exponentiation (divide and conquer).

Intermediate Problems

1. Generate all subsets of a set (iterative and recursive versions).
2. Generate all permutations of a string.
3. Generate all valid combinations of n pairs of parentheses.
4. Given a digit string, print all letter combinations (phone keypad).
5. Partition a string into all possible palindromic substrings.

Advanced Problems

1. Solve the N-Queens problem for $n = 8$ and count the solutions.
2. Solve Sudoku (9x9) with backtracking and print the solution.
3. Knight's tour on an $n \times n$ board.
4. Find all ways to climb n stairs with steps of size 1, 2, 3 (print the ways).
5. Word break - find all sentences that can be formed from a dictionary.

Interview-Style Problems

1. "Generate all distinct permutations of a string that may contain duplicates."
2. "Given a grid, count the number of paths from top-left to bottom-right moving only right/down (recursion then optimise with DP)."

Competitive Programming Problems

1. Rat in a maze - find all paths from (0,0) to (n-1,n-1).
2. Generate all partitions of an integer n (integer partitions).

Challenge Exercises

- Solve the "knight dialer" problem: count phone numbers a knight can dial.
 - Implement a memoised Fibonacci with `unordered_map` and compare call counts.
 - Write a non-recursive backtracking solver (explicit stack) for N-Queens.
-

Solutions: Recursion and Backtracking

Beginner Solutions

1. $n == 0 ? 0 : n + \text{sum}(n-1)$.
2. $\text{reverse}(s) = \text{reverse}(s.\text{substr}(1)) + s[0]$ (careful with copying) or swap-based.
3. Compare ends, recurse on the inner string.
4. Print n , recurse $n-1$, then print n (like Section 8.4).
5. $\text{pow}(a,b) = b\%2 ? a*\text{pow}(a,b-1) : (x=\text{pow}(a,b/2), x*x) - O(\log b)$.

Intermediate Solutions

1. Subsets: include/exclude recursion (Section 8.5.3).
2. Permutations: for each position swap with every later index, recurse, swap back.

1. Parentheses: track open/close counts; add '(' if $\text{open} < n$; add ')' if $\text{close} < \text{open}$.
2. Phone letters: recursive DFS over the digits with a mapping array.
3. Palindromic partitions: DFS; at each index try every palindrome prefix, recurse.

Advanced Solutions

1. N-Queens (Section 8.6.1) counts 92 solutions for $n = 8$.
2. Sudoku: find the first empty cell, try 1-9 that satisfy row/col/box, recurse; undo on failure. Use bitmasks for speed.
3. Knight's tour: backtracking over moves (8 directions), warnsdorff's heuristic (try the move with fewest onward options) makes it fast.
4. Climb stairs: DFS printing sequences (base case: $\text{sum} == n$).
5. Word break sentences: DP to find if possible, then DFS to reconstruct using memoisation on the index.

Interview-Style Solutions

1. Sort the string; in the loop skip j where $s[j] == s[j-1]$ to avoid duplicates.
2. $\text{Paths}(i,j) = \text{paths}(i+1,j) + \text{paths}(i,j+1)$; base at $(n-1,m-1)=1$. Memoise for $O(nm)$.

Competitive Programming Solutions

1. Rat in a maze: DFS over 4 directions with visited marks; collect paths.
2. Integer partitions: recursive $\text{part}(n, \text{max})$ choosing the next part $\leq \text{max}$.

Challenge Solutions

- Knight dialer: memoised DFS over (position, moves-left); the transition graph is small (10 cells).
- Memoised fib: count calls - fib(40) makes ~331k calls naive vs 41 with memo.
- Explicit-stack N-Queens: push (col, row) frames; pop to undo - more code but no recursion-depth limits.

MODULE

Linear Data Structures

Linked lists, stacks and queues: sequence structures with style

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

9. Linked Lists

9.1 Introduction

A **linked list** is a sequence of nodes where each node stores a value and a pointer to the next node (and, for doubly linked lists, to the previous node). Unlike an array, elements are *not* contiguous: inserting or deleting a node is $O(1)$ if you already have a pointer to the location, but random access becomes $O(n)$ because you must walk the chain.

Node

A node is a struct holding data and a next pointer (plus prev for doubly linked lists). The list is managed by a head pointer; a tail pointer makes appends $O(1)$.

9.2 Theory: The Cost Structure

Operation	Singly	Doubly	Notes
access head	$O(1)$	$O(1)$	
access index i	$O(n)$	$O(n)$	walk
insert at head	$O(1)$	$O(1)$	
insert at tail (with tail ptr)	$O(1)$	$O(1)$	
insert after a known node	$O(1)$	$O(1)$	
delete a known node	$O(1)^*$	$O(1)$	*singly: need prev
find a value	$O(n)$	$O(n)$	

Linked list vs array

Arrays win on cache locality and random access. Linked lists win on *splitting, merging, and constant-time insertion at arbitrary known positions*. In practice, `std::vector` is almost always faster; `std::list` exists when you truly need arbitrary insertion without invalidating iterators.

9.3 Real-World Analogy

A linked list is a **paper chain / scavenger-hunt clues**. Each clue tells you where the next clue is hidden. Adding a clue between two others is easy - you only rewrite two "where is the next" hints. But finding the 100th clue requires following 99 clues.

9.4 Implementing a Singly Linked List (C++17)

Memory ownership

With raw new/delete you own every node. Forgetting to delete leaks memory; double-deleting crashes. `std::list`, `std::forward_list` handle all of this for you - only hand-roll a linked list when asked or when teaching.

9.5 Reversal - the classic interview problem

9.6 Two Pointers on Linked Lists

9.6.1 Detect a cycle (Floyd's tortoise and hare)

9.6.2 Find the middle node

9.6.3 Find the k-th from the end

Advance fast by k first, then move both until fast hits the end; slow is the answer.

9.7 Sorting and Merging Lists

- Merge sort is natural for lists ($O(n \log n)$): split at middle, sort halves, merge two sorted lists.
- Merging two sorted lists: walk both with a tail pointer.

9.8 Common Mistakes

- Dereferencing nullptr (forgetting to check `c && c->next`).
- Not saving `nx = cur->next` before overwriting `cur->next` during reversal.
- Forgetting to delete (leak) or double-deleting nodes.
- Creating cycles accidentally when inserting.
- Confusing head vs tail after operations.

9.9 Applications

- Undo/redo in editors (doubly linked history), music playlists, LRU cache eviction order (doubly linked list + hash map), adjacency lists in graphs, symbol tables, file system directory links, and the hash-map chaining buckets.

9.10 Interview Tips

- Memorise the reversal, the hare-and-tortoise cycle detection, and the dummy-head pattern for clean insertion.
- For "remove nth node from end", use the fast/slow two-pointer trick.
- Say the space complexity of recursion (recursive reversal is $O(n)$ stack).
- Many "linked list" interview problems are just two-pointer problems in disguise.

9.11 Summary

- $O(1)$ insertion/deletion at known positions; $O(n)$ random access.
 - Reversal = iterate with prev/cur/next.
 - Cycle detection, middle, k-from-end: slow/fast pointers.
 - Merge sort is the natural $O(n \log n)$ list sort.
-

Practice Set: Linked Lists

Beginner Problems

1. Build a linked list from a vector and print it.
2. Count the nodes and compute the sum of values.
3. Insert a value in sorted order.
4. Find the n th node from the head.
5. Delete the first occurrence of a value.

Intermediate Problems

1. Reverse a linked list iteratively and recursively.
2. Detect a cycle and find the node where the cycle begins.
3. Merge two sorted linked lists.
4. Remove the n th node from the end in one pass.
5. Check whether a linked list is a palindrome ($O(n)$ time, $O(1)$ space).

Advanced Problems

1. Reverse nodes in k -group (LeetCode 25).
2. Add two numbers represented by linked lists.
3. Sort a linked list in $O(n \log n)$ (merge sort).
4. Flatten a multilevel doubly linked list.
5. Copy a linked list with random pointers (LeetCode 138).

Interview-Style Problems

1. "Intersection of two linked lists" - find the node where they meet.
2. "Rotate a list to the right by k positions."

Competitive Programming Problems

1. Given two linked lists, detect if they intersect and find the intersection point.
2. Split a linked list into two halves and interleave them (reorder list).

Challenge Exercises

- Implement an LRU cache with a doubly linked list and hash map (see Chapter 12).
 - Implement `std::forward_list`-style algorithms: partition a list by a value without allocating.
 - Solve "swap nodes in pairs" both iteratively and recursively.
-

Solutions: Linked Lists

Beginner Solutions

1-5. Straightforward pointer walks with the Node struct from Section 9.4.

Intermediate Solutions

1. Iterative reversal (Section 9.5); recursive: `newHead = reverse(head->next); head->next->next = head; head->next = nullptr; return newHead;`
2. Find the cycle with two pointers; then reset one to head and move both one step at a time - they meet at the cycle start.

1. mergeSorted (Section 9.7).
2. Dummy head + fast advanced by n, then move both until fast hits end; delete `slow->next`.
3. Find middle, reverse the second half, compare halves, optionally restore.

Advanced Solutions

1. Reverse k nodes at a time, stitching groups with a dummy head.
2. Walk both lists with carry; create nodes as you go.
3. Merge sort (Section 9.7): split via slow/fast, sort halves, merge.

1. Flatten: DFS/stack over the multilevel structure, rethread next pointers.
2. Clone with a hash map `node->clone`, then set next/random pointers (or $O(1)$ space interleaving trick).

Interview-Style Solutions

1. Walk both lists with two pointers; when one hits the end, switch to the other list's head - they meet at the intersection (or null).

1. Count length n ; $k \% n$; find new tail at $n-k$; rewire the cycle.

Competitive Programming Solutions

1. Same as 16 (find where the tails align; walk to intersection).
2. Reorder list: split at middle, reverse second half, interleave.

Challenge Solutions

- LRU cache: doubly linked list of (key,value) + hash map key->node; on access move node to front; on insert evict the tail node (Chapter 12).
- Partition a list by x : build less and greater chains, concatenate; no new allocation if you reuse nodes.
- Swap pairs: recursion is elegant: `swap(head->next->next)` on the rest, then rewire.

10. Stacks

10.1 Introduction

A **stack** is a LIFO (Last-In, First-Out) sequence. Only the most recently added element is accessible. The operations are push (add to top), pop (remove top), and top/peek (read top). Stacks are the data structure behind recursion, undo buttons, function calls, and expression evaluation.

Stack

A stack supports: - push(x) - add x to the top: O(1) - pop() - remove the top: O(1) - top() - read the top: O(1) - empty() - O(1)

10.2 Theory: LIFO discipline

The element removed first is the most recently added: Last In, First Out.

10.3 Real-World Analogy

A stack of **plates in a cafeteria** (or a stack of pancakes). You can only take the plate from the very top. The plate you put down last is the one you grab first. Undo in an editor works the same way: your *latest* action is undone first.

10.4 Implementation on top of vector

Array-backed stacks are best

A stack needs only push/pop at one end - that is exactly a vector's $O(1)$ end. Array-backed stacks have perfect cache locality and use $O(1)$ extra space per element. Linked-list stacks (a singly linked list with head = top) are used only when you cannot bound the stack size.

10.5 Classic Problems with Stacks

10.5.1 Balanced parentheses

10.5.2 Reverse Polish (postfix) evaluation

10.5.3 Next Greater Element (NGE)

For each element, find the nearest element to its right that is larger.

10.5.4 Monotonic stack pattern

The NGE problem is an instance of the **monotonic stack**: maintain a stack whose values (or indices) are strictly monotonic (here, decreasing). Every element is pushed once and popped once $\rightarrow O(n)$. This pattern solves histogram area, sliding window maximum (with a deque), daily temperatures, and more.

10.6 Dry Run: balanced parentheses on "{[()]}"

10.7 Complexity

Operation	Array-backed	Linked-list backed
push	$O(1)$ amortized	$O(1)$
pop	$O(1)$	$O(1)$
top	$O(1)$	$O(1)$
space	$O(n)$	$O(n)$ + pointer overhead
cache friendliness	excellent	poor

10.8 Common Mistakes

- Calling `top()`/`pop()` on an empty stack (undefined behaviour).
- Confusing stack (LIFO) with queue (FIFO).
- In NGE problems, forgetting that the stack stores *indices*, not values.
- Not handling unbalanced parentheses when the string ends non-empty.
- Using a stack when a simple variable (running max) suffices.

10.9 Applications

- Function call stack (each call pushes a frame - this is why recursion works).
- Undo/redo, browser back-button history.
- Expression conversion and evaluation (infix $\rightarrow$ postfix using the Shunting-yard algorithm), syntax parsing in compilers.
- Depth-first search, backtracking, tree traversals (iterative versions).
- Matching problems: HTML/XML tag matching, code braces.

10.10 Interview Tips

- If a problem mentions "most recent", "matching", "nested", or "previous/next greater/smaller", reach for a stack.
- Monotonic stacks turn many $O(n^2)$ problems into $O(n)$.
- Practice: valid parentheses, min stack, NGE, daily temperatures, largest rectangle in histogram, evaluate postfix.
- For "design a min stack in $O(1)$ ", keep a second stack of minimums (push the new min each time).

10.11 Summary

- Stack = LIFO; all operations $O(1)$.
 - Best backed by a vector (cache friendly).
 - Monotonic stack solves "next/previous greater/smaller" in $O(n)$.
 - Stacks power recursion, parsing, DFS, and undo.
-

Practice Set: Stacks

Beginner Problems

1. Reverse a vector using a stack.
2. Check balanced parentheses for one bracket type.
3. Implement a stack with an array and simulate push/pop.
4. Convert a decimal number to binary using a stack.
5. Given a string, print characters in reverse order using a stack.

Intermediate Problems

1. Min stack: support `getMin()` in $O(1)$.
2. Next Greater Element (print the result for every element).
3. Evaluate a Reverse Polish expression.
4. Sort a stack using only another stack.
5. Daily temperatures: days until a warmer day.

Advanced Problems

1. Largest rectangle in a histogram ($O(n)$ monotonic stack).
2. Trapping rain water with a stack (or two pointers).
3. Infix to postfix conversion (Shunting-yard).
4. Longest valid parentheses substring.
5. Basic calculator: evaluate expressions with `+, -, (, )` (and later `*, /`).

Interview-Style Problems

1. "Design a stack that supports push, pop, top, and retrieving the minimum in $O(1)$."
2. "Remove the duplicate letters so every letter appears once and result is smallest lexicographically."

Competitive Programming Problems

1. Given an array, for each element find the nearest smaller element on the left (or right) using a monotonic stack.
2. Maximal rectangle in a binary matrix (histogram trick, $O(nm)$).

Challenge Exercises

- Implement the "largest rectangle in histogram" and explain why each bar is pushed/popped once.
 - Implement an infix-to-postfix converter supporting `+` `-` `/` and parentheses.
 - Design a stack that also supports `getMax` in $O(1)$ and works for negative numbers.
-

Solutions: Stacks

Beginner Solutions

1. Push all then pop all into the result.
2. Push '(' on '('; pop on ')'; check empty at the end.
3. Use a vector or fixed array with a top index.
4. Push $n\%2$, $n/=2$, until 0; pop to print binary digits.
5. Same as 1 with a string.

Intermediate Solutions

1. Keep two stacks; push the current min onto the min-stack on every push.

1. Monotonic stack (Section 10.5.3).
2. RPN evaluation (Section 10.5.2).
3. Pop from the input; insert into the sorted stack while `sorted.top() > x`, move those back, then push.

1. Monotonic stack storing indices; when $a[i] > a[\text{st.top()}]$, $\text{answer} = i - \text{top}$.

Advanced Solutions

1. For each bar, find the first smaller bar on the left and right using two monotonic passes (or one pass popping on smaller): $\text{area} = h \cdot (r - l - 1)$.

1. Rain water: push indices; when current height > top, the popped bar can hold $\min(\text{neighbors}) - h[\text{popped}]$ across the span.
2. Shunting-yard: numbers to output; operators to a stack with precedence rules; parentheses frame the stack.
3. DP or stack on '(' indices; longest valid = max over i of i - start.
4. Two-stack expression evaluation (numbers + operators) with precedence.

Interview-Style Solutions

1. MinStack (Solution 6) - mention the $O(1)$ extra per push alternative using a single stack with encoding.
2. Count frequencies, then a monotonic stack with a "used" flag and remaining count to produce the lexicographically smallest subsequence.

Competitive Programming Solutions

1. Nearest smaller: monotonic increasing stack; answer when popping or when pushing (store index of nearest smaller).
2. For each row, treat it as a histogram of consecutive 1s above; run the largest-rectangle routine on each row $\rightarrow O(nm)$.

Challenge Solutions

- Histogram: each index is pushed once and popped once; amortized $O(n)$.
- Shunting-yard handles precedence and associativity with two stacks.
- For getMax: maintain a pair-stack of (value, currentMax); or encode the delta trick: push $2 * x - \text{oldMax}$ to recover both from a single stack.

11. Queues and Deques

11.1 Introduction

A **queue** is a FIFO (First-In, First-Out) sequence: the element added earliest leaves first. A **deque** (double-ended queue) allows insertion and removal at *both* ends. Queues model waiting lines, scheduling, and BFS; deques power sliding-window algorithms and are the backbone of many online algorithms.

Queue / Deque

Queue: push at the back, pop from the front, `front()` peek, $O(1)$ all. Deque: `push_front`, `pop_front`, `push_back`, `pop_back`, `front`, `back` - all $O(1)$.

11.2 Theory: FIFO discipline

The first element in is the first out - exactly like a line at the store.

11.3 Real-World Analogy

A **queue at a ticket counter** (or a printer queue). The person who joined the line first is served first; newcomers join at the back. A deque is a line with a **VIP door on both ends** - the concierge can admit or dismiss people from either side.

11.4 Implementations

11.4.1 Queue via ring buffer ($O(1)$, no copying)

Ring buffer

Using $(\text{index} + 1) \% \text{capacity}$ reuses freed slots at the front, so push/pop never shift elements. This is how real queues are implemented (including BFS).

11.4.2 `std::queue` and `std::deque`

11.5 Breadth-First Search (BFS) - the queue's star algorithm

BFS visits nodes in order of distance from the start - layer by layer. The queue holds nodes to visit next. (Graph theory detail in Chapter 19.)

BFS gives shortest paths in unweighted graphs

Because the queue processes nodes in increasing distance order, the first time a node is reached is via a shortest path. This is exactly why BFS, not DFS, solves "minimum number of moves".

11.6 Sliding Window Maximum with a Deque - $O(n)$

Keep a deque of candidate indices with strictly decreasing values. The front is always the window maximum.

11.7 Dry Run: maxSlidingWindow on [1,3,-1,-3,5,3,6,7], $k = 3$

11.8 Priority: What about priority_queue?

A `priority_queue` is *not* FIFO - it always removes the largest (or smallest) element. It is implemented with a heap (Chapter 16). Choose it when the "most important" element must be served first.

11.9 Complexity Summary

Structure	push back	push front	pop front	pop back	peek	random access
queue (deque)	$O(1)$	-	$O(1)$	-	$O(1)$	no
deque	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
vector	$O(1)^*$	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$
priority_queue	$O(\log n)$	-	-	-	$O(1)$ top	no

11.10 Common Mistakes

- Calling `front()`/`pop()` on an empty queue.
- Using a stack where FIFO is needed (or vice versa).
- Forgetting that queue has no random access and no `clear()` iteration.
- In sliding window, forgetting to evict out-of-window indices from the front.
- Storing copies of large objects in queues (store indices instead).

11.11 Applications

- OS scheduling (round-robin), printer and CPU job queues.
- BFS for shortest paths, web crawler frontiers, cache invalidation queues.
- Message queues / task queues (producer-consumer).
- Sliding window statistics, network packet buffering, streaming median windows.

11.12 Interview Tips

- Any "minimum number of steps / moves / days" problem on a grid or graph -> BFS.
- Sliding window problems: a deque keeps the window's extreme in $O(1)$.
- Say "queue for BFS, deque for sliding-window extremes, `priority_queue` for k-best".
- Remember: queue has no iterator - if you need one, use deque.

11.13 Summary

- Queue: FIFO; deque: both ends; both $O(1)$.
- Ring buffer implements a queue without shifting.
- BFS = queue; sliding-window max = deque.
- `priority_queue` is for "extreme first", not FIFO.

Practice Set: Queues and Deques

Beginner Problems

1. Implement a queue using two stacks.
2. Reverse a queue using a stack.
3. Simulate a ticket counter: process customers in FIFO order.
4. Implement a circular queue with a fixed capacity.
5. Use a deque to check whether a string is a palindrome.

Intermediate Problems

1. Sliding window maximum ($O(n)$ with deque).
2. Design a "recent counter" that returns how many pings happened in the last 3000 ms.
3. Rotting oranges: minimum time for all oranges to rot (BFS from all rotten).
4. Implement a stack using two queues.
5. First unique character in a stream (queue + frequency array).

Advanced Problems

1. Shortest path in a grid with obstacles (0/1 BFS with deque).
2. Sliding window minimum and maximum together (min and max deques).
3. Perfect squares: least number of perfect squares summing to n (BFS).
4. Open the lock: minimum turns to reach a target 4-digit code (BFS).
5. Number of islands (BFS/DFS) - counting connected components in a grid.

Interview-Style Problems

1. "Design a hit counter that tracks hits in the last 5 minutes."
2. "Find the minimum number of jumps to reach the end (BFS on indices / greedy)."

Competitive Programming Problems

1. Given a grid with two types of cells, find the shortest path (0-1 BFS).
2. Sliding window median with a window of size k (two heaps or multiset + two pointers).

Challenge Exercises

- Implement a deque from scratch using a circular array.
 - Implement BFS that also returns the path (store the parent of each node).
 - Solve "sliding window minimum" with a monotonic increasing deque and compare with the maximum version.
-

Solutions: Queues and Deques

Beginner Solutions

1. Push into s1; on pop, if s2 empty, move all of s1 into s2, then pop s2 top.

1. Pop all into a stack, then push them back - reversed.
2. Enqueue customers in arrival order, dequeue one at a time.
3. Ring buffer (Section 11.4.1) with head/tail wrapping.
4. Pop front and back alternately and compare.

Intermediate Solutions

1. maxSlidingWindow (Section 11.6).
2. Store ping timestamps in a queue; evict timestamps $< t - 3000$.

1. BFS from all rotten oranges simultaneously; the last layer depth is the answer (check that fresh oranges remain $\rightarrow -1$).
2. Stack with two queues: push to q1; on pop move all but last to q2, pop it, swap queues.

1. Queue of (char, firstIndex); frequency array; pop from front until the front char has frequency 1; that is the answer.

Advanced Solutions

1. 0-1 BFS: deque; edge weight 0 -> push_front, weight 1 -> push_back.

1. Two deques (min-increasing, max-decreasing); both evicted from front by window.
2. BFS over numbers: start from 0, next = cur + perfect squares; first time n is reached gives the minimum count.
3. BFS on 4-digit states; each state has 8 neighbours (each wheel +1/-1); skip deadends; return distance to target.
4. Grid DFS/BFS per unvisited land cell; each BFS marks one island.

Interview-Style Solutions

1. Hit counter: queue of timestamps (or circular bucket array for 5 minutes).
2. Jump Game II: BFS layers over indices where you can jump -> $O(n)$ greedy actually optimal (farthest reachable).

Competitive Programming Solutions

1. 0-1 BFS (Solution 11) - the deque prioritizes zero-cost moves.
2. Two heaps with lazy deletion (or `std::multiset` + sliding window): maintain the lower half (max-heap) and upper half (min-heap), rebalancing on each step.

Challenge Solutions

- Circular deque: an array with head/tail wrapping; grow on demand by doubling.
- BFS with parents: store `parent[v]` when relaxing; reconstruct by walking back.
- Sliding minimum: identical logic to maximum with `<` flipped to `>`.

MODULE

Hash Tables

Mapping keys to values in expected constant time

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

12. Hash Tables

12.1 Introduction

A **hash table** stores key-value pairs and supports insert, delete, and lookup in **$O(1)$ average time**. It achieves this by computing an *index* from the key with a hash function, then storing the pair in a bucket at that index. Hash tables are `std::unordered_map`/`std::unordered_set` in C++, and they appear in almost every non-trivial solution you will write.

Hash function

A hash function $h(\text{key})$ maps a key to an integer index in $[0, \text{tableSize})$. A good hash spreads keys uniformly and is cheap to compute.

12.2 Theory: From Key to Address

Two different keys can map to the same index - that is a **collision**. Hash tables differ mainly in how they resolve collisions.

12.3 Real-World Analogy

A hash table is a **library's call-number index**. You compute where a book "should" live from its title (a formula), and go straight to that shelf - no scanning every book. When two books get the same number, they share a shelf (bucket) and you scan only that tiny shelf.

12.4 Collision Resolution

12.4.1 Chaining (separate chaining)

Each bucket holds a linked list (or vector) of entries.

12.4.2 Open addressing

Store entries directly in the array; on collision, probe the next free slot.

Chaining vs open addressing

Chaining: simplest, handles high load factors, needs pointers/extra memory. Open addressing: cache friendly, but degrades badly above ~70% load and requires careful deletion (tombstones). C++ `unordered_map` implementations typically use chaining (with small bucket arrays).

12.5 Load Factor and Rehashing

Load factor

load factor $\alpha = (\text{number of entries}) / (\text{number of buckets})$

For chaining, expected bucket length = α , so lookups are $O(1 + \alpha)$. Standard libraries rehash (double the buckets and reinsert everything) when α exceeds ~ 1.0 , keeping α bounded and lookups $O(1)$ expected.

12.6 Hash Functions in C++

Don't hash mutable keys

If you insert an object into a hash set and then modify the fields used by its hash, the object becomes unreachable (its bucket is wrong). Treat set keys as immutable.

12.7 Complexity Analysis

Operation	Average	Worst case	Notes
insert	$O(1)$	$O(n)$	worst = all collide; rehash amortized $O(1)$
find	$O(1)$	$O(n)$	
erase	$O(1)$	$O(n)$	
iterate all	$O(n)$	$O(n)$	no order guarantee
memory	$O(n)$	$O(n)$	extra for buckets

"O(1)" means expected

The $O(1)$ is *expected*: it assumes the hash spreads keys uniformly. An adversary who controls the keys can force all keys into one bucket $\rightarrow O(n)$. This is why hashing-based containers are called "average-case" data structures.

12.8 Applications

- Caches (LRU via hash map + list), database indexes (hash index for equality).
- Symbol tables in compilers, spell-checker dictionaries, network routers (lookup tables), deduplication, counting frequencies, "does it exist" checks.
- Graph adjacency sets, union-find maps, memoization tables in DP.

12.9 Common Mistakes

- Iterating an `unordered_map` and relying on any order - there is none.
- Using a hash map for sorted iteration or range queries (use `map`).
- Modifying a key after insertion.

- Forgetting a good hash for custom types -> $O(n)$ behaviour.
- Using `unordered_map` with floating keys (precision/NaN issues).

12.10 Interview Tips

- "Can we do it in $O(1)$?" -> think hash map.
- Two-sum, anagram grouping, longest-consecutive-sequence, subarray-sum-equal-K, first-unique-character are all "hash map" problems.
- Say "expected $O(n)$ time, $O(n)$ space" to be precise.
- If the key domain is small (e.g., $0..10^5$), a plain array is a faster "hash".

12.11 Summary

- Hash tables: $O(1)$ expected insert/find/delete via a hash function.
 - Collisions resolved by chaining or probing; load factor kept low by rehashing.
 - C++: `unordered_map` / `unordered_set`.
 - No ordering; expected, not worst-case, bounds.
-

Practice Set: Hash Tables

Beginner Problems

1. Count the frequency of each element in an array.
2. Check if an array contains duplicates ($O(n)$).
3. Find the first non-repeating character in a string.
4. Check whether two strings are anagrams using a frequency table.
5. Given an array, find whether two elements differ by exactly k .

Intermediate Problems

1. Two sum - return indices of the pair summing to a target.
2. Longest consecutive sequence in $O(n)$.
3. Group anagrams from a list of strings.
4. Find the subarray with sum equal to K (negative values allowed).
5. Intersection of two arrays (unique values, $O(n)$).

Advanced Problems

1. Design an LRU cache (get/put in $O(1)$).
2. Longest substring with at most K distinct characters (sliding window + map).
3. Count the number of subarrays whose sum equals K (prefix sums + map).
4. Find all pairs of numbers in two arrays whose sum is a target ($O(n+m)$).
5. Design a data structure supporting insert, delete, search, and getRandom in $O(1)$.

Interview-Style Problems

1. "Determine if two strings are close" (swap any two chars, or transform all occurrences of a char - LeetCode 1657).
2. "Find all start indices of anagrams of p in s " (sliding window + fingerprint).

Competitive Programming Problems

1. Given n points, find the number of pairs with Manhattan distance $= a$ value (hashing by coordinate transforms).
2. Longest subarray with sum divisible by K (prefix-sum mod K hashing).

Challenge Exercises

- Implement a hash table from scratch with linear probing and rehashing.
 - Implement an LRU cache and explain why the doubly linked list + map combination gives $O(1)$ both ways.
 - Implement a rolling-hash based string matching (Rabin-Karp) - Chapter 32.
-

Solutions: Hash Tables

Beginner Solutions

1. `unordered_map<int,int> f; for (x : a) f[x]++;`
2. `unordered_set<int> s; for (x : a) if (s.count(x)) return true; s.insert(x);`
3. Frequency array; scan the string for the first char with frequency 1.
4. Count into `freq[26]`, decrement with the other string, all must be 0.
5. Store values in a set; check `a[i]-k` and `a[i]+k` in the set.

Intermediate Solutions

1. Map value->index while scanning; check `target - a[i]` in the map.

1. Insert all into a set; for each `x`, if `x-1` not in set, walk `x+1`, `x+2`... counting the streak; total $O(n)$.
2. Key = sorted word (or 26-count tuple); group by key.
3. Prefix sums + map of first index for each prefix value.
4. Put one array in a set; iterate the other, collect members; or two-pointer on sorted versions.

Advanced Solutions

1. LRU: `unordered_map<key, list<pair<key,val>>::iterator> + list`; get moves to front, put evicts from back.

1. Sliding window with counts per char; when distinct $> K$ shrink the left.
2. Map of prefix-sum frequency: for each prefix, add the count of (prefix - K) seen so far.
3. Build a set/map for one array; for each element of the other, look up the complement.
4. vector + map value $\rightarrow$ index; getRandom via index; erase via swap-with-last.

Interview-Style Solutions

1. Close strings: same character set, and the frequency multiset (sorted) is equal.
2. Fixed-size window fingerprint (26 counts) compared to p's fingerprint; collect matching indices.

Competitive Programming Solutions

1. Transform coordinates: count pairs via hashing on $x+y$ and $x-y$ after rotating 45 degrees for Manhattan distances.
2. Map prefix $\% K \rightarrow$ first index (or count); equal remainders $\Rightarrow$ divisible segment.

Challenge Solutions

- Linear probing: store (key, val, state) with tombstones; rehash when load > 0.7 .
- LRU $O(1)$: the map gives $O(1)$ access to a node; the list gives $O(1)$ move-to-front and eviction; this is the canonical "two data structures combined" answer.
- Rabin-Karp: hash(pattern) compared to rolling hashes of text windows.

MODULE

Trees and Binary Trees

Hierarchical data: the shape of information

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

13. Trees and Binary Trees

13.1 Introduction

A **tree** is a connected, acyclic graph: a hierarchy of nodes where each node (except the root) has exactly one parent, and any number of children. Trees model everything hierarchical: file systems, HTML documents, organisational charts, expression syntax, and search structures.

A **binary tree** restricts every node to at most two children - left and right - and is the foundation of BSTs, heaps, and expression trees.

Tree terminology

- **Root:** the top node (no parent).
- **Leaf:** a node with no children.
- **Edge:** a parent-child connection.
- **Height:** number of edges on the longest root-to-leaf path.
- **Depth:** number of edges from the root to the node.
- **Subtree:** a node plus all its descendants.

13.2 Theory: Properties of Trees

Property	Formula
Nodes vs edges	edges = nodes - 1 (for a connected tree)
Max nodes at depth d	2^d
Max nodes in a binary tree of height h	$2^{(h+1)} - 1$
Height of a binary tree with n nodes	at least $\log_2(n+1) - 1$
Leaves vs internal (full binary tree)	leaves = internal + 1

13.3 Real-World Analogy

A tree is a **family tree** or an **org chart**. The CEO is the root; VPs are children; individual contributors are leaves. You can reach anyone by following a unique chain from the root, and there is exactly one way to get there (no cycles).

13.4 Representing Binary Trees in C++

13.5 Traversals - visit every node exactly once

13.5.1 Preorder: root, left, right

13.5.2 Inorder: left, root, right

13.5.3 Postorder: left, right, root

13.5.4 Level-order (BFS) with a queue

Iterative inorder (explicit stack)

13.6 Level-by-Level Traversal (with depth)

13.7 Core Recursive Algorithms

Height / max depth

Count nodes / sum / find

Check if it is a balanced tree (heights differ by ≤ 1)

13.8 Dry Run: inorder traversal of the tree above

13.9 Complexity

Each traversal visits every node once: **$O(n)$** time, **$O(h)$** extra space for recursion (h = height), and $O(n)$ space for level-order with a queue.

13.10 Applications

- File systems and DOM trees; routing (routing tables as tries/B-trees).
- Expression trees (compilers), decision trees (ML), parse trees.
- Huffman coding trees (compression), heap as array-indexed complete tree.
- Search engines (inverted index trees), databases (B-trees).

13.11 Common Mistakes

- Dereferencing a null child before checking.
- Mixing up the three depth-first orders.
- Forgetting the queue in level-order (using a stack instead gives wrong order).
- Recursion stack overflow on very deep (skewed) trees - use iteration.
- Off-by-one in height (edges vs nodes).

13.12 Interview Tips

- "Tree" problems are almost always recursion: solve left, solve right, combine.
- Traversal order reveals structure: inorder on a BST is sorted.
- Preorder + inorder uniquely reconstruct a binary tree (common problem).
- For "diameter", "max path sum", "lowest common ancestor" think of the post-order combine pattern.

13.13 Summary

- Trees: $n-1$ edges, no cycles, one root.
- Binary trees: ≤ 2 children; traversals = pre/in/post/level.
- Height $O(h)$ recursion depth; each traversal is $O(n)$.

- Recursion is the natural tool; iteration needs explicit stacks/queues.
-

Practice Set: Trees

Beginner Problems

1. Count the number of nodes.
2. Compute the height / max depth.
3. Print the tree in inorder, preorder, and postorder.
4. Find the sum of all node values.
5. Count the number of leaves.

Intermediate Problems

1. Print level-order traversal level by level.
2. Check whether a binary tree is a BST.
3. Find the diameter of a binary tree (longest path between two nodes).
4. Check if two binary trees are identical.
5. Mirror a binary tree (invert left/right).

Advanced Problems

1. Lowest common ancestor of two nodes (binary tree, not BST).
2. Construct a binary tree from preorder + inorder arrays.
3. Maximum path sum in a binary tree (paths may start/end anywhere).
4. Flatten a binary tree into a linked list in place (preorder order).
5. Serialize and deserialize a binary tree.

Interview-Style Problems

1. "Check if a binary tree is a subtree of another."
2. "Boundary traversal of a binary tree" (left boundary, leaves, right boundary).

Competitive Programming Problems

1. Count all paths in a binary tree whose sum equals K (prefix sums in tree DFS).
2. Find the maximum width of a binary tree (level-order with indices).

Challenge Exercises

- Implement Morris traversal (inorder without recursion or stack, $O(1)$ space).
 - Reconstruct a binary tree from level-order + inorder.
 - Solve "vertical order traversal" (column grouping with BFS).
-

Solutions: Trees

Beginner Solutions

1-5. Direct recursive formulas:

Intermediate Solutions

1. Level-order with the sz loop (Section 13.6).
2. BST check: pass (min, max) bounds down; every node must fit.

1. Diameter: post-order; at each node, diameter = $\max(\text{leftH} + \text{rightH})$ across all.

1. Both null $\rightarrow$ true; one null $\rightarrow$ false; compare data and recurse both children.
2. Swap children recursively (`swap(r->left, r->right)`).

Advanced Solutions

1. LCA: post-order; if a node's subtree contains both targets (or the node is one of them) it is the LCA.

1. The first element of preorder is the root; split the inorder by the root.
2. Post-order combine: $\text{best} = \max(\text{best}, \text{left} + \text{right} + \text{node})$ but return only one branch + node upward.

3. Traverse preorder; keep a prev pointer; rewire $\text{prev} \rightarrow \text{right} = \text{cur}$, $\text{left} = \text{null}$.
4. Serialize with preorder + sentinel for nulls; deserialize from the stream.

Interview-Style Solutions

1. For every node in the big tree, isSubtree check against the small tree ($O(nm)$) or hash-based $O(n+m)$.
2. Boundary: left spine (excluding leaves), then leaves (inorder), then right spine reversed (excluding leaves and root duplication).

Competitive Programming Solutions

1. DFS with a map of prefix sums; on entering a node add, on exiting subtract.

1. Level-order with heap indices ($\text{left} = 2i$, $\text{right} = 2i+1$); track max spread.

Challenge Solutions

- Morris: for each node, link its predecessor's right pointer back to it, then walk; restore the tree afterwards.
- Level + inorder: BFS the levels; the root is $\text{level}[0]$; recurse using inorder.
- Vertical order: BFS with column numbers; group nodes by column (ordered by row, then value).

14. Binary Search Trees

14.1 Introduction

A **Binary Search Tree (BST)** is a binary tree with the *BST property*: for every node, all values in the left subtree are smaller and all values in the right subtree are larger. This ordering turns tree traversal into search: at each step you discard half the tree, giving $O(h)$ operations where h is the height.

BST property

For every node with value v : - left subtree contains only values $< v$ - right subtree contains only values $> v$ - (typically) no duplicates

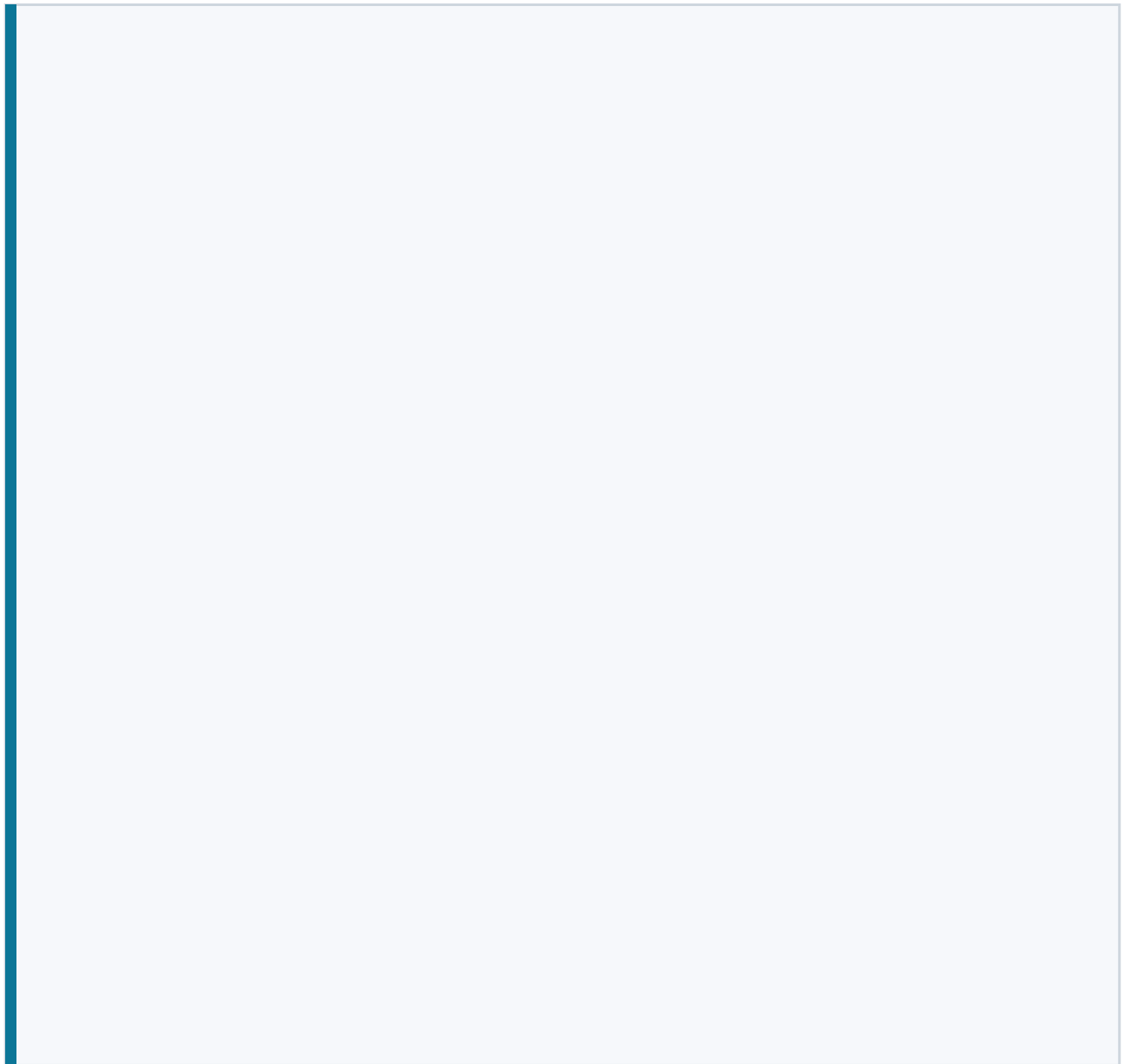
Inorder traversal of a BST yields the values in **sorted order**.

14.2 Theory: Why Search Becomes $O(h)$

14.3 Real-World Analogy

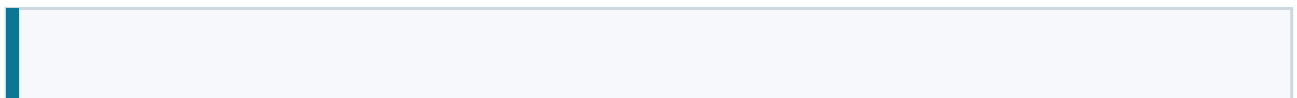
A BST is a **telephone directory organized by a "guess-half" game**: "Is the name before or after M?" Each question halves the candidates. Balanced, a BST finds anything in about $\log_2(n)$ questions - the same idea as binary search on a sorted array, but with cheap inserts/deletes.

14.4 Implementation (C++17)

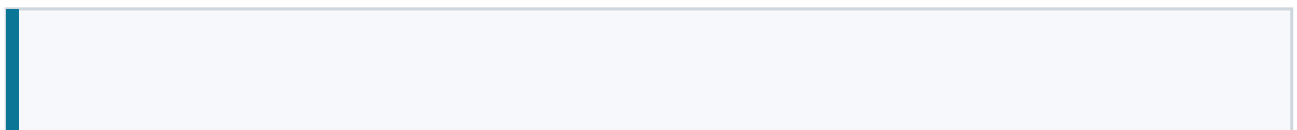


14.5 Operations Step by Step

Insert 18 into the tree from Section 14.2



Delete 10 (node with two children)



Deletion with two children

Copy the in-order **successor** (or predecessor) into the node, then delete that successor from the subtree. The successor always has at most one child, so the hard case reduces to the easy case.

14.6 Complexity Analysis

Operation	Balanced	Worst case (skewed)
insert	$O(\log n)$	$O(n)$
search	$O(\log n)$	$O(n)$
delete	$O(\log n)$	$O(n)$
min / max	$O(\log n)$	$O(n)$
inorder traversal	$O(n)$	$O(n)$
space	$O(n)$	$O(n)$

The height problem

Inserting sorted data (1, 2, 3, ...) creates a right-skewed "tree" that is effectively a linked list: $h = n$, and operations degrade to $O(n)$. Self-balancing trees (AVL in Chapter 15, red-black in practice) fix this by keeping $h = O(\log n)$.

14.7 BST vs Other Structures

Structure	Search	Insert	Delete	Sorted iteration
sorted array	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$
linked list	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$
hash table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	no
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$ sorted
priority_queue	$O(n)$	$O(\log n)$	-	no

BSTs uniquely combine logarithmic search with *sorted* iteration and in-order neighbour queries - that is why `std::map`/`std::set` are red-black trees.

14.8 Applications

- `std::map`, `std::set`, `std::multiset` (red-black trees).
- Database indexes (B-trees generalize BSTs to disk blocks).
- In-memory key-value stores, ordered caches, nearest-neighbour queries.
- Interval trees (augmented BSTs) for overlapping-range problems.
- K-d trees (multi-dimensional BSTs) for spatial search.

14.9 Common Mistakes

- Violating the BST property on insertion (inserting duplicates or $\leq$ / $\geq$ wrong).

- Deleting a node with two children by just removing it (loses the subtree).
- Forgetting to handle the one-child case in erase.
- Assuming $O(\log n)$ without balancing.
- Using a BST where a hash table (unordered) or array would be simpler/faster.

14.10 Interview Tips

- Inorder traversal of a BST = sorted output - use it for "kth smallest".
- "Validate BST" via (min, max) bounds (Chapter 13, Solution 7).
- Successor = min of right subtree; predecessor = max of left subtree.
- "Convert sorted array to BST" -> pick the middle recursively (balanced).
- Know the four cases of BST deletion cold.

14.11 Summary

- BST property: $\text{left} < \text{node} < \text{right}$; inorder is sorted.
 - $O(h)$ operations; $O(\log n)$ only when balanced.
 - Deletion with two children uses the successor.
 - Balanced BSTs are the basis of map/set.
-

Practice Set: Binary Search Trees

Beginner Problems

1. Implement search, insert, and inorder traversal.
2. Find the minimum and maximum value.
3. Compute the sum of all values.
4. Count nodes with values in a range $[lo, hi]$.
5. Find the k -th smallest value (inorder).

Intermediate Problems

1. Validate whether a binary tree is a BST.
2. Find the in-order successor of a given node (with or without parent pointers).
3. Convert a sorted array into a balanced BST.
4. Convert a BST into a sorted doubly linked list (in place).
5. Given two BSTs, merge into one sorted array.

Advanced Problems

1. Delete a node (all cases) and prove the successor approach works.
2. Find the lowest common ancestor in a BST ($O(h)$, compare values).
3. Check whether a BST has a pair summing to K (two-pointer on inorder).
4. Recover a BST whose two nodes were swapped (inorder anomaly).
5. Trim a BST to keep only values in $[lo, hi]$.

Interview-Style Problems

1. "Given a BST, find the closest value to a target."
2. "Given preorder traversal, reconstruct the BST."

Competitive Programming Problems

1. Count distinct BSTs with n distinct keys (Catalan numbers - also DP).
2. Range sum queries on a BST: sum of all values in $[lo, hi]$ in $O(h + k)$.

Challenge Exercises

- Implement a BST with parent pointers and all operations.
 - Implement Morris inorder traversal on your BST ($O(1)$ space).
 - Augment a BST to answer "how many values are $\leq x$ " in $O(\log n)$ (order statistics).
-

Solutions: Binary Search Trees

Beginner Solutions

1-5. Direct implementations:

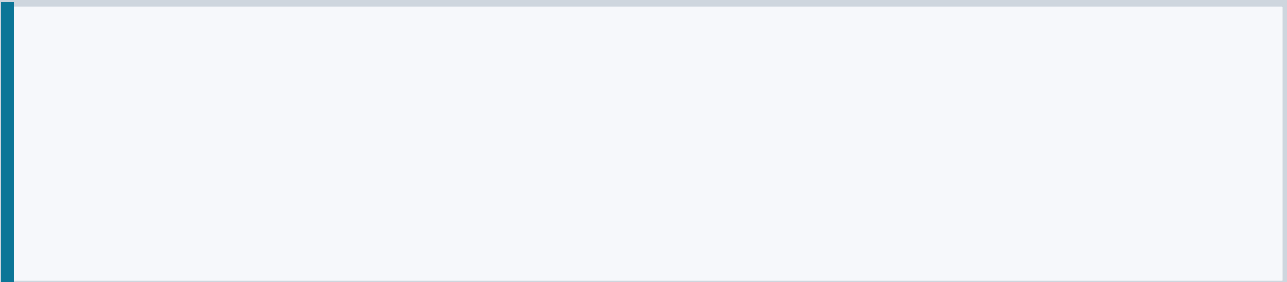
Intermediate Solutions

1. Bounds check (Chapter 13, Solution 7).
2. If the node has a right child: min of right. Else climb until you come from the left - that ancestor is the successor.
3. `build(l, r): mid = (l+r)/2; root = a[mid]; recurse on [l,mid) and (mid,r].`

1. Inorder walk; link each node to the previous one (head = leftmost).
2. Collect both inorders, merge like two sorted arrays.
3. Four cases: leaf, one child, two children (successor copy).
4. If both < node go left; both > node go right; otherwise node is the LCA.
5. Inorder gives a sorted array; use two pointers to find the pair.
6. Inorder: find the two positions where order is violated and swap their values.
7. Trim: if value < lo keep only the right subtree (trimmed); if > hi keep left.

Interview-Style Solutions

1. Walk down, tracking the closest: compare at each step, choose the child toward the target.

- 
1. Preorder + BST property: use a stack; the next value smaller than the top becomes a left child, otherwise pop until the right place for a right child.

Competitive Programming Solutions

1. Catalan number $C(n) = (2n)! / ((n+1)! n!)$; also DP: $dp[n] = \sum dp[i]dp[n-1-i]$.
2. Range sum: recurse, pruning branches fully outside $[lo, hi]$; visit only relevant nodes.

Challenge Solutions

- Parent pointers simplify successor/predecessor (no stack needed).
- Morris traversal threads the tree with temporary right links (Chapter 13).
- Order statistics: store subtree sizes; k-th smallest and count- $\leq$ are $O(\log n)$.

15. AVL Trees

15.1 Introduction

An **AVL tree** (named after Adelson-Velsky and Landis, 1962) is a self-balancing BST: after every insert or delete it guarantees that the heights of the two children of every node differ by **at most 1**. This "balance invariant" caps the height at $\sim 1.44 \cdot \log_2(n)$, so every operation is guaranteed $O(\log n)$ - no worst-case blow-up.

Balance factor

$\text{balance}(\text{node}) = \text{height}(\text{left}) - \text{height}(\text{right})$

An AVL tree keeps balance in $\{-1, 0, +1\}$ at every node. When an insertion or deletion pushes a node out of that range, it is fixed with rotations.

15.2 Theory: Height Bound

The AVL invariant implies a height bound: let $N(h)$ be the minimum number of nodes in an AVL tree of height h . Then:

$N(0) = 1, N(1) = 2, N(h) = 1 + N(h-1) + N(h-2)$

Solving gives $N(h) \geq \text{Fib}(h+3) - 1 \sim (\phi^h)/\sqrt{5}$, so $h \leq 1.44 \cdot \log_2(n)$.

In plain words: with a million nodes, the height stays around 20-21 instead of potentially a million in a skewed BST.

15.3 Real-World Analogy

An AVL tree is a **self-fixing bookshelf**. Whenever you insert or remove a book in a way that makes the shelf lopsided (too tall on one side), the librarian re-arranges the neighbouring shelves with a simple "rotation" so the whole bookcase stays compact. No shelf ever gets more than one book taller than its neighbour.

15.4 Rotations - the two basic moves

A rotation swaps a node with one of its children to rebalance, while preserving the BST ordering. There are four cases:

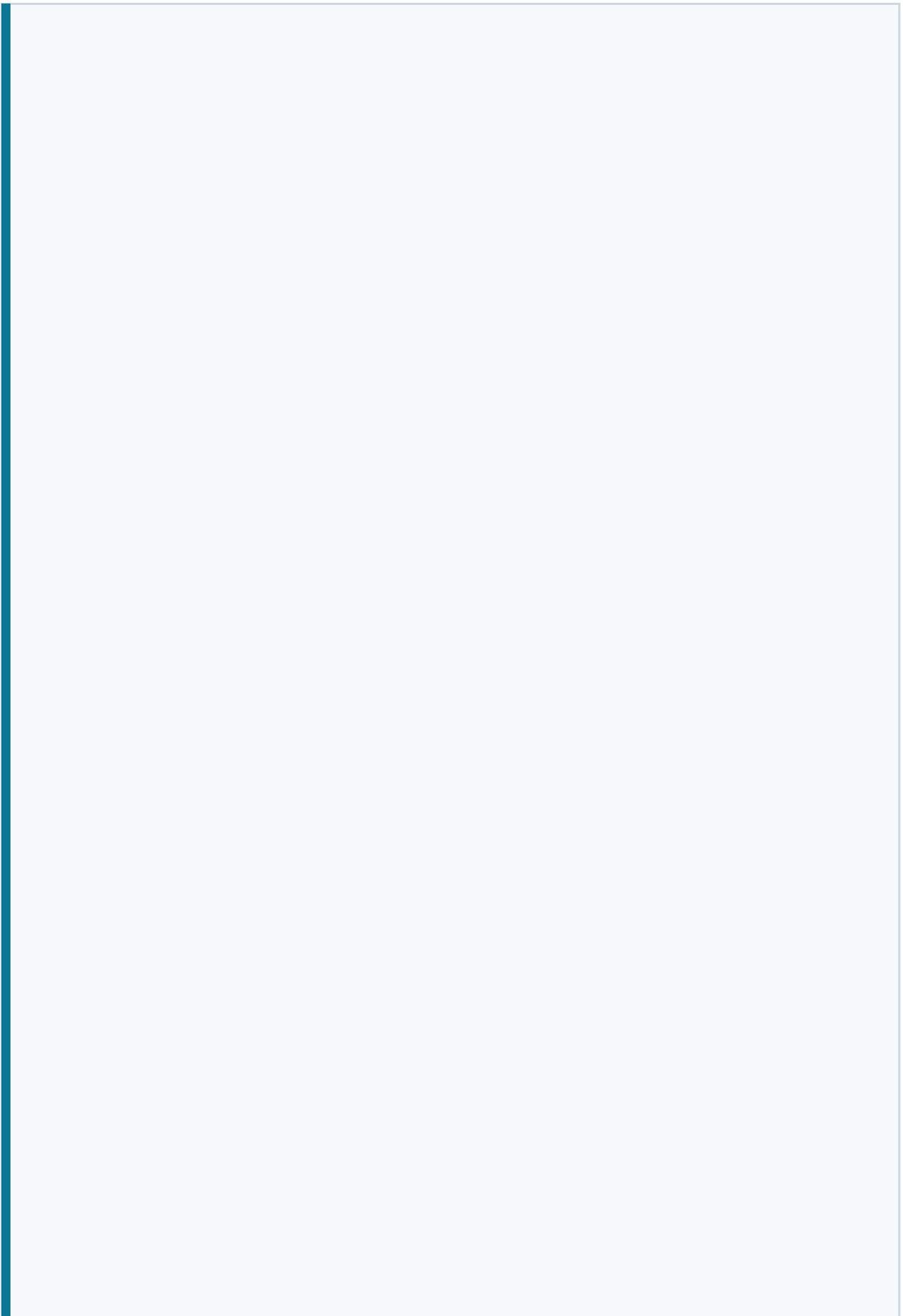
15.4.1 Right rotation (LL case)

15.4.2 Left rotation (RR case)

15.4.3 Left-right (LR case) = left rotate child, then right rotate

15.4.4 Right-left (RL case) = right rotate child, then left rotate

15.5 Full Implementation (C++17)



15.6 Dry Run: insert 1,2,3,4 into an empty AVL tree

15.7 Complexity Analysis

Operation	AVL (worst case)	Plain BST (worst)	Hash table
insert	$O(\log n)$	$O(n)$	$O(1)$ avg
search	$O(\log n)$	$O(n)$	$O(1)$ avg
delete	$O(\log n)$	$O(n)$	$O(1)$ avg
min/max	$O(\log n)$	$O(n)$	$O(n)$
sorted iteration	$O(n)$	$O(n)$	no
space	$O(n)$	$O(n)$	$O(n)$

15.8 Advantages and Disadvantages

Advantages

- Guaranteed $O(\log n)$ for all operations - safe against adversarial input.
- Maintains sorted order; supports successor/predecessor/range queries.
- Simple to reason about compared to red-black trees.

Disadvantages

- Each insert/delete may perform up to $O(\log n)$ rotations (only one *rotation pair* per insert, though).
- Storage overhead: 1 extra height field per node.
- Red-black trees allow fewer rotations on delete, so they are often preferred in libraries (`std::map`).

15.9 Applications

- `std::map`/`std::set` use red-black trees (same idea, different balance rule).
- Anywhere worst-case guarantees matter: routers, OS kernels, real-time systems.
- Implementing sorted containers with order statistics (augment with subtree size).
- Database in-memory indexes and caching layers.

15.10 Common Mistakes

- Forgetting to update the height after rotation/insertion.
- Applying only one rotation for the double-rotation cases (LR/RL).
- Checking balance before updating heights.
- Not rebalancing after deletion (deletion also requires fixes up the path).
- Copy-pasting rotations without re-linking the returned root into the parent.

15.11 Interview Tips

- You will rarely write a full AVL tree in an interview, but you MUST explain the idea: balance factor + 4 rotation cases keep height $O(\log n)$.
- Know LL/RR (single) vs LR/RL (double) by heart with the diagrams.
- "Why not just sort and binary search?" -> inserts/deletes are $O(\log n)$ vs $O(n)$.
- Contrast with red-black: AVL is more strictly balanced (faster search, slower inserts/updates).

15.12 Summary

- Balance factor in $\{-1, 0, 1\}$; height $\leq 1.44 \log_2(n)$.
- Rotations (LL/RR/LR/RL) restore balance after insert/delete.
- Guaranteed $O(\log n)$; sorted iteration for free.
- The conceptual base of all balanced trees (red-black, B-trees, treaps).

Practice Set: AVL Trees

Beginner Problems

1. Compute the balance factor of every node in a given tree.
2. Trace rotations on paper for inserting 5,3,7,2,4 into an empty AVL tree.
3. Implement height updates correctly in insert.
4. Verify (by a helper function) that a tree is a valid AVL tree.
5. Insert a sequence that triggers each of the four rotation cases.

Intermediate Problems

1. Implement `rotateRight` and `rotateLeft` with correct re-linking.
2. Implement insert with rebalancing (all 4 cases).
3. Implement the successor-based deletion with rebalancing.
4. Count nodes in $O(\log n)$ time using a stored subtree size (order statistics).
5. Build an AVL tree and print level-order after each insert.

Advanced Problems

1. Prove the height bound using Fibonacci numbers.
2. Implement a function `isAVL` that checks both BST ordering and balance.
3. Implement range queries: count values in $[lo, hi]$ in $O(\log n + k)$.
4. Implement k-th smallest in $O(\log n)$ using subtree sizes.
5. Merge two AVL trees into one in $O(m \log(n+m))$ by insertion.

Interview-Style Problems

1. "Explain the difference between AVL and red-black trees, and when you would choose each."
2. "Design an ordered set that also supports 'count values $\leq x$ ' in $O(\log n)$."

Competitive Programming Problems

1. Maintain a running set with insert/delete and query the median after each operation (AVL + order statistics).
2. Given two arrays, find the number of inversions using an AVL tree with subtree sizes.

Challenge Exercises

- Implement an iterative (non-recursive) AVL insert with an explicit stack.
 - Add a `rank(x)` operation (0-based index of value x) to the AVL tree.
 - Implement a treap (BST + heap priorities) and compare its simplicity with AVL.
-

Solutions: AVL Trees

Beginner Solutions

1. $\text{balance} = \text{height}(\text{left}) - \text{height}(\text{right})$; compute heights bottom-up.
2. Insert 5 $\rightarrow$ [5]; insert 3 $\rightarrow$ 5 with left 3; insert 7 $\rightarrow$ balanced; insert 2 $\rightarrow$ 4 $\rightarrow$ at node 5: $\text{balance}(5)=+2 \rightarrow$ LL case $\rightarrow$ $\text{rotateRight}(5)$.

3-5. Straightforward with the implementation in Section 15.5.

Intermediate Solutions

6-8. Use the full implementation above; test each rotation case with a small sequence and verify with level-order prints. 9. Store sz per node; $\text{sz} = 1 + \text{sz}(\text{left}) + \text{sz}(\text{right})$. 10. Level-order BFS after each insert; watch balance factors change.

Advanced Solutions

1. $N(h) = 1 + N(h-1) + N(h-2)$; $N(h) \geq \text{Fib}(h+3)-1$; Fibonacci grows as ϕ^h , hence $h = O(\log n)$.
2. isAVL : check BST ordering with bounds AND $\text{abs}(\text{balance}) \leq 1$ for every node.

1. Range count: recurse; if $n \rightarrow \text{data} < \text{lo}$ go right, if $> \text{hi}$ go left, else $\text{count} = 1 + \text{count}(\text{left}, \text{lo}, \text{hi}) + \text{count}(\text{right}, \text{lo}, \text{hi})$ with pruning.
2. k -th smallest via sz : if $k \leq \text{sz}(\text{left})$ go left; if $k == \text{sz}(\text{left})+1$ return node; else go right with $k -= \text{sz}(\text{left})+1$.
3. Insert all nodes of the smaller tree into the larger one (BST insert, $O(m \log(n+m))$), or flatten-merge-rebuild for $O(n+m)$.

Interview-Style Solutions

1. AVL: stricter balance -> faster lookups, more rotations on writes. Red-black: relaxed balance -> faster writes, slightly taller. Databases and language runtimes tend to use red-black; AVL when reads dominate.
2. Augmented BST with subtree sizes; count $\leq x$ by descending: when going right add $\text{sz}(\text{left}) + 1$.

Competitive Programming Solutions

1. Keep an augmented AVL; median = k-th smallest with $k = \text{size}/2$.
2. Inversions: insert elements left to right; for each new element, rank counts how many inserted values are greater ($\text{sz} - \text{rank}$).

Challenge Solutions

- Iterative insert: push the path onto a stack, then rebalance upward after the insert.
- $\text{rank}(x)$: descending; if $x < \text{node}$ go left; if $x > \text{node}$ add $\text{sz}(\text{left}) + 1$; equal -> $\text{sz}(\text{left})$.
- Treap: random priorities give expected $O(\log n)$ without rotations-on-delete; often easier to write than AVL.

16. Heaps and Priority Queues

16.1 Introduction

A **heap** is a complete binary tree that satisfies the *heap property*: every parent is $\geq$ (max-heap) or $\leq$ (min-heap) its children. Because the largest (or smallest) element is always at the root, a heap gives $O(1)$ access to the extreme and $O(\log n)$ insert/remove. Wrapped in a nice interface, it is the **priority queue** - `std::priority_queue` in C++.

Heap property

Max-heap: $\text{value}(\text{parent}) \geq \text{value}(\text{child})$ for every node. Min-heap: $\text{value}(\text{parent}) \leq \text{value}(\text{child})$ for every node. The heap is *complete*: all levels are full except possibly the last, filled left-to-right. Completeness lets us store the tree in an **array** with index arithmetic.

16.2 Theory: Array Representation

For a node at index i (0-based):

Index arithmetic

left child = $2i + 1$ right child = $2i + 2$ parent = $(i - 1) / 2$

Example max-heap (implicit in an array):

Node 84 (index 1): left = $2 \cdot 1 + 1 = 3$ (value 60), right = $2 \cdot 1 + 2 = 4$ (20). ✓

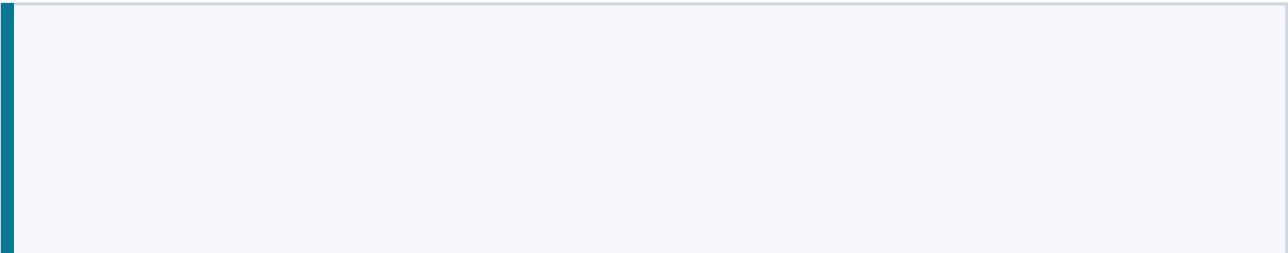
16.3 Real-World Analogy

A heap is the **hospital emergency room triage board**: patients are "inserted" when they arrive, but the *most urgent* one is always treated first - not the one who arrived first. New patients bubble up toward the top if they are urgent; after treatment, the next most urgent bubbles up. The rest of the queue is left roughly unordered - only the extreme matters.

16.4 The Two Core Operations

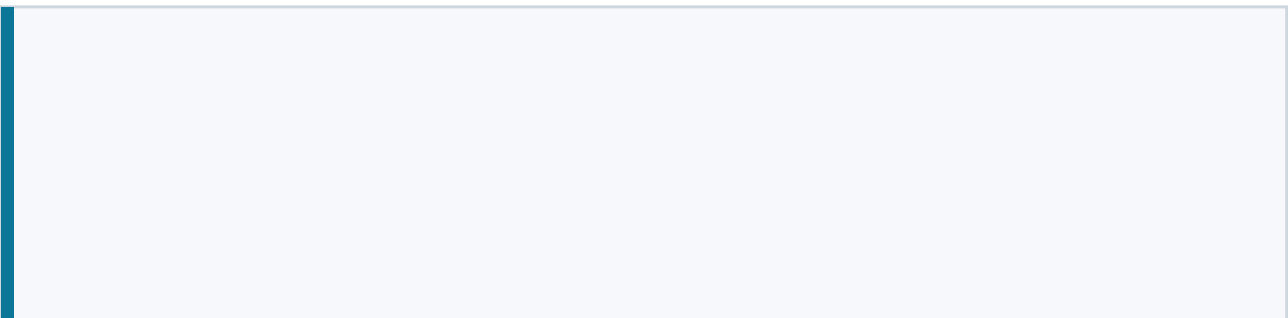
16.4.1 siftUp (bubble up) - used by push

Insert at the end, then swap with the parent while the heap property is violated.



16.4.2 siftDown (bubble down) - used by pop and buildHeap

Remove the root, move the last element to the root, then swap down with the larger child.



16.5 Complete Min-Heap Implementation (C++17)

16.6 Building a Heap in $O(n)$

Inserting n elements one by one is $O(n \log n)$. Building the heap from an existing array with bottom-up `siftDown` is **$O(n)$** .

Why $O(n)$?

Only the bottom level does no work; the level above does at most 1 swap each, and so on. Total work $\sim n/2 + n/4 + n/8 + \dots = n$. The "sum of geometric series" argument is a favourite interview question.

16.7 Dry Run: push 4, 8, 2 into a min-heap

16.8 `std::priority_queue`

priority_queue quirks

- It is a max-heap by default; flip with `greater<int>`.
- No iteration, no erase, no clear beyond reassignment.
- Comparator ordering is "reversed intuition": `cmp(a,b) == true` means a comes *after* b in the priority order (a is *lower* priority).

16.9 Complexity Analysis

Operation	Heap	Notes
top	$O(1)$	root
push	$O(\log n)$	siftUp
pop	$O(\log n)$	siftDown
build	$O(n)$	bottom-up
find (arbitrary value)	$O(n)$	no search support
space	$O(n)$	array-backed

16.10 Classic Heap Problems

16.10.1 Top-K largest (min-heap of size K)

Push; if size exceeds K , pop the smallest. The heap keeps the K largest, `top()` is the K -th largest. $O(n \log K)$.

16.10.2 Merge K sorted lists (heap of heads)

Push every list head; repeatedly pop the smallest, push its successor.

16.10.3 Median of a stream (two heaps)

Max-heap holds the lower half, min-heap the upper half; rebalance so sizes differ by ≤ 1 .

16.11 Applications

- Dijkstra's shortest path (priority queue by distance) - Chapter 20.
- Scheduling (OS task queues, priority scheduling), Huffman coding tree building.
- k -way merge, top- K , sliding window median, *A search*, *event simulations*.

16.12 Common Mistakes

- Confusing min-heap and max-heap directions in `priority_queue`.
- Using the wrong comparator direction (`priority_queue` comparator is inverted).
- $O(n)$ find when you assumed search was fast.
- Building a heap element-by-element instead of $O(n)$ `buildHeap`.
- Off-by-one in array indexing ($2i+1$, $2i+2$, $(i-1)/2$).

16.13 Interview Tips

- Whenever you need " k largest/smallest" or "always process the most/least important", say priority queue.
- The heap property is *partial*: it is not a sorted array - say so explicitly.
- For "top K frequent", combine a frequency map + heap.
- Be ready to implement `siftUp/siftDown` on a whiteboard - know both directions.

16.14 Summary

- Heap = complete binary tree + heap property, stored in an array.
- push/pop $O(\log n)$, top $O(1)$, build $O(n)$.
- `std::priority_queue` is a max-heap by default.

- Two-heap structures solve medians; heaps power Dijkstra and Huffman.
-

Practice Set: Heaps

Beginner Problems

1. Implement a min-heap with push/pop/top and test it.
2. Convert an array into a heap in $O(n)$ and verify the property.
3. Print the heap as a level-order array after a few operations.
4. Sort an array using a heap (heapsort).
5. Find the k-th largest element using a min-heap of size k.

Intermediate Problems

1. Merge K sorted arrays using a priority queue.
2. Median of a stream (two heaps).
3. Top K frequent words (frequency map + heap).
4. "Ugly numbers": the nth number whose only prime factors are 2, 3, 5.
5. Find the k closest points to the origin (heap of size k).

Advanced Problems

1. Design a data structure for the "k largest elements in a stream".
2. Find the k-th smallest sum of a pair from two sorted arrays (heap + visited set).
3. Sliding window median (two heaps with lazy deletion).
4. Minimum cost to connect ropes (always combine the two smallest).
5. Rearrange a string so no two adjacent characters are equal.

Interview-Style Problems

1. "Given an almost sorted array where each element is at most k away, sort it in $O(n \log k)$."
2. "Find the smallest range covering at least one element from k lists."

Competitive Programming Problems

1. Dijkstra's algorithm using a priority queue (shortest path).
2. Find the K-th smallest prime fraction (or the classic "K-th smallest element in a sorted matrix").

Challenge Exercises

- Implement a binary heap that supports decreaseKey (needed for Dijkstra with a heap on custom vertices).
 - Prove that buildHeap is $O(n)$.
 - Implement a pairing heap or Fibonacci heap at a conceptual level and compare.
-

Solutions: Heaps

Beginner Solutions

1-4. Implementations in Sections 16.4-16.6; heapsort = build + repeat pop.

1. Min-heap of size k: push; if size > k pop smallest; top() is the k-th largest.

Intermediate Solutions

1. Heap of (value, listIndex, elementIndex); pop -> push next element of that list.

1. Two-heap MedianFinder (Section 16.10.3).
2. Count frequencies, then a min-heap of size k ordered by (count, word).
3. Generate numbers with a heap seeded at 1; pop x, push x2, x3, x5 (*dedupe*).
4. Max-heap of size k ordered by squared distance; top() is the k-closest bound.

Advanced Solutions

1. KthLargest class with a min-heap of size k; add() pushes and trims.
2. Start with (a[0]+b[0]); expand to (i+1,j) and (i,j+1) with a visited set.
3. Two heaps + a deleted map for lazy removal of window-leaving elements.
4. Min-heap of rope lengths; combine two smallest each step, push the sum.

1. Count frequencies; max-heap by count; take two most frequent alternately (impossible if any count $> (n+1)/2$).

Interview-Style Solutions

1. Sliding min-heap of size $k+1$ (Chapter 7, Solution 11).
2. Heap of (value, list) for one element per list; track the range; move the smallest element forward each step to shrink the range.

Competitive Programming Solutions

1. Dijkstra with `priority_queue<pair<dist,node>>` (Chapter 20).
2. Sorted matrix: heap of candidates expanding right/down; or binary search on value with counting.

Challenge Solutions

- decreaseKey: keep a `pos[]` array mapping node $\rightarrow$ heap index; siftUp after decreasing.
- buildHeap $O(n)$: each node at height h does $O(h)$ work; sum of $h \cdot (\text{\#nodes at } h)$ telescopes to $O(n)$.
- Fibonacci heap: lazy insert, $O(1)$ insert/decreaseKey, $O(\log n)$ extract-min - great for dense Dijkstra but complex to implement.

17. Tries

17.1 Introduction

A **trie** (pronounced "try", from *retrieval*, also called a prefix tree) is a tree where each node stores one character, and paths from the root spell out strings. Tries give $O(L)$ lookups/inserts for strings of length L , independent of how many strings are stored. They are the natural structure for autocomplete, spell checking, and prefix queries.

Trie

Each node has a map (or array) of children indexed by character, plus a flag `isEnd` marking the end of a word. A path from root to any node represents a prefix; a path to a node with `isEnd` represents a stored word.

17.2 Theory: Sharing Prefixes

Why tries beat sets of strings

- Insert/find/delete: $O(L)$ where L = word length - no hashing, no comparisons.
- Prefix queries ("all words starting with 'ca'") are $O(L + \text{output})$.
- Sorted iteration: an in-order traversal of a trie outputs words in alphabetical order.

17.3 Real-World Analogy

A trie is a **living-dictionary with shared bookmarks**. Instead of writing the whole word at each entry, you follow a labelled hallway per letter. "car", "card" and "cart" share the same hallway through 'c', 'a', 'r', then split. Typing "c-a" puts you right where all those words branch off - that is autocomplete.

17.4 Implementation (C++17)

Child storage

An array of 26 pointers is fast but heavy (26 pointers per node). For sparse/Unicode alphabets use `unordered_map<char, TrieNode*>` or a sorted vector of children. Array is fine for lowercase ASCII.

17.5 Operations and Complexity

Operation	Time	Notes
insert	$O(L)$	create missing nodes
search	$O(L)$	exact word
startsWith	$O(L)$	prefix existence
autocomplete	$O(L + k)$	DFS from prefix, k = results
delete	$O(L)$	unmark + prune (optional)
space	$O(\text{total chars})$	worst: no sharing -> sum of lengths

17.6 Dry Run: insert "tea", then search "ted" in a trie

17.7 Trie Applications

- Autocomplete / predictive text, T9 phone keypad.
- Spell checkers and word games (boggle, word squares).
- IP routing (longest-prefix match in a binary trie).
- Compression (patricia/radix trees), search-engine suggestion, bioinformatics (DNA sequence indexing).

17.8 Common Mistakes

- Forgetting the `isEnd` flag and returning true for prefixes of words.
- Using a fixed 26-array for non-lowercase data (spaces, uppercase, Unicode).
- Not freeing trie memory (many small nodes; use `unique_ptr` or a pool).
- Assuming trie search is $O(1)$ like a hash map - it is $O(L)$, which is usually faster per operation because no hashing cost.
- Inserting empty strings without a flag at the root.

17.9 Trie vs Alternatives

Structure	Search	Insert	Prefix query	Space
<code>unordered_set</code>	$O(L)$ avg	$O(L)$ avg	no	high (hashes)
<code>set</code> (sorted)	$O(L \log n)$	$O(L \log n)$	via <code>lower_bound</code>	medium
trie	$O(L)$	$O(L)$	$O(L + k)$	high per char
suffix trie	-	-	substring	very high

17.10 Interview Tips

- "Prefix", "autocomplete", "all words starting with", "dictionary lookup of substrings" -> trie.
- Combine trie with DFS/backtracking for word-search variants.
- For "longest common prefix of two words", walk the trie while paths match.
- Say " $O(L)$ per operation where L is the word length" and note the space trade-off.

17.11 Summary

- Trie = prefix-sharing tree; insert/search/startsWith in $O(L)$.
- Array children for small alphabets, maps for large/Unicode.
- Perfect for autocomplete and prefix queries.
- Trade memory for speed compared to hash sets.

Practice Set: Tries

Beginner Problems

1. Implement insert, search, and startsWith.
2. Count the number of words in the trie.
3. Count words with a given prefix.
4. Print all words stored in the trie (in alphabetical order).
5. Check if a word exists that is a prefix of another word.

Intermediate Problems

1. Implement autocomplete: return up to k suggestions for a prefix.
2. Longest common prefix of all words (trie approach).
3. Given a dictionary, find all words that can be formed by removing one char repeatedly (word ladder via trie).
4. Implement T9: map digit strings to possible words.
5. Find the longest word in a dictionary that can be built from other words in the dictionary.

Advanced Problems

1. Word search II: find all dictionary words in a 2D grid (trie + backtracking).
2. Design a "prefix and suffix search" data structure.
3. Implement a compressed trie (radix tree) - analyse the space savings.
4. XOR maximization: given numbers, find the max XOR with a query (binary trie).
5. Count distinct substrings of a string using a trie (insert all suffixes).

Interview-Style Problems

1. "Design a search autocomplete system" (trie + frequency ranking).
2. "Implement a phone directory using a trie."

Competitive Programming Problems

1. Maximum XOR of two numbers in an array (binary trie, $O(32n)$).
2. Palindrome pairs: find pairs of words whose concatenation is a palindrome (trie of reversed words).

Challenge Exercises

- Implement a trie with unique_ptr children and a destructor that frees all nodes.
 - Implement deletion that prunes subtrees with no descendants.
 - Build a suffix trie for a string and answer substring-membership queries.
-

Solutions: Tries

Beginner Solutions

1-5. Straightforward on the Section 17.4 implementation:

Intermediate Solutions

1. Walk to the prefix node; DFS from there collecting words (stop at k).
2. Walk while all children have exactly one path; the walked string is the LCP.
3. Trie + BFS/DFS over states, trying each removal; or DP with a set.
4. For each digit, try all 3-4 letters at each trie node, collecting valid words.
5. Sort by length desc; DFS over prefixes; memoise reachable states.

Advanced Solutions

1. Trie of the dictionary + grid DFS: on a match, record and continue (don't stop) to find overlapping words. Use the visited mark/undo pattern.

1. Store (prefix, suffix) pairs in the trie of suffix#prefix; search the encoded key.
2. Radix/patricia trie: merge single-child chains into one node with a substring; fewer nodes, complex but powerful (used in real routers).
3. Binary trie of 32-bit numbers; for max XOR, at each bit prefer the opposite branch.

1. Insert every suffix $s[i..]$ into a trie; the number of nodes - 1 = distinct substrings.

Interview-Style Solutions

1. Trie nodes store a frequency; suggestions = top-k words under the prefix (heap by frequency).
2. Phone directory: trie of names; search by prefix; maintain "available" flags per node.

Competitive Programming Solutions

1. Binary trie (solution 14); $O(32n)$ time, $O(32n)$ space.

2. Build a trie of reversed words; for each word, walk and check palindrome conditions on the remainder.

Challenge Solutions

- `unique_ptr` children: the destructor runs recursively automatically.
- Delete: if no other word uses the branch and no descendants, free upward.
- Suffix trie: insert each suffix; substring query = walk the trie.

MODULE

Revision 1

Consolidate: C++, STL, Complexity, Arrays, Strings, Searching, Sorting

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

Revision 1: C++ and Complexity Through Sorting

R1.1 One-Page Revision Sheet

- **Growth order:** $1 < \log n < n < n \log n < n^2 < n!$.
- **Big-O** = upper bound; Ω = lower; Θ = tight. Drop constants and low terms.
- **Amortized:** vector `push_back` is $O(1)$ amortized (geometric growth).
- **Master theorem:** $T(n) = aT(n/b) + f(n) \rightarrow$ three cases.
- **Prefix sums:** `rangeSum(L,R) = pref[R+1] - pref[L]`.
- **Two pointers:** sorted array, pair-sum problems $O(n)$.
- **Sliding window:** fixed/variable window, add right, remove left.
- **Binary search:** `lo=0, hi=n-1, mid = lo+(hi-lo)/2`; boundary variants (`lower_bound/upper_bound`); binary search on the answer for min-max problems.
- **Strings:** frequency arrays for anagrams; two pointers for palindromes; substr copies - use `string_view`.
- **Sorting:** merge $O(n \log n)$ stable; quick $O(n^2)$ worst (pivot!); heap $O(n \log n)$ in place; counting/radix linear for small key domains.
- **Dutch national flag** sorts $O(1/2)$ in one pass.

R1.2 Important Formulas

- n elements sorted $\rightarrow$ at most $n \log_2 n$ comparisons (comparison-sort lower bound).
- $\text{Sum } 1..n = n(n+1)/2$; $\text{Sum of squares} = n(n+1)(2n+1)/6$.
- 2^k comparisons distinguish 2^k outcomes; $\log_2(n!) \sim n \log_2 n - 1.44n$.
- Integer sqrt via binary search; *midmid may overflow $\rightarrow$ use long long.*
- Sliding window size k : $O(n)$ with a deque for min/max.

R1.3 Cheat Sheet: Complexity of Common Operations

Container	Access index	Insert	Erase	Find
vector	$O(1)$	$O(1)$ end*	$O(n)$	$O(n)$
list	$O(n)$	$O(1)$ at node	$O(1)$ at node	$O(n)$
deque	$O(1)$	$O(1)$ ends	$O(1)$ ends	$O(n)$
set/map	-	$O(\log n)$	$O(\log n)$	$O(\log n)$
unordered_set/map	-	$O(1)^*$	$O(1)^*$	$O(1)^*$
priority_queue	-	$O(\log n)$	$O(\log n)$ pop	$O(n)$

- amortized / average.

R1.4 Common Interview Patterns So Far

- **Prefix sum pattern:** subarray-sum queries $\rightarrow O(1)$ after $O(n)$ build.
- **Two-pointer pattern:** sorted pair targets, palindromes, merging.
- **Sliding window pattern:** subarray max/min/count-with-constraint.
- **Boundary binary search pattern:** first/last occurrence, lower/upper bound.
- **Binary search on answer pattern:** "minimise the maximum" + greedy check.
- **Frequency table pattern:** anagrams, duplicates, first-unique.
- **Partition pattern (quicksort):** kth-smallest via quickselect.
- **Divide and conquer pattern:** merge sort; count inversions during merge.

R1.5 Multiple-Choice Questions

1. Which complexity is smallest? a) $O(2^n)$ b) $O(n^2)$ c) $O(n \log n)$ d) $O(\log n)$
 2. `vector::push_back` worst-case single operation is: a) $O(1)$ b) $O(n)$ c) $O(n \log n)$
 3. Binary search on a sorted array of 10^6 elements needs at most about: a) 10 b) 20 c) 10^6 d) 1000
 4. In merge sort the recurrence is: a) $T(n)=T(n-1)+n$ b) $T(n)=2T(n/2)+n$ c) $T(n)=T(n/2)+1$
 5. `lower_bound` on a sorted array returns: a) first element $> x$ b) first element $\geq x$ c) index of x or -1
 6. Which sort is stable? a) quicksort b) heapsort c) merge sort d) selection sort
 7. Sliding window maximum with a deque is: a) $O(nk)$ b) $O(n \log k)$ c) $O(n)$ d) $O(k)$
 8. The trie search cost for a word of length L is: a) $O(1)$ b) $O(L)$ c) $O(\log L)$
 9. Quickselect for the kth smallest has expected time: a) $O(n)$ b) $O(n \log n)$ c) $O(n^2)$ d) $O(\log n)$
 10. Two pointers on a sorted array for pair-sum is: a) $O(n^2)$ b) $O(n \log n)$ c) $O(n)$ d) $O(\log n)$
- Answers:** 1-d, 2-b, 3-b, 4-b, 5-b, 6-c, 7-c, 8-b, 9-a, 10-c.

R1.6 True / False Questions

1. $O(n)$ is always faster than $O(n^2)$ for small n . (F - constants matter for small n)
2. Binary search requires a sorted array. (T)
3. `std::sort` is always $O(n \log n)$ worst case. (F - introsort is, but the standard only guarantees $O(n \log n)$; it is in practice)
4. Merge sort uses $O(n)$ extra space. (T)
5. Quicksort is stable. (F)
6. Insertion sort runs in $O(n)$ on an already sorted array. (T)
7. A hash table gives $O(1)$ worst-case lookup. (F - expected/amortized)
8. Prefix sums answer range-sum queries in $O(1)$. (T)
9. `std::unordered_map` preserves insertion order. (F)
10. Radix sort can beat $O(n \log n)$. (T)

R1.7 Short-Answer Questions

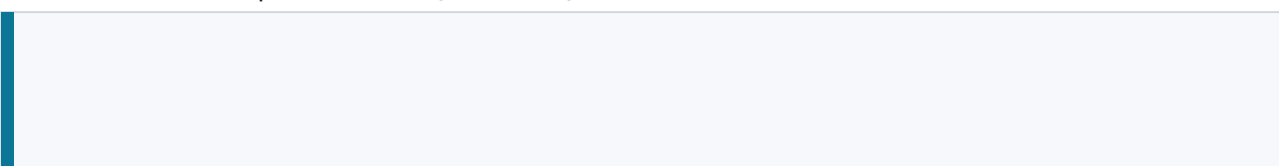
1. Why is $\text{lo} + (\text{hi} - \text{lo}) / 2$ preferred over $(\text{lo} + \text{hi}) / 2$?
2. When would you choose a hash map over a balanced BST, and vice versa?
3. What makes insertion sort fast on nearly-sorted arrays?
4. Explain why building a heap from an array is $O(n)$ while inserting n elements is $O(n \log n)$.
5. Why does binary search on the answer work for "split array largest sum"?

R1.8 Debugging Exercises

1. `for (int i = 0; i < v.size() - 1; ++i) ... with v.size() == 0.` Bug?
2. `int mid = (lo + hi) / 2; with lo=hi=210^9.` Bug?
3. `while (lo <= hi) { mid=...; if (a[mid] < x) lo = mid; else hi = mid - 1; }` - can it loop forever?
4. Sorting an array of structs by one field with `std::sort` and comparing with `==`.
5. `v.erase(v.begin() + 2)` inside a loop over `v`.

R1.9 Code Tracing Exercises

1. What does this print for `v = {3,1,4,1,5}`?



1. Trace `lower_bound` on `a={1,3,3,3,5}` for `x=3`. What index does it return?
2. What is the output of the two-pointer pair-sum check for `K=8` on `{1,2,4,5,7}`?

R1.10 Answers to Short/Debug/Trace

Short answers: 1. Avoids signed integer overflow of `lo+hi`; also standard idiom. 2. Hash map: $O(1)$ average, no order. BST: sorted iteration, `lower_bound`, guaranteed $O(\log n)$ worst case. 3. Almost-sorted input: each element moves only a few positions; the inner while runs $O(1)$ average $\rightarrow O(n)$ total. 4. `buildHeap` applies `siftDown` from level `h-1`; total work telescopes to $O(n)$. Insert does `siftUp` each time, costing $O(\log n)$ per element $\rightarrow O(n \log n)$. 5. The predicate "can the array be split into k subarrays with max sum $\leq X$ " is monotone in X , so binary search applies; the greedy check is $O(n)$.

Debugging: 1. `v.size() - 1` is `size_t`: with size 0 it is `SIZE_MAX`, the loop runs forever (or UB). Cast to int or use `(int)v.size() - 1`. 2. `lo+hi` overflows 32-bit int. Use `lo + (hi-lo)/2`. 3. Yes: when `a[mid] < x`, `lo = mid` does not shrink the range if `lo==mid`; use `lo = mid + 1`. 4. `==` compares addresses/identity for structs; define operator`==` (or compare fields) if you mean value equality. 5. Erasing shifts indices; the loop skips the element that moved into position 2. Use iterators + `erase` return or `erase` from back to front.

Code tracing: 1. `x ^= v[i] ^ (i+1)` accumulates `(v[i] XOR (i+1))`: for `v={3,1,4,1,5}`, the XOR of $(3^1)^{(1^2)}^{(4^3)}^{(1^4)}^{(5^5)} = 3^1 \wedge 1^2 \wedge 4^3 \wedge 1^4 \wedge 0 =$ compute pairwise: bits cancel to a single value; do the arithmetic on paper - the trick is that `i+1` runs 1..n and `v` contains 1..5 with 1 duplicated and missing 2 -> the result is the missing number (2). 2. `lower_bound(3)` on `{1,3,3,3,5}` returns index 1. 3. `K=8: i=0,j=4 -> 1+7=8 -> true`.

R1.11 What to Revise Before Moving On

- You should be able to *write* binary search and its boundary variants in 60 seconds without bugs.
- You should be able to state the time/space of every operation on every container in the cheat sheet.
- You should be able to explain `buildHeap = O(n)` and the comparison-sort lower bound in an interview.
- Do the debugging exercises again until each bug is instant.

R1.12 Progress Check

If you can solve: Two Sum (hash), Longest Substring Without Repeats (sliding window), Find First and Last Position (boundary binary search), Merge Sorted Arrays (two pointers), Count Inversions (merge sort) - you have solid command of modules 1-4. Move on to recursion and linear data structures with confidence.

MODULE

Graphs I

Modeling relationships: representations, BFS, DFS, connectivity

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

18. Graphs: Representations and Fundamentals

18.1 Introduction

A **graph** is a set of **vertices** (nodes) connected by **edges**. Graphs model networks of any kind: roads, social connections, web links, dependencies, flows. Mastering graphs unlocks an enormous class of problems - reachability, shortest paths, connectivity, ordering, and matching.

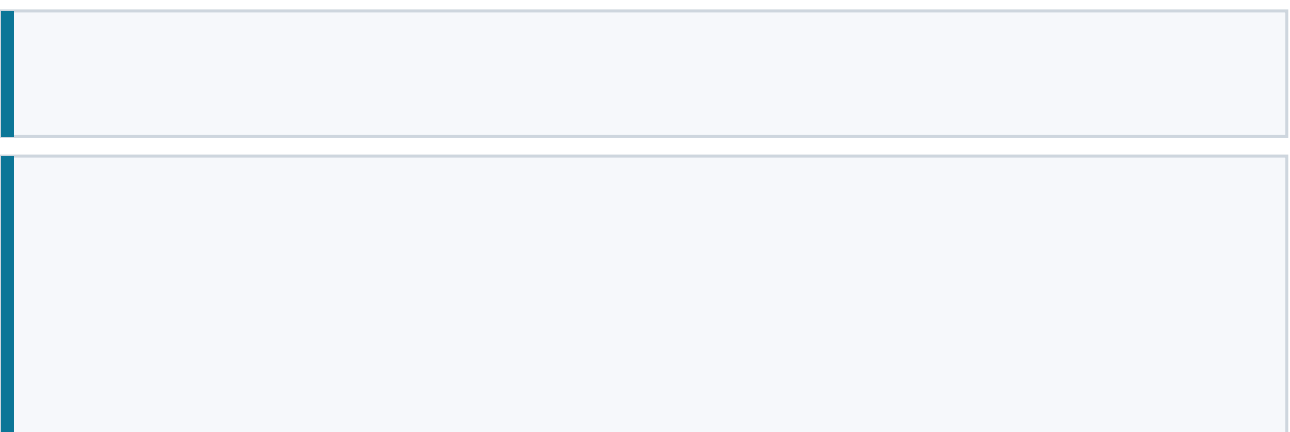
Graph vocabulary

- **Undirected edge:** $\{u, v\}$ - connection both ways.
- **Directed edge:** (u, v) - from u to v (arc).
- **Degree:** number of incident edges (in-degree/out-degree for directed).
- **Path / cycle:** sequence of edges; a cycle returns to its start.
- **Connected component:** maximal set of vertices mutually reachable.
- **Weighted graph:** each edge has a cost.
- **Dense vs sparse:** dense $\sim n^2$ edges, sparse $\sim n$ edges.

18.2 Theory: Representations

18.2.1 Adjacency list - the default

For each vertex, a list of its neighbours. Space $O(V + E)$.



18.2.2 Adjacency matrix - dense graphs

$\text{mat}[u][v] = \text{weight (or 1)}$. Space $O(n^2)$; $O(1)$ edge lookup.

18.2.3 Edge list - simple and sortable

Representation	Space	Edge lookup	Iterate neighbours	Best for
adjacency list	$O(V+E)$	$O(\text{deg})$	$O(\text{deg})$	sparse, BFS/DFS/Dijkstra
adjacency matrix	$O(V^2)$	$O(1)$	$O(V)$	dense, Floyd-Warshall
edge list	$O(E)$	$O(E)$	$O(E)$	Kruskal, Bellman-Ford

18.3 Real-World Analogy

A graph is a **subway map**. Stations are vertices, tracks are edges. You do not care about the physical layout - only who connects to whom. The map (adjacency list) lets you find all stations reachable in one stop, plan routes (shortest paths), and see if the whole city is connected.

18.4 Why the representation matters

- BFS/DFS iterate a vertex's neighbours: adjacency lists are ideal.
- Checking "is u adjacent to v" repeatedly: matrix wins.
- Sorting edges by weight: edge list.

The same graph problem can have very different practical performance depending on the representation - always choose before coding.

18.5 Common Mistakes

- Forgetting to add BOTH directions for undirected edges.
- Using an adjacency matrix for a sparse graph ($n = 10^5 \rightarrow$ impossible).
- 1-based vs 0-based indexing confusion.
- Modifying a graph while iterating over it.
- Ignoring self-loops and parallel edges unless the problem forbids them.

18.6 Applications

- Social networks (friendship), road/shortest-route maps, the web (hyperlink graph), dependency graphs (build systems), circuit design, network flow.

18.7 Interview Tips

- Ask: directed or undirected? weighted? possibly disconnected? self-loops?
- Say "I'll use an adjacency list since the graph is sparse."
- For counting problems, name the traversal you will use (BFS for distances, DFS for reachability/cycles).

- Draw the graph; test small cases by hand.

18.8 Summary

- Graph = V vertices + E edges; directed/undirected/weighted variants.
 - Adjacency list (default), matrix (dense), edge list (sortable).
 - Choose the representation based on the graph's shape and your algorithm.
-

19. Graph Traversal: BFS and DFS

19.1 Introduction

Traversal = visiting every vertex (and edge) exactly once, in a structured order. The two workhorses are **Depth-First Search (DFS)**, which follows one path as far as possible before backtracking (natural for recursion), and **Breadth-First Search (BFS)**, which explores level by level (natural for a queue). Together they solve reachability, connectivity, cycle detection, shortest paths in unweighted graphs, bipartite checking, and topological order.

19.2 DFS - recursion (or explicit stack)

DFS properties

- Visits a whole connected component in one call.
- Produces a **DFS tree** (the edges used); non-tree edges are back/cross edges.
- Detects cycles: a back edge to an ancestor in an undirected graph.
- Space $O(V)$ recursion; time $O(V + E)$.

19.3 BFS - the queue

Why BFS gives shortest paths

Vertices are dequeued in non-decreasing distance order, so the first time a vertex is discovered, it is via a shortest path. That is why BFS (not DFS) answers "minimum number of moves".

19.4 Connected Components

Count components with a loop over all vertices.

19.5 Cycle Detection

- **Undirected:** during DFS, if we reach an already-visited vertex that is not the parent, a cycle exists.
- **Directed:** track recursion stack; a back edge to a vertex on the current DFS stack indicates a cycle.

19.6 Bipartite Graph Check

A graph is bipartite if its vertices can be 2-coloured so that every edge joins different colours. Odd-length cycles make a graph non-bipartite.

19.7 Flood Fill and Grid BFS

Grids are graphs with 4- or 8-neighbour adjacency (see Chapter 11 BFS example). DFS flood-fill counts islands; BFS computes distances.

19.8 Complexity

Both DFS and BFS: $O(V + E)$ time, $O(V)$ space (visited + recursion/queue).

19.9 Common Mistakes

- Forgetting the visited check -> infinite loops / exponential blow-up.
- Using DFS for shortest path in an unweighted graph.
- Not resetting visited between searches.
- BFS re-processing nodes: dist check must happen at push time, not pop time.
- Off-by-one with 1-based vertices.

19.10 Applications

- Web crawling (BFS frontier), social "degrees of separation", game AI (shortest moves), puzzle solvers (minimum steps), maze solving, garbage collection (DFS).

19.11 Interview Tips

- "Minimum steps/moves/levels" -> BFS. "Connectivity/cycles/topological" -> DFS.
- For grids, use dr/dc arrays; check bounds before visiting.
- Say " $O(V+E)$ " and name which traversal and why.
- Practise: number of islands, 01 matrix distances, rotten oranges, knight moves, word ladder.

19.12 Summary

- DFS: stack/recursion, goes deep; components, cycles, topological order.
- BFS: queue, level order; shortest distances in unweighted graphs.

- Both $O(V+E)$. Choose by what you need to compute.
-

Practice Set: Graphs, BFS and DFS

Beginner Problems

1. Build an adjacency list from edge input and print it.
2. Count connected components.
3. Check reachability from a given source (DFS and BFS).
4. BFS distance from a source on an unweighted graph.
5. Detect a cycle in an undirected graph.

Intermediate Problems

1. Check whether a graph is bipartite (2-colouring).
2. Number of islands in a grid (flood fill).
3. Rotting oranges (multi-source BFS).
4. Detect a cycle in a directed graph.
5. Find all nodes within k steps of a source (BFS depth limit).

Advanced Problems

1. Shortest path in a binary grid with obstacles (0-1 BFS / BFS on 0/1 weights).
2. Word ladder: minimum transformations from beginWord to endWord.
3. Minimum time to walk through a maze (BFS on states).
4. Find bridges in an undirected graph (Tarjan's algorithm).
5. 01 Matrix: distance to the nearest 0 for every cell (multi-source BFS).

Interview-Style Problems

1. "Find the shortest path from the entrance to the exit in a maze (print it)."
2. "Clone a graph" (deep copy of a connected graph).

Competitive Programming Problems

1. Given a knight on a chessboard, minimum moves to a target (BFS).
2. Find the shortest path where you may break at most k walls (BFS on (cell,k)).

Challenge Exercises

- Implement DFS iteratively with an explicit stack that mimics recursion exactly (including post-order).
 - Compute all reachable nodes from every node without $O(V(V+E))$ time (SCC idea - Chapter 24).
 - Prove that BFS distances are correct by induction on the distance level.
-

Solutions: Graphs, BFS and DFS

Beginner Solutions

1-5. Standard implementations from Sections 19.2-19.5.

Intermediate Solutions

1. BFS 2-colouring (Section 19.6).
2. Flood fill: for each unvisited '1', DFS mark the whole island, count++.
3. Multi-source BFS from all rotten oranges; answer = last layer depth.
4. State-based DFS (Section 19.5) or Kahn's algorithm (in-degree count).
5. BFS but stop expanding beyond depth k; track depth per node.

Advanced Solutions

1. 0-1 BFS: deque; move to empty cell costs 0 (push_front), to wall costs 1 (push_back).
2. BFS where each word is a node and edges connect words differing by one char (build transitions lazily).

1. BFS over (row, col) states; some problems need (row, col, keys/mask) states.
2. Tarjan: $\text{low}[u] = \min(\text{discovery}, \text{low of children})$; an edge (u,v) is a bridge if $\text{low}[v] > \text{disc}[u]$.
3. Multi-source BFS: seed all zeros as distance 0; BFS outward.

Interview-Style Solutions

1. BFS; store parent of each cell; reconstruct by walking back from the exit.
2. BFS/DFS over the graph, creating clones keyed by the original node.

Competitive Programming Solutions

1. BFS on 64 cells; knight moves are 8 offsets.
2. BFS on (cell, remainingWalls): expand to wall if remainingWalls > 0.

Challenge Solutions

- Iterative DFS: push (node, nextChildIndex); pop when done -> mimics post-order.

- SCC: Kosaraju/Tarjan (Chapter 24) do it in $O(V+E)$.
- BFS correctness: prove all nodes at distance d are discovered before any at distance $d+1$ (queue order invariant).

20. Shortest Path Algorithms

20.1 Introduction

Given a weighted graph, find the minimum-cost path between two vertices (or from one vertex to all others). The right algorithm depends on the graph's shape:

- **Unweighted** -> BFS (Chapter 19).
- **Non-negative weights** -> Dijkstra (with a priority queue).
- **Negative weights, no negative cycles** -> Bellman-Ford.
- **All pairs** -> Floyd-Warshall.
- **DAG** -> topological-order DP (also handles negatives).

20.2 Dijkstra's Algorithm - $O((V+E) \log V)$

Greedy: repeatedly relax the nearest unprocessed vertex.

Relaxation

Relaxing edge (u, v) means: if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$. Dijkstra repeatedly relaxes from the closest unvisited vertex. Because all weights are non-negative, the closest unvisited vertex's distance is final when popped.

Dijkstra fails with negative edges

With a negative edge, an already-finalized vertex could be improved later. Negative weights require Bellman-Ford (or SPFA). Dijkstra on negative cycles: never.

20.3 Dry Run: Dijkstra on the graph below from A

Simpler example:

20.4 Bellman-Ford - $O(V \cdot E)$, handles negatives

Relax all edges $V-1$ times; a V -th pass that improves something detects a negative cycle reachable from the source.

SPFA

The queue-based SPFA is Bellman-Ford in practice: relax edges of recently changed vertices. Fast average, but $O(V \cdot E)$ worst case - use it carefully in contests.

20.5 Floyd-Warshall - all pairs, $O(V^3)$

Detect negative cycles: $\text{dist}[i][i] < 0$ after the loop.

20.6 Shortest Path in a DAG - $O(V + E)$ via topological order

20.7 Which algorithm when?

Scenario	Algorithm	Complexity
unweighted	BFS	$O(V+E)$
non-negative weights	Dijkstra	$O((V+E) \log V)$
negative weights (no neg. cycle)	Bellman-Ford	$O(V \cdot E)$
all pairs	Floyd-Warshall	$O(V^3)$
DAG (any weights)	topo + relax	$O(V+E)$
negative cycle detection	Bellman-Ford extra pass	$O(V \cdot E)$

20.8 Real-World Analogy

Dijkstra is the **GPS navigator** that always expands the "closest not-yet-fixed" city first - like pouring water over terrain: it reaches nearer places first. Bellman-Ford is the **cautious traveller** who tries every road, over and over, until no improvement is possible - slower but works even with toll roads that pay you (negative edges).

20.9 Common Mistakes

- Using Dijkstra with negative edges.
- Forgetting the "stale entry" skip (`if (d > dist[u]) continue;`) in Dijkstra.
- INF too small; use `LLONG_MAX/4` style sentinels and check overflow.
- Bellman-Ford relaxing from uninitialized `dist[u] == INF`.
- Floyd-Warshall wrong loop order (`k` must be outermost).
- Not realizing Dijkstra with an adjacency matrix is $O(V^2)$ (fine for dense).

20.10 Applications

- GPS routing, network routing protocols (OSPF = Dijkstra), ride-hailing ETA.
- Social-network influence spread, game pathfinding ($A = \text{Dijkstra} + \text{heuristic}$).
- Currency arbitrage detection (Bellman-Ford on log exchange rates, negative cycle).
- Trip planning with constraints.

20.11 Interview Tips

- The moment weights are non-negative and you need shortest path -> Dijkstra.
- For "minimum cost with k stops" -> Bellman-Ford (k relaxation rounds).

- For "all pairs" on a small graph $\rightarrow$ Floyd-Warshall; say " $O(V^3)$ ".
- Mention A *when there is a good heuristic to the goal*.
- Practise: network delay time, cheapest flights within K stops, path with max probability.

20.12 Summary

- Dijkstra: greedy + priority queue, non-negative only, $O((V+E) \log V)$.
 - Bellman-Ford: relax all edges $V-1$ times; handles negatives; detects cycles.
 - Floyd-Warshall: all pairs, $O(V^3)$, k-outermost.
 - DAG: topological order gives $O(V+E)$ (and DP solutions).
-

Practice Set: Shortest Paths

Beginner Problems

1. BFS distances on an unweighted graph (Chapter 19).
2. Dijkstra on a small graph; trace every priority-queue pop.
3. Detect whether a graph has a negative cycle.
4. Compute the path (not just distance) with Dijkstra.
5. Shortest path in an unweighted grid (BFS).

Intermediate Problems

1. Network delay time (Dijkstra from source, answer = max distance).
2. Cheapest flights within K stops (Bellman-Ford for K+1 rounds).
3. Path with maximum probability (Dijkstra on log-probabilities / max-heap).
4. Second shortest path from s to t.
5. Minimum cost path in a grid with 0/1 costs (0-1 BFS).

Advanced Problems

1. Find the shortest cycle in an undirected weighted graph.
2. All-pairs shortest paths with Floyd-Warshall; detect negative cycles.
3. "Design the graph": reconstruct the path with k edges having the minimum total weight.
4. Reachability DP in a DAG for "paths from every node".
5. Currency arbitrage (Bellman-Ford on exchange rates with negative-log edges).

Interview-Style Problems

1. "Given a weighted graph, find the minimum distance from source to target where you can make at most one edge free."
2. "Course schedule: can all courses be finished (topological sort of a DAG)?"

Competitive Programming Problems

1. The "travelling salesman in a small DAG" - shortest Hamiltonian path via DP.
2. 0-1 BFS on a grid with teleport cells.

Challenge Exercises

- Implement Dijkstra with an adjacency matrix and argue when it is better.
 - Prove Dijkstra's correctness by induction on the number of finalized vertices.
 - Implement SPFA and compare with Dijkstra on random graphs.
-

Solutions: Shortest Paths

Beginner Solutions

1-3,5. Direct implementations (Sections 20.2-20.4). 4. Store $\text{parent}[v] = u$ when relaxing; reconstruct from t backward.

Intermediate Solutions

1. Dijkstra; answer = max over reachable dist (if any unreachable $\rightarrow -1$).
2. Bellman-Ford restricted to $K+1$ rounds with a copy of the dist array per round.

1. Max-heap of (probability, node) - standard Dijkstra with multiplication and $>$ flipped.
2. Keep two distances per node (best, second-best); a modified Dijkstra relaxes both.
3. 0-1 BFS (Chapter 19, Solution 11).

Advanced Solutions

1. For each edge (u,v) , run Dijkstra from u excluding that edge; $\text{cycle} = \text{dist}[v] + w$. Min over edges. $O(E \cdot (V+E) \log V)$.
2. Floyd-Warshall (Section 20.5); check $\text{dist}[i][i] < 0$.
3. DP over (node, used edges) with min cost; or Bellman-Ford layered.
4. Topological DP: number of paths from each node = $1 + \text{sum of children}$.
5. Weight = $-\log(\text{rate})$; negative cycle = arbitrage opportunity.

Interview-Style Solutions

1. Dijkstra on an augmented graph with state (node, couponsUsed in $\{0,1\}$).

1. Kahn's topological sort; if processed count $< n$ there is a cycle.

Competitive Programming Solutions

1. DP over subsets (bitmask) for tiny n (TSP): $\text{dp}[\text{mask}][\text{last}]$.
2. 0-1 BFS; teleport edges cost 0.

Challenge Solutions

- Matrix Dijkstra: $O(V^2)$ is better than $O((V+E)\log V)$ when $E \sim V^2$ (dense).
- Proof: when a vertex is finalized (popped with its smallest tentative distance), no path through other unfinished vertices can improve it (all edges ≥ 0).
- SPFA usually beats Dijkstra on sparse graphs with small weights but has no guaranteed bound.

21. Minimum Spanning Trees and Union-Find

21.1 Introduction

A **spanning tree** connects all vertices of an undirected graph with exactly $V-1$ edges and no cycles. A **minimum spanning tree (MST)** is the spanning tree with the smallest total edge weight - the cheapest way to keep everything connected. Two classical algorithms solve it: **Kruskal** (sort edges + Union-Find) and **Prim** (grow from a seed with a priority queue).

The secret weapon behind Kruskal is the **Union-Find** (Disjoint Set Union, DSU) data structure, which tracks and merges connected sets in nearly $O(1)$ amortized.

21.2 Union-Find (Disjoint Set Union)

The two optimisations

- **Path compression:** point every node on the lookup path straight at the root.
 - **Union by rank:** attach the shorter tree under the taller one.
- Together they give an inverse-Ackermann amortised time per op - effectively $O(1)$.

21.3 Dry Run: DSU

21.4 Kruskal's Algorithm - $O(E \log E)$

Sort edges by weight and add each edge that connects two different components.

21.5 Prim's Algorithm - $O((V+E) \log V)$

Start from any vertex; repeatedly attach the cheapest edge from the tree to a vertex not yet in it.

21.6 MST Properties

Cut property

For any cut (a split of vertices), the minimum-weight edge crossing the cut is in some MST. Prim adds exactly the minimum crossing edge of the tree's cut; Kruskal adds minimum crossing edges of component cuts.

- Cycle property: the heaviest edge of any cycle is never in an MST.
- An MST exists iff the graph is connected.
- MST is not necessarily unique (equal-weight edges).
- Number of MSTs: can be counted with Kruskal + component counting.

21.7 Real-World Analogy

Think of connecting villages with roads of different construction costs. Kruskal is like **hiring a contractor who builds the cheapest road first, regardless of location**, skipping any road that would form a loop. Prim is like **growing one construction crew outward from the first village**, always taking the cheapest road from the current territory. Union-Find is the **shared paperwork**: it instantly answers "are these two villages already connected?"

21.8 Applications

- Network design: electrical grids, water pipes, fibre, transportation.
- Approximate TSP: doubling an MST gives a 2-approximation.
- Clustering: remove the $K-1$ heaviest MST edges $\rightarrow$ K clusters.
- Image segmentation, single-linkage clustering.

21.9 Common Mistakes

- Kruskal: forgetting to check `same(u, v)` $\rightarrow$ cycles.
- Prim: forgetting the `inTree` skip; stale entries in the priority queue.
- DSU find without path compression $\rightarrow$ $O(n)$ chains.
- Union by rank combined incorrectly with `parent[b] = a` (must use roots).
- Off-by-one: an MST has exactly $V-1$ edges; if the graph is disconnected you will not reach it.

20.? Interview Tips

- Say "I'll use Kruskal with Union-Find" when edges are the natural input.
- Say "Prim with a priority queue" when the adjacency list is given.
- State the complexity: Kruskal $O(E \log E)$ (sorting dominates), Prim $O((V+E) \log V)$.
- Know: DSU also solves connected-components queries dynamically and cycle detection in undirected graphs.
- Practise: connect all cities with minimum cost, MST after deleting an edge, critical connections (bridges, Chapter 24).

21.10 Summary

- Union-Find: find + unite, path compression + union by rank, $\sim O(1)$ amortised.
 - Kruskal: sort edges, add safe ones - $O(E \log E)$.
 - Prim: grow a tree with a priority queue - $O((V+E) \log V)$.
 - Both rely on the cut property; pick by input shape.
-

Practice Set: MST and Union-Find

Beginner Problems

1. Implement DSU and answer "are a and b connected?" queries.
2. Count the number of connected components after each union.
3. Run Kruskal by hand on a 5-node graph.
4. Run Prim by hand on the same graph.
5. Detect a cycle in an undirected graph with DSU (as you add edges).

Intermediate Problems

1. MST total weight with Kruskal.
2. MST with Prim on an adjacency list.
3. Find the second MST (remove each MST edge and re-run).
4. Number of connected components of a graph after removing k edges.
5. Maximum spanning tree (negate weights or use max-heap).

Advanced Problems

1. MST with "virtual node" trick (connect to a special node with special edges).
2. Minimum cost to connect all points (complete graph of coordinates; sort distances).
3. Dynamic connectivity: process edge additions and connectivity queries online.
4. Find the MST of a graph where some edges must be included.
5. Critical edges: which edges appear in EVERY MST?

Interview-Style Problems

1. "Connect all cities with minimum total road cost (Kruskal)."
2. "Detect a cycle in an undirected graph" (DSU, $O(\alpha)$).

Competitive Programming Problems

1. Kruskal with DSU + counting: the number of MSTs of a small graph.
2. "Bottleneck" spanning tree: minimize the maximum edge weight (it is the MST's max edge).

Challenge Exercises

- Prove the cut property and the cycle property.
 - Prove DSU's inverse-Ackermann bound (read the proof, then explain it).
 - Implement Kruskal without sorting (counting sort by weight for small weights).
-

Solutions: MST and Union-Find

Beginner Solutions

1-2. DSU with a count field decremented on each successful unite.

3-4. Hand-traces (Section 21.4 style). 5. As edges are added, a `same(u, v)` that returns true means a cycle.

Intermediate Solutions

6-7. Section 21.4-21.5. 8. Run Kruskal once for MST weight and edge set; for each MST edge, re-run Kruskal forbidding it, take the min over runs. 9. Process removals backwards as additions with DSU. 10. Sort descending (or use the same code with negative weights).

Advanced Solutions

1. Add an imaginary node connected to each city with the "build a new plant" cost; run MST.
2. Kruskal on all $O(n^2)$ pair distances, but only keep the n closest per point (prim-like) or just $O(n^2 \log n)$ for small n .
3. DSU supports additions and connectivity queries natively - amortised $\sim O(1)$.
4. Force-include those edges by uniting their endpoints first, then Kruskal.
5. An edge is in every MST iff it is the unique minimum-weight edge crossing some cut. Check via component counting during Kruskal over equal-weight groups.

Interview-Style Solutions

16-17. Straightforward Kruskal / DSU cycle check.

Competitive Programming Solutions

1. Group equal-weight edges in Kruskal; count connected assignments within each group (matrix-tree theorem or brute force for small graphs).
2. The max edge of the MST; Kruskal naturally minimizes it (bottleneck property).

Challenge Solutions

- Cut property: exchange argument - if a tree avoids the min cut edge, swap it in.

- Cycle property: dual of the cut property.
- Counting sort by weight when weights are small integers gives $O(E)$ Kruskal.

22. Topological Sorting

22.1 Introduction

A **topological order** of a **directed acyclic graph (DAG)** lists the vertices so that every edge (u, v) has u before v . It is the "dependency order": prerequisites first, then dependents. A DAG is required - a cycle makes ordering impossible (and you can detect that with the same algorithms).

22.2 Theory: When is it possible?

- A DAG always has at least one topological order (in fact, at least one source - a vertex with in-degree 0).
- A graph with a directed cycle has none.
- The order is not unique in general.

22.3 Kahn's Algorithm - $O(V + E)$

Repeatedly remove a source (in-degree 0) and decrease its neighbours' in-degrees.

Smallest-order variant

Use a `priority_queue` instead of a queue to get the lexicographically smallest topological order (classic competitive problem).

22.4 Dry Run: Kahn's

22.5 DFS-based Topological Sort

Post-order DFS reversed: finish order from a DFS on the graph, then reverse.

The state trick for cycle detection

A plain `vis` cannot tell you if you are back on the *current* DFS path. Use the three-state array (0/1/2) from Section 19.5 to detect cycles in a directed graph while running the DFS topo sort.

22.6 Longest Path in a DAG (via topo order)

With no cycles, a simple linear DP over the topological order computes the longest path (or shortest with negatives) - Section 20.6 uses the same idea.

22.7 Real-World Analogy

Topological sort is **planning your university semester schedule**. Some courses have prerequisites: you cannot take Algorithms before Discrete Math. You lay out semesters so every course comes after its prerequisites. If a prerequisite cycle existed (AI requires Algorithms, Algorithms requires AI), the schedule is impossible.

22.8 Applications

- Build systems (make), package managers (apt/npm dependency resolution).
- Course prerequisites, task scheduling, job sequencing.
- Instruction scheduling / circuit evaluation order.

- Finding the longest path in a DAG (critical path analysis in projects).

22.9 Common Mistakes

- Running topo sort on a cyclic graph without detection (Kahn's catches it: `order.size() != n`).
- Confusing DFS post-order with the topo order (must reverse).
- Not handling disconnected DAGs (start from every source/unvisited node).
- Using BFS (wrong for this - BFS is for levels, not dependencies).
- Forgetting the priority-queue variant when "smallest order" is required.

22.10 Interview Tips

- "Ordering with prerequisites" -> topological sort, immediately.
- Mention Kahn's if you want cycle detection built in.
- For DAG DP (longest path, number of paths), say "process in topological order".
- Practise: course schedule I and II, alien dictionary, minimum height trees.

22.11 Summary

- Topological order exists iff the graph is a DAG.
 - Kahn: remove in-degree-0 nodes; a leftover means a cycle. $O(V+E)$.
 - DFS: reverse post-order. $O(V+E)$.
 - Enables $O(V+E)$ DP on DAGs (longest paths, reachability counts).
-

Practice Set: Topological Sorting

Beginner Problems

1. Compute a topological order with Kahn's on a sample DAG.
2. Detect a cycle in a directed graph.
3. Compute in-degrees for every node.
4. Print the lexicographically smallest topological order.
5. Reverse a DAG (transpose) and topologically sort it.

Intermediate Problems

1. Course Schedule: can all courses finish (LeetCode 207).
2. Course Schedule II: return an order (LC 210).
3. Alien Dictionary: derive the ordering of letters.
4. Number of paths from node 0 to node n-1 in a DAG.
5. Longest path in a DAG (critical path).

Advanced Problems

1. Find all topological orders of a small graph (backtracking).
2. Minimum time to complete all tasks given durations and dependencies (DP on topo order).
3. Longest increasing path in a matrix (DAG over cells).
4. Lexicographically smallest order with constraints (heap + Kahn).
5. Sequence reconstruction: is a sequence consistent with a given DAG?

Interview-Style Problems

1. "Given prerequisites pairs, return a valid order to take the courses."
2. "Find if there is a cycle in a directed graph" (topological / state DFS).

Competitive Programming Problems

1. Fox and the tree: topological sort with in-degree priority for min-height tree.
2. Minimum steps to visit all nodes of a DAG respecting edges.

Challenge Exercises

- Prove that reverse post-order of DFS is a valid topological order.
 - Prove Kahn's algorithm correctness (invariant: sources have no unmet predecessors).
 - Design an $O(V+E)$ algorithm to count the number of topological orders (answer: DP over subsets for small n , or use $DP[n] = \text{sum over sources}$).
-

Solutions: Topological Sorting

Beginner Solutions

1-3. Section 22.3 implementations.

1. Transpose then Kahn/DFS - same order property (reverse of original).

Intermediate Solutions

6-7. Kahn's with cycle detection; LC 210 returns the order. 8. Build directed edges from letter order comparisons; topo-sort; handle conflicts as cycles. 9. DP on topo order: $ways[v] += ways[u]$.

1. $dist[v] = \max(dist[v], dist[u] + w)$ (Section 22.6).

Advanced Solutions

1. Backtracking: at each step choose any remaining source; recurse; output when empty.
2. $finish[v] = dur[v] + \max \text{ over dependencies of } finish[u]$.
3. Cell $(r,c) \rightarrow$ neighbours with greater value; DFS with memoised DP on topo order of the DAG.
4. Kahn with a min-heap (Section 22.5 solution 4).
5. Simulate the sequence: check that every consecutive pair is an edge and that in-degrees align (needs care) - the sequence is the unique topo order iff it is an order AND at each step the next element is the only source.

Interview-Style Solutions

16-17. Kahn's / state DFS as above.

Competitive Programming Solutions

1. Build the tree, root candidates via min-in-degree removal (two BFS/DFS from diameter ends).
2. DP on topo order: earliest completion times.

Challenge Solutions

- Reverse post-order proof: every edge (u,v) means v finishes before u in DFS post-order, so u appears before v in the reversed list.
- Kahn invariant: nodes in the queue have no unprocessed predecessors.
- Counting orders: $dp[mask] =$ ways to arrange the subset $mask$; for each source in the remaining set, $dp[mask \cup \{v\}] += dp[mask]$. $O(2^n \cdot n)$.

23. Advanced Graph Algorithms

23.1 Introduction

These algorithms solve the "harder" graph problems: splitting a directed graph into strongly connected pieces, finding the fragile edges/vertices whose removal disconnects the graph, and finding trails that use every edge exactly once. They share a foundation in DFS with careful bookkeeping of discovery times.

23.2 Strongly Connected Components - Kosaraju's Algorithm

A **strongly connected component (SCC)** is a maximal set of vertices where every vertex can reach every other. Contracting SCCs turns any directed graph into a DAG - the condensation graph.

Kosaraju in 3 passes

1. Run DFS, record finish order.
 2. Transpose the graph (reverse all edges).
 3. Run DFS on the transpose in reverse finish order; each DFS tree is an SCC.
- Time $O(V + E)$.

Tarjan's SCC (single pass)

Tarjan's algorithm computes SCCs in one DFS using a low-link value and a stack - $O(V+E)$ with no transpose. Slightly trickier; Kosaraju is easier to reproduce in interviews.

23.3 Bridges and Articulation Points - Tarjan's Algorithm

Definitions

- **Bridge:** an edge whose removal increases the number of connected components.
- **Articulation point (cut vertex):** a vertex whose removal (and its edges) increases the number of components.

The key idea is the **low-link value**: $\text{low}[u]$ = the smallest discovery time reachable from the DFS subtree of u , including via at most one back edge.

Bridge rule

Edge (u, v) is a bridge iff $\text{low}[v] > \text{disc}[u]$. For articulation points: - root with ≥ 2 children, OR - non-root u where some child v has $\text{low}[v] \geq \text{disc}[u]$.

23.4 Dry Run: Bridges

23.5 Euler Paths and Circuits

Euler trail/circuit

An **Euler circuit** is a closed trail that uses every edge exactly once and returns to the start. An **Euler trail** uses every edge exactly once (may not close). Hierholzer's algorithm builds one in $O(E)$.

For an undirected graph: - Euler circuit: connected (ignoring degree-0 vertices) AND every vertex has even degree. - Euler trail: connected AND exactly 0 or 2 vertices have odd degree (trail starts/ends at them).

Hierholzer gotchas

- Removing the reverse edge is the fragile part; using a multiset or edge indices avoids $O(E)$ erasure.
- Check the even-degree condition first, else the path is wrong.
- The circuit starts where it ends (choose any start with edges).

23.6 Real-World Analogy

- **SCCs**: a friend circle where everyone can reach everyone (via forwards and introductions) - even if the wider network is sparse, inside a circle every member reaches every other.
- **Bridges**: the only bridge over a river - if it is destroyed, the island is cut off; a town may be an articulation point if all traffic passes through it.

- **Euler:** a **snowplow** that must cover every street exactly once and return to the depot; or drawing a figure without lifting the pen or retracing a line.

23.7 Applications

- SCC: social circles, web link analysis (condensation DAG), compiler optimizations, 2-SAT solving.
- Bridges/articulation: network reliability (telecom, road, power), single point of failure analysis ("critical connections").
- Euler: route inspection (postman problem variant), DNA fragment assembly, de Bruijn sequences, word-chain puzzles.

23.8 Common Mistakes

- Kosaraju: forgetting the transpose; iterating finish order in the wrong direction.
- Tarjan: comparing $\text{low}[v] > \text{disc}[u]$ (bridge) vs $\text{low}[v] \geq \text{disc}[u]$ (articulation) and root special-casing.
- Not skipping the parent edge by index (multi-edge graphs break " $v \neq \text{parent}$ ").
- Euler: forgetting even-degree check; missing odd-degree count of exactly 0 or 2.
- Treating SCC algorithms as connected-components algorithms on undirected graphs (they are directed-graph tools).

23.9 Interview Tips

- "Critical connections / removing this edge disconnects" -> bridges.
- "Single point of failure" -> articulation points.
- "Cover every edge exactly once" -> Euler; state the degree condition first.
- "SCC / condensation DAG" -> Kosaraju (easy to explain) or Tarjan.
- Complexity to quote: all these are $O(V+E)$.
- Practise: critical connections (LeetCode 1192), 2-SAT, redundant connection.

23.10 Summary

- SCC: Kosaraju (finish order + transpose) or Tarjan (low-link). Condensation is a DAG.
 - Bridges: $\text{low}[v] > \text{disc}[u]$; articulation: $\text{low}[v] \geq \text{disc}[u]$ with root rule.
 - Euler: even degrees for circuit; exactly 0/2 odd for trail; Hierholzer $O(E)$.
 - All are DFS-based, $O(V+E)$.
-

Practice Set: Advanced Graph Algorithms

Beginner Problems

1. Identify SCCs of a small directed graph by hand.
2. Find all bridges in an undirected graph (Tarjan).
3. Find all articulation points.
4. Check if an undirected graph has an Euler circuit.
5. Build an Euler trail for a graph with exactly two odd-degree vertices.

Intermediate Problems

1. Kosaraju: count SCCs and build the condensation DAG.
2. Find the number of source SCCs and sink SCCs.
3. Minimum edges to add to make a directed graph strongly connected.
4. "Critical connections" - return all bridges (LC 1192).
5. Verify an Eulerian circuit exists in a directed graph (in == out degrees).

Advanced Problems

1. 2-SAT: decide satisfiability using SCCs of the implication graph.
2. De Bruijn sequence construction via Hierholzer.
3. Find the minimum number of roads to add so the whole undirected graph becomes icoscedents-free (biconnected).
4. Word chain: order words so each ends where the next begins (Euler).
5. Articulation points with an online (incremental) twist.

Interview-Style Problems

1. "Given a network of servers and cables, find all cables whose failure disconnects the network."
2. "Can you draw this shape without lifting your pen or retracing a line?"

Competitive Programming Problems

1. Pushing all SCCs: minimum number of leaf SCCs to connect everything.
2. "The Palindromic Path": use SCC + DP on condensation DAG.

Challenge Exercises

- Prove Kosaraju's correctness (why the transpose + finish order works).
 - Prove the bridge condition $\text{low}[v] > \text{disc}[u]$.
 - Implement Tarjan's SCC and compare with Kosaraju on random graphs.
 - Prove the Euler degree conditions.
-

Solutions: Advanced Graph Algorithms

Beginner Solutions

1. Hand-compute reachability sets; merge mutually reachable vertices.
- 2-3. Tarjan from Section 23.3 with the articulation special-cases. 4-5. Degree parity check; Hierholzer from Section 23.5.

Intermediate Solutions

1. Kosaraju returns a comp array; condensation edge $u' \rightarrow v'$ if any original edge crosses SCCs.
2. Count SCCs with in-degree 0 / out-degree 0 in the condensation DAG.
3. Answer = $\max(\text{\#source SCCs}, \text{\#sink SCCs})$ (after condensation), with a special case for a single SCC (0 edges).
4. Tarjan bridges; return the edge list.
5. For directed Euler circuit: every vertex $\text{indeg} == \text{outdeg}$ and all edges in one weak component.

Advanced Solutions

1. For each variable x , nodes x and $\neg x$; add implications as edges; satisfiable iff x and $\neg x$ are never in the same SCC.
2. Vertices = all length- $(k-1)$ strings, edges = appending one digit; find Euler circuit $\rightarrow$ the sequence.
3. After removing bridges, the graph is biconnected components; contract each and add $\text{ceil}(\text{leaves}/2)$ edges between leaf components.
4. Words are edges between (first letter, last letter); Euler trail over letters.
5. Incremental articulation: recompute is too slow; use offline divide and conquer or maintain biconnected structures (research-level; mention "incremental biconnectivity is a hard problem").

Interview-Style Solutions

1. Bridges (Tarjan) - LC 1192.
2. Check Eulerian trail conditions; if satisfied, trace it.

Competitive Programming Solutions

1. Condensation DAG; add edges to join sink to source SCCs: $\max(\text{sources}, \text{sinks})$ (single SCC = 0).
2. SCC contraction then DP on the DAG for max path value.

Challenge Solutions

- Kosaraju: if u can reach v in G , then in the transpose v reaches u ; processing in reverse finish order peels off SCCs whose members are all finished before any outside vertex that reaches them.
- Bridge condition: $\text{low}[v] > \text{disc}[u]$ means no path from v 's subtree to u or any ancestor without crossing (u,v) - exactly the definition of a bridge.
- Both SCC algorithms are $O(V+E)$; Tarjan uses one pass and a stack.
- Even-degree condition: entering and leaving a vertex along the trail must pair up incident edges; the two odd vertices (trail) provide the unmatched start/end.

MODULE

Algorithms II

Greedy, Divide and Conquer, Dynamic Programming, Bit Tricks

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

24. Greedy Algorithms

24.1 Introduction

A **greedy algorithm** makes the locally optimal choice at every step, hoping the local choices combine into a global optimum. Greedy does NOT always work - but for many classic problems (activity selection, Huffman codes, coin change with certain denominations, MST, Dijkstra, fractional knapsack) it does. The hard part is proving *why*.

24.2 Theory: Exchange Argument

Proving greedy - the exchange argument

1. Assume your greedy solution is different from an optimal solution.
2. Show you can "swap" the first differing element so the optimal solution becomes closer to the greedy one without losing optimality.
3. Repeat until the optimal solution equals the greedy one - so the greedy solution is optimal.

24.3 Classic Greedy Problems

24.3.1 Activity Selection

Maximum number of non-overlapping intervals - always pick the one that ends earliest.

Why ends-earliest wins

Picking the interval that finishes earliest leaves the most room for the rest - any optimal solution can swap its first interval for the earliest-ending one.

24.3.2 Fractional Knapsack

Take items by value/weight ratio; you may take fractions, so greedy is optimal.

24.3.3 Huffman Coding

Repeatedly merge the two least-frequent symbols (min-heap) - optimal prefix-free code (Chapter 16 heaps example, Section 16.6).

24.3.4 Coin Change (canonical systems)

For canonical denominations (e.g., 1,5,10,25), greedy works. For arbitrary sets (e.g., 1,3,4 with amount 6) it fails: greedy gives $4+1+1=3$ coins, optimum is $3+3=2$. **When denominations are arbitrary, use DP (Chapter 26).**

24.4 Dry Run: Activity Selection

24.5 When Greedy Fails - and what to do

- Coin change with non-canonical denominations -> DP.
- 0/1 knapsack -> DP (greedy by ratio fails).
- Travelling salesman -> exponential search / approximations.
- General interval partitioning can still be greedy (min-heap of rooms), but "maximum value subset" variants are DP.

Test before trusting greedy

Before coding a greedy solution, try to construct a counterexample. If you find one, switch to DP or search. In interviews, the exchange argument is your proof.

24.6 Real-World Analogy

Greedy is like a **hiker who always takes the steepest immediate step** toward the summit. On a convex hill that reaches the global top; on a ridge with a valley in between, it stalls. Activity selection is choosing which conference talks to attend to maximize the count - you skip anything that overlaps your current earliest-ending pick.

24.7 Applications

- Scheduling (CPU, intervals, deadlines), data compression (Huffman, LZ), routing (Dijkstra), MST (Kruskal/Prim), task assignment, job sequencing with deadlines.

24.8 Common Mistakes

- Applying greedy where a counterexample exists (always test!).
- Forgetting to sort before scanning.
- Not proving optimality (exchange argument) in interviews.
- Using greedy for 0/1 knapsack (ratio-based) - wrong.
- Overflow when computing ratios; compare by cross-multiplication instead.

24.9 Interview Tips

- "Maximum number of non-overlapping intervals" -> sort by end, take greedily.
- Say "this is optimal because of an exchange argument" and sketch it.
- Mention the counterexample for coin change to show depth.
- Practise: jump game, gas station, candy distribution, remove k digits.

24.10 Summary

- Greedy = locally optimal choices; optimal only with a proof.
 - Exchange argument or matroid theory justify most classics.
 - Sorting is usually the first step.
 - When unsure, find a counterexample; if none, prove - then code.
-

Practice Set: Greedy Algorithms

Beginner Problems

1. Activity selection (non-overlapping max count).
2. Minimum number of platforms for trains (interval overlap).
3. Fractional knapsack.
4. Sort by finish time vs start time - explain the difference.
5. Minimum number of coins (canonical denominations).

Intermediate Problems

1. Jump Game: can you reach the last index?
2. Gas station circuit (LC 134).
3. Huffman coding tree construction with a min-heap.
4. Job sequencing with deadlines (max profit).
5. Remove k digits to make the smallest number.

Advanced Problems

1. Candy: minimum candies given ratings and neighbours rule (LC 135).
2. Minimum number of intervals to remove to make the rest non-overlapping.
3. Greedy with a heap: find k largest elements / merge k sorted lists.
4. Split array into equal-sum subsets (greedy + backtracking).
5. "Minimum cost to cut a board" (reverse greedy / Huffman idea).

Interview-Style Problems

1. "Maximum meetings in one room given start/end times."
2. "Assign cookies so the most children are satisfied" (LC 455).

Competitive Programming Problems

1. "Task scheduler: minimum idle time" (LC 621, greedy + heap).
2. "Rearrange string so no two adjacent are the same."

Challenge Exercises

- Prove activity selection optimality with an exchange argument.
 - Find a coin set where greedy fails and DP fixes it.
 - Prove the fractional knapsack greedy is optimal (ratio ordering).
-

Solutions: Greedy Algorithms

Beginner Solutions

1. Section 24.3.1.

1. Section 24.3.2. 4. Finish time frees the room earliest. 5. Largest denominations first; canonical systems are safe.

Intermediate Solutions

1. Track maxReach; fail if you ever sit on an index beyond reach.

1. If total gas < total cost return -1; otherwise single-pass greedy with running deficit - start at the index after the lowest cumulative deficit.
2. Merge two smallest frequencies until one node (Section 16.6).
3. Sort by profit descending; use a DSU or boolean array to place each job at the latest free slot $\leq$ deadline.
4. Stack: pop larger digits while k remains and the next digit is smaller; then trim leading zeros.

Advanced Solutions

1. Two passes: left-to-right ensure rating rule, right-to-left again; sum maxes.
2. Greedy by end time (activity selection on the complement) - answer = total - selected.
3. Max-heap of the k best / min-heap of k largest; merge k sorted lists with a heap of heads.
4. Greedy largest-first bucket fill + backtracking (subset-sum style).
5. Cut the most expensive lines first (reverse of building): each full cut multiplies segments - process by descending cost (Huffman-like).

Interview-Style Solutions

1. Activity selection. 17. Sort both arrays, greedily assign the smallest cookie that satisfies each child.

Competitive Programming Solutions

1. Count the most frequent task; $\text{answer} = \max(n, (\text{maxFreq}-1)(n+1)+\text{countOfMax})$.
2. Greedy with a max-heap of (frequency, char); append the most frequent char that differs from the last.

Challenge Solutions

- Exchange: replace the optimal's first interval with the greedy's first (earliest end) - still feasible, no worse; induct.
- Coin set {1,3,4}: amount 6 -> greedy 4+1+1 (3 coins) vs 3+3 (2 coins).
- Fractional: any optimal solution not taking the best-ratio item can swap in a fraction of it and improve.

25. Divide and Conquer

25.1 Introduction

Divide and conquer (D&C) breaks a problem into smaller independent subproblems, solves them recursively, and **combines** the results. The prototypes are merge sort, quicksort, binary search, and fast exponentiation - plus classics like max subarray and counting inversions. It is powerful when the combination step is cheap relative to the reduction in size.

25.2 The D&C template

Template

1. **Divide**: split input into 2+ smaller pieces.
2. **Conquer**: solve each piece recursively (base case when tiny).
3. **Combine**: merge the sub-results into the answer for the whole input.

25.3 Merge Sort - $O(n \log n)$

Merge sort is stable and predictable

Always $O(n \log n)$ regardless of input. The merge step is where we can count inversions (see below).

25.4 Quicksort - $O(n \log n)$ average, $O(n^2)$ worst

Partition around a pivot; recurse on each side.

Worst case and randomization

Sorted input with a fixed pivot $\rightarrow O(n^2)$. Pick the pivot randomly (or use median of three) to make $O(n \log n)$ expected. Quicksort is NOT stable.

25.5 Counting Inversions (with merge sort)

An inversion is a pair $(i < j)$ with $a[i] > a[j]$. During the merge, when an element from the right half is placed before some remaining elements of the left half, each such element forms an inversion.

25.6 Maximum Subarray Sum - $O(n)$ via D&C or Kadane

Kadane's is the simple linear solution; the D&C version splits at the middle and combines best-left, best-right, best-crossing - $O(n \log n)$.

25.7 Other D&C classics

- **Fast exponentiation** (Section 8.7): $x^n = (x^{(n/2)})^2$ or $x \cdot x^{(n-1)}$.
- **Binary search** (Chapter 6): divide the search space in half each step.
- **Closest pair of points**: $O(n \log n)$, split by x , recurse, check a strip.
- **Strassen / Karatsuba**: fewer recursive multiplications (mention only).

25.8 Real-World Analogy

D&C is like **organising a library by the book spine's first letter**: split the catalogue in halves recursively until single books, then merge sorted stacks. Merge sort's merge step is a librarian merging two already-sorted piles by always taking the smaller visible book.

25.9 Common Mistakes

- Missing base cases (empty array, single element) -> infinite recursion.
- Quicksort worst case on sorted input (randomise!).
- Merge sort copying back with the wrong offset.
- Counting inversions with the wrong leftover count.
- Confusing D&C with DP: D&C subproblems are independent; DP subproblems overlap.

25.10 Applications

- Sorting (the workhorse), searching, median/selection (quickselect), big integer multiplication, matrix multiplication, Fourier transform (FFT), computational geometry, parallel algorithms.

25.11 Interview Tips

- "Count pairs with some ordering property" often = merge-sort inversion trick.
- "Closest pair" and "count range sums" scream D&C (or BIT).
- Quote the recurrence and its solution: $T(n) = 2T(n/2) + O(n) \rightarrow O(n \log n)$.
- Practise: merge intervals no - count inversions, reverse pairs, sort list.

25.12 Summary

- Divide / conquer / combine; subproblems are independent.

- Merge sort: stable $O(n \log n)$; quicksort: fast expected $O(n \log n)$.
 - The merge step is reusable (inversions, range queries).
 - Master theorem governs most D&C recurrences (Section 3.9).
-

Practice Set: Divide and Conquer

Beginner Problems

1. Implement merge sort by hand on a small array.
2. Implement quicksort (Lomuto partition).
3. Implement binary search recursively.
4. Implement fast exponentiation.
5. Find the k-th smallest element with quickselect.

Intermediate Problems

1. Count inversions in an array ($O(n \log n)$).
2. Maximum subarray sum (D&C and Kadane).
3. Sort a linked list using merge sort ($O(n \log n)$, $O(1)$ extra).
4. Count of smaller numbers after self (merge-sort / BIT).
5. Merge k sorted lists (D&C or heap).

Advanced Problems

1. Closest pair of points ($O(n \log n)$).
2. Count "reverse pairs" $i < j$ and $a[i] > 2a[j]$.
3. Skyline problem (merge two skylines).
4. Find the majority element (D&C or Boyer-Moore).
5. Karatsuba-style polynomial multiplication (mention only / small impl).

Interview-Style Problems

1. "Sort an array in $O(n \log n)$ and explain stability."
2. "Find the k-th largest element without full sort" (quickselect, $O(n)$ expected).

Competitive Programming Problems

1. "Count pairs with sum $< k$ " using divide-and-conquer counting (merge).
2. Median of two sorted arrays ($O(\log(n+m))$) - binary search over the cut.

Challenge Exercises

- Prove merge sort's $O(n \log n)$ bound with a recurrence tree.
 - Explain why quicksort is not stable and how to make it stable.
 - Implement iterative (bottom-up) merge sort.
-

Solutions: Divide and Conquer

Beginner Solutions

1-4. Section 25.3-25.7.

Intermediate Solutions

1. Section 25.5. 7. Section 25.6 (Kadane is the interview-favourite).

1. Merge-sort counting smaller-from-right; equivalently a Fenwick tree after coordinate compression.
2. Merge two at a time repeatedly, or a min-heap of list heads.

Advanced Solutions

1. Sort by x ; split; closest(left), closest(right); strip of width $2d$ around the boundary; sort strip by y ; check $O(n)$ candidates per point.
2. Same as inversions but compare $a[i] > 2a[j]$ in the merge.
3. Recursively merge two skylines by comparing heights at each x .
4. Divide in halves, majority candidate must appear in a half (or Boyer-Moore).

1. Karatsuba: three multiplications instead of four on halves.

Interview-Style Solutions

1. Merge sort is stable; quicksort is not. Explain both.
2. Quickselect $O(n)$ expected; pivot randomisation avoids $O(n^2)$.

Competitive Programming Solutions

1. Sort; two pointers counting pairs - or use merge counting for $i < j$ pairs.

2. Binary search over the split of the smaller array such that the partition is correct; answer from the boundary elements.

Challenge Solutions

- Recurrence tree: $n + 2(n/2) + 4(n/4) + \dots = n \log n$ levels of work n each.
- Stability: quicksort swaps across long distances, breaking equal-key order; stable versions use extra space or tie-breaking keys.
- Bottom-up merge sort: double the run length each pass, merge adjacent runs.

26. Dynamic Programming

26.1 Introduction

Dynamic programming (DP) is the art of solving problems with **overlapping subproblems** by solving each subproblem once and storing the answer. Where divide and conquer splits into *independent* pieces, DP reuses results across pieces. The two ingredients are:

- **Optimal substructure:** an optimal solution contains optimal solutions to its subproblems.
- **Overlapping subproblems:** the same subproblems recur many times.

DP is written either **top-down** (recursion + memoisation) or **bottom-up** (iterative table fill).

26.2 The 5-step DP recipe

Recipe

1. **Define the state:** what does `dp[i]` (or `dp[i][j]`) mean?
2. **Base cases:** smallest inputs you can answer directly.
3. **Recurrence:** how does a bigger state depend on smaller ones?
4. **Order of computation:** ensure dependencies are computed first.
5. **Answer:** which state(s) hold the final result?

26.3 Fibonacci - the canonical example

Without memoisation, fib is $O(2^n)$. With it, $O(n)$. That is the whole point.

26.4 1D DP examples

26.4.1 Climbing stairs

$dp[i]$ = number of ways to reach step i : $dp[i] = dp[i-1] + dp[i-2]$.

26.4.2 House robber

$dp[i]$ = max loot from houses $0..i$: $\max(dp[i-1], dp[i-2] + a[i])$.

26.4.3 Longest increasing subsequence (LIS)

$dp[i]$ = length of LIS ending at $i = 1 + \max(dp[j])$ for $j < i$ with $a[j] < a[i]$; $O(n^2)$, with a patience-sorting $O(n \log n)$ variant using binary search.

26.5 2D DP examples

26.5.1 Grid paths (with obstacles)

$dp[i][j]$ = number of ways to reach cell $(i, j) = dp[i-1][j] + dp[i][j-1]$ (0 if obstacle).

26.5.2 Edit distance

$dp[i][j]$ = min edits to convert $s[0..i)$ to $t[0..j)$.

26.5.3 0/1 knapsack

$dp[i][w]$ = max value using the first i items with capacity w .

Why descending in knapsack

Iterating w downward means $dp[w - wt[i]]$ still holds the *previous* item's row, so each item is used at most once. Ascending would allow unlimited reuse (unbounded knapsack).

26.6 Memoisation: the same language

26.7 State-space explosions and when to stop

- 1D states: usually $O(n)$ time, $O(1)$ - $O(n)$ memory.
- 2D states (i, j) : $O(n^2)$ time; watch out for $n \geq 10^4$.

- Bitmask DP: 2^n states - only for $n \leq 20$.
- Interval DP, tree DP, digit DP: specialist patterns (covered below).

26.8 Common DP patterns worth knowing

Pattern library

- **Subset/knapsack**: boolean reachability, dp over capacity.
- **Longest common subsequence (LCS)**: $dp[i][j]$ matching.
- **Interval DP**: $dp[l][r]$ (e.g., burst balloons, matrix chain).
- **Tree DP**: $dp[node]$ over subtrees (e.g., diameter, max independent set).
- **Digit DP**: count numbers $\leq N$ with a property, digit by digit.
- **Bitmask DP**: TSP for $n \leq 20$.

26.9 Real-World Analogy

DP is a **chef who keeps a pantry of prepared sauces**. When a recipe calls for the same stock reduction he made last week, he reuses it instead of simmering for hours again (memoisation). Naive recursion is the chef who throws away every batch and re-cooks from scratch - exponential effort. Fibonacci without memo is cooking the same stock thousands of times.

26.10 Common Mistakes

- Forgetting the base case or setting wrong default values (-1 vs 0).
- Wrong loop order (must fill dependencies first).
- Off-by-one in dp indices ($dp[i]$ meaning "first i elements" vs "ending at i ").
- Not distinguishing "ways" vs "min/max" (sum vs min) - different recurrences.
- Integer overflow with large answers (use long long).
- Knapsack with ascending inner loop accidentally making it unbounded.

26.11 Applications

- Genomics (sequence alignment = edit distance), route planning, resource allocation, speech/parsing, stock trading, text just, game strategies, compression (LZW/LZ77), recommendation systems.

26.12 Interview Tips

- Name the state and recurrence out loud before coding.
- Start with brute-force recursion, then add memoisation - easier to get right.

- Ask: "what's the input size?" to size the table.
- Common starter problems: house robber, coin change, LIS/LCS, edit distance, unique paths, knapsack, word break, palindrome partitioning.
- Say the complexity: state count x transition cost.

26.13 Summary

- DP = optimal substructure + overlapping subproblems.
 - Top-down (memo) or bottom-up (table) - same complexity.
 - 1D, 2D, interval, tree, digit, bitmask - choose by state shape.
 - The 5-step recipe (state/base/recurrence/order/answer) works for every DP.
-

Practice Set: Dynamic Programming

Beginner Problems

1. Fibonacci with memo and with a table.
2. Climbing stairs (number of ways).
3. House robber (max non-adjacent sum).
4. Unique paths in a grid.
5. Min cost climbing stairs.

Intermediate Problems

1. Longest increasing subsequence ($O(n^2)$ and $O(n \log n)$).
2. Longest common subsequence.
3. Coin change (minimum coins).
4. 0/1 knapsack (max value, W up to ~ 2000).
5. Edit distance.

Advanced Problems

1. Word break (string segmentable into dictionary words).
2. Longest palindromic substring / subsequence.
3. Burst balloons (interval DP).
4. Max sum of non-adjacent elements in a tree (house robber III).
5. Partition equal subset sum (boolean knapsack).

Interview-Style Problems

1. "Minimum path sum in a grid with obstacles."
2. "Number of ways to decode a digit string (LC 91)."

Competitive Programming Problems

1. Digit DP: count numbers in $[L, R]$ without consecutive 1s in binary.
2. TSP on a small graph ($n \leq 15$) with bitmask DP.

Challenge Exercises

- Prove LIS patience-sorting correctness.
 - Reconstruct the actual subsequence/path (store parents), not just the value.
 - Convert every top-down solution here to bottom-up and vice versa.
-

Solutions: Dynamic Programming

Beginner Solutions

1-5. Direct implementations (Sections 26.3-26.5).

Intermediate Solutions

1. Section 26.4.3 (both variants). 7. Standard matching DP.

1. Section 26.5.3. 10. Section 26.5.2.

Advanced Solutions

1. $dp[i] = \text{can prefix } [0,i) \text{ be segmented; try every dictionary word as a suffix.}$

1. LPS: $dp[i][j] = s[i]==s[j] \ \&\& \ dp[i+1][j-1]$; palindromic subsequence: standard LCS with the reversed string.

2. See above. 14. Tree DP: $dp[node][taken] = \text{max over children.}$

3. $dp[w] = \text{reachable with first } i \text{ items (bitset trick: } bits \ |= \ bits \ll v).$

Interview-Style Solutions

1. Grid DP with obstacle check (Section 26.5.1).

2. $dp[i] = \text{number of decodings of prefix; careful with '0'}$.

Competitive Programming Solutions

1. Digit DP with states (position, lastBit, tight). Count = $f(R) - f(L-1)$.
2. $dp[mask][last]$: min cost visiting set mask ending at last; transition over the next city. $O(2^n \cdot n^2)$.

Challenge Solutions

- LIS patience: tails is always sorted; binary search keeps it minimal; length of tails = LIS length.
- Parent reconstruction: keep a $prev[i]$ array in LIS/LCS/path DPs.
- Top-down to bottom-up: mirror the recurrence with a table and the right loop order; memo to table is mechanical once the state is clear.

27. Bit Manipulation

27.1 Introduction

Bit manipulation works directly on the binary representation of integers. It is astonishingly fast (single instructions), compact, and a favourite of interviewers. A solid grasp of bitwise operators plus a toolbox of tricks solves many otherwise non-obvious problems.

27.2 The operators

Operator	Meaning	Example		
`&`	AND	$6 \& 3 = 2$ (110 & 011)		
` `	OR		$6 3 = 7$	
`^`	XOR	$6 ^ 3 = 5$ (different bits)		
`~`	NOT	$\sim 0 = -1$		
`<<`	left shift	$1 << 3 = 8$ (multiply by 2^n)		
`>>`	right shift	$8 >> 2 = 2$ (divide by 2^n)		

XOR superpowers

- $a \wedge a = 0$ and $a \wedge 0 = a$.
- XOR is commutative and associative -> pairs cancel, singles survive.
- $a \wedge b = c$ implies $b \wedge c = a$ (self-inverse, usable for swap and ciphers).

27.3 The essential trick sheet

27.4 Classic problems

27.4.1 Find the single non-duplicated number

Every number appears twice, one appears once: XOR everything - pairs cancel.

27.4.2 Find the missing number in 0..n

XOR all indices and all values; the survivor is the missing one.

27.4.3 Two non-duplicated numbers

XOR everything $\rightarrow a \oplus b$. Take the lowest differing bit; split numbers by it; XOR each group $\rightarrow a$ and b .

27.4.4 Power of two / power of four

Power of two: $x > 0 \ \&\& \ (x \ \& \ (x-1)) == 0$. Power of four additionally needs the single bit on an even position: $x \ \& \ 0x55555555$.

27.4.5 Count set bits (Brian Kernighan)

27.4.6 Subsets via bitmask

Enumerate all 2^n subsets of an array.

27.5 The `__builtin` toolbox (gcc/clang)

27.6 Real-World Analogy

Bits are **light switches in a long corridor**. A bitmask is the whole row of switches; $x \mid (1 \ll i)$ turns on the i -th switch, $x \& \sim(1 \ll i)$ turns it off, $x \wedge (1 \ll i)$ toggles it, and $(x \gg i) \& 1$ checks it. XOR's self-inverse property is like a **two-way walkie-talkie**: pressing the button twice returns you to the same state.

27.7 Applications

- Flags and permissions (read/write/execute), set operations on subsets, hash functions, checksums (CRC), error detection (parity), colour compression, high-performance state machines, DP on subsets (TSP), cache addressing.

27.8 Common Mistakes

- Signed vs unsigned shifts (right shift of a negative int is implementation defined in older C++; use unsigned for portable shifts).
- $1 \ll 31$ overflows int - use $1LL \ll 31$ for long long.
- Operator precedence: $(x \gg i) \& 1$ needs the parentheses.
- Forgetting $x > 0$ in the power-of-two check (0 passes $(x \& x-1) == 0$).
- Mixing int and long long in shifts -> silent overflow.

27.9 Interview Tips

- "Single/finding the unique element" -> XOR.
- "Count bits / power of two / even parity" -> builtins or the trick sheet.
- "All subsets / combinations" -> bitmask enumeration (only for $n \leq 20$).
- For DP-on-subsets, say " 2^n states with bitmask transitions".
- Practise: single number, number of 1 bits, reverse bits, missing number, sum of two integers without +, bitwise AND of range.

27.10 Summary

- Six operators: `&` `|` `^` `~` `<<` `>>`; know their truth tables.
 - Trick sheet: set/clear/flip/test, lowest set bit, clear lowest, popcount.
 - XOR cancels pairs - the workhorse of "unique element" problems.
 - Bitmasks enumerate subsets and drive 2^n DP.
 - Use `__builtin_*` for fast, correct primitives.
-

Practice Set: Bit Manipulation

Beginner Problems

1. Check if a number is a power of two.
2. Count set bits (Kernighan and builtin).
3. Find the single non-duplicated number.
4. Find the missing number in 0..n.
5. Flip all bits of a 32-bit integer (and its value in the full range).

Intermediate Problems

1. Reverse the bits of a 32-bit integer.
2. Two non-duplicated numbers in an array.
3. Bitwise AND of a range of numbers.
4. Sum two integers without + or -.
5. Hamming distance (XOR then popcount).

Advanced Problems

1. Number of bit flips to convert a to b (LC 461 variant).
2. Maximum XOR of two numbers in an array (trie or divide).
3. Count numbers from 1 to n with exactly k set bits (DP + popcount).
4. Gray code generation (binary reflected Gray code).
5. Power of four without loops.

Interview-Style Problems

1. "Find the single number appearing once when all others appear three times."
2. "Design a lightweight 'distinct count' or subset enumeration using bitmask."

Competitive Programming Problems

1. Subset sum reachability with a bitset (`bits |= bits << v`) - faster than DP.
2. "Minimum XOR pair" in an array.

Challenge Exercises

- Prove $a \oplus a = 0$ and the two-single-numbers algorithm.
 - Explain why $x \& (x - 1)$ clears the lowest set bit.
 - Implement multiplication and division by powers of two with shifts.
-

Solutions: Bit Manipulation

Beginner Solutions

1-4. Section 27.3-27.4.

Intermediate Solutions

1. Above; faster: swap byte pairs or use SWAR tricks.
2. Section 27.4.3.

1. Iterative: $a \oplus b$ are the sum bits, $(a \& b) \ll 1$ are the carries; repeat until no carry.

1. `__builtin_popcount(a ^ b)`.

Advanced Solutions

1. `__builtin_popcount(a ^ b)`.
2. Build a binary trie of the numbers; for each number, greedily choose the opposite bit to maximise XOR.

1. $g[n] = g[n-1]$ prefixed 0 ++ reverse($g[n-1]$) prefixed 1, or $i \oplus (i \gg 1)$.
2. Power of four: power of two AND $(x \& 0x55555555) \neq 0$.

Interview-Style Solutions

1. Count each bit position across all numbers mod 3; assemble the result.

1. Subset enumeration with a mask (Section 27.4.6).

Competitive Programming Solutions

1. `bitset<N> bits; bits[0]=1; for v: bits |= bits << v;` - answer is popcount reachable positions.
2. Sort; the minimum XOR pair is always adjacent in sorted order (or use a trie).

Challenge Solutions

- XOR properties follow from the truth table ($a \oplus a = 0$, $a \oplus 0 = a$, associativity).
- $x \& (x-1)$: the lowest 1 bit becomes 0 because $x-1$ borrows through trailing zeros.
- Shifts multiply/divide by 2^k exactly for unsigned arithmetic.

MODULE

Advanced Data Structures

Segment Trees, Fenwick Trees, Sparse Tables

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

28. Segment Tree

28.1 Introduction

A **segment tree** is a full binary tree over an array that answers **range queries** (sum, min, max, gcd, ...) and supports **point updates**, both in **$O(\log n)$** . It trades $O(n)$ space for flexibility: unlike a prefix sum, it keeps working when the array changes.

28.2 Theory: The structure

- Leaves store array elements; internal nodes store the answer for their segment.
- Root covers $[0, n)$; a node covering $[l, r]$ splits into children covering $[l, \text{mid}]$ and $[\text{mid}+1, r]$, $\text{mid} = (l+r)/2$.
- Any range $[ql, qr]$ decomposes into $O(\log n)$ tree nodes (disjoint cover).
- A point update touches $O(\log n)$ nodes along the root-to-leaf path.

28.3 Implementation (iterative - simpler and fast)

Half-open intervals

The iterative version uses half-open $[l, r)$. Many competitive templates use `query(l, r)` meaning $[l, r)$. Decide one convention and be consistent.

28.4 Recursive segment tree with lazy propagation

For **range updates** (add v to every element of $[l, r]$), use lazy propagation: defer updates to whole nodes and push them down only when needed.

`+= v; }`

Lazy propagation complexity

Each range update/query touches $O(\log n)$ nodes plus the pushed paths - still $O(\log n)$. Without pushing, answers become stale. The key invariant: a node's value is correct for its whole segment, and lazy waits to be pushed to children.

28.5 Variants

- **Min/max/gcd/OR** segment trees: just change the merge operation.
- **Segment tree beats**: range-chmin, range-chmax (advanced).
- **Merge sort tree**: each node stores its segment's sorted values - answers "how many values in $[l, r]$ are $< x$ " in $O(\log^2 n)$.
- **Persistent segment tree**: keep every version after each update ($O(\log n)$ extra per update) - used for "k-th smallest in a subarray".

28.6 Dry Run: iterative query [1, 4) on the tree above

28.7 Real-World Analogy

A segment tree is a **tournament-style running tally**. At a sports venue, the scoreboard shows the total, each section shows a subtotal, each seat shows a number. To know the total for seats 40-80, you combine the few boards covering exactly that range. If a player's score changes, you update only the boards on the path to the top.

28.8 Applications

- Dynamic range queries with updates (sum/min/max/gcd), interval scheduling metrics, K-th smallest in a range (persistent), calendar overlap queries, finance (moving averages with updates), geometry (sweep line with event handling).

28.9 Common Mistakes

- Wrong size: iterative needs $2*n$, recursive needs $4*n$ (safe upper bound).
- Half-open vs closed interval confusion.
- Forgetting to push before descending in lazy trees.
- Merge function not associative-safe (order matters for min/max? it is commutative; but for max-prefix/suffix it is not - be careful).

- Integer overflow in `t[node] += v * (r-l+1)`.
- Building on the wrong array / not rebuilding after resize.

28.10 Interview Tips

- "Range query with updates" -> segment tree or Fenwick (Fenwick for sum only).
- State " $O(\log n)$ per query/update, $O(n)$ build".
- For "count of elements $< x$ in a range" mention the merge sort tree or BIT.
- Practise: range sum query + mutable, count of smaller numbers after self, skyline (with sweepline), online sum.

28.11 Summary

- Segment tree = full binary tree over segments; range query + point update $O(\log n)$.
 - Iterative version is short; recursive version handles lazy (range updates).
 - Merge function defines the semantics (sum/min/max/gcd/...).
 - Extensions: merge-sort tree, persistent tree, segment tree beats.
-

29. Fenwick Tree (Binary Indexed Tree)

29.1 Introduction

The **Fenwick tree** (BIT) answers **prefix sums** and supports **point updates** in **$O(\log n)$** , with $O(n)$ space and a tiny, elegant implementation. It is the go-to for online prefix-sum problems where the array changes.

29.2 Theory: The key idea

$BIT[i]$ stores the sum of a range ending at i whose length is the lowest set bit of i . Prefix sums follow the $i \rightarrow i \& -i$ chain; updates follow $i \rightarrow i \& -i$.

29.3 Implementation

Inversion counting with a BIT

Compress values to ranks, scan right to left; for each $a[i]$, add count of ranks $< a[i]$ already seen ($query(rank-1)$), then $add(rank, 1)$. Same $O(n \log n)$ as merge sort counting, easier to extend (k inversions, etc.).

29.4 Dry Run: sum(7)

29.5 Fenwick vs Segment Tree

Feature	Fenwick	Segment tree
Range sum + point update	$O(\log n)$	$O(\log n)$
Range update (lazy)	via difference trick	native

Feature	Fenwick	Segment tree
Min/max/gcd queries	no	yes
Code size	~10 lines	~40 lines
Persistent / beats	no	yes

29.6 Range update + range query (difference trick)

Two BITs maintain $d[i] = a[i] - a[i-1]$. A range add on $[l, r]$ by v : $\text{add}(l, v)$, $\text{add}(r+1, -v)$. Prefix sum of $a = \text{sum over } d \text{ position - correction (needs a second BIT holding id[i])}$.

29.7 Real-World Analogy

The BIT is like a **bank ledger with checkpoints**. Your statement page 4 keeps a running total from page 1; page 6 totals pages 5-6; page 8 totals everything. Balance up to page 7 = page 4's total + page 6's + page 7's own entries - you add a handful of pre-computed totals instead of scanning all pages.

29.8 Common Mistakes

- Forgetting 1-indexing (bit[0] unused; query(l-1) not query(l)).
- Using `idx & -idx` with signed overflow on negative of `INT_MIN`.
- Querying beyond n .
- Using a BIT where a lazy range update is needed and not using the difference trick.
- Confusing update direction (add goes up, sum goes down).

29.9 Applications

- Inversion counts, order statistics, running medians, prefix sums with updates, sweep-line event processing, "count of smaller numbers", k -th smallest via binary lifting on the BIT.

29.10 Interview Tips

- "Prefix sum with point updates" -> Fenwick, and say " $O(\log n)$ ".
- Know the BIT k -th smallest: binary lifting using bit indices (sum of lowbit jumps).
- Practise: count inversions, range sum with updates, online "insert + k -th largest".

29.11 Summary

- BIT = prefix sums + point updates in $O(\log n)$, $O(n)$ space.
- Chain: update up (`idx += lowbit`), query down (`idx -= lowbit`).
- Best for sums; segment tree when you need min/max/gcd or range updates.
- Extend with difference trick for range updates and binary lifting for order statistics.

30. Sparse Table

30.1 Introduction

The **sparse table** answers **idempotent range queries** (min, max, gcd, LCP) in **$O(1)$** after **$O(n \log n)$** preprocessing. Unlike the segment tree it is immutable - no updates - but it is the fastest possible answerer.

Idempotent operations

An operation is idempotent for range queries if overlapping ranges give the same answer: min/max/gcd are ($\min(a,b) = \min(a,\min(a,b))$). Sum is NOT idempotent - the sparse table cannot answer range sums.

30.2 Theory: The structure

$st[k][i]$ = the answer for the range $[i, i + 2^k)$. Build by doubling: $st[k][i] = \text{merge}(st[k-1][i], st[k-1][i + 2^{k-1}])$.

30.3 Dry Run: RMQ on [1, 3] of [9, 2, 5, 4]

30.4 Comparisons

Structure	Build	Query	Update	Use for
Prefix sum	$O(n)$	$O(1)$	none	static sums

Structure	Build	Query	Update	Use for
Sparse table	$O(n \log n)$	$O(1)$	none	static idempotent
Fenwick	$O(n \log n)$	$O(\log n)$	$O(\log n)$	dynamic sums
Segment tree	$O(n)$	$O(\log n)$	$O(\log n)$	everything dynamic

30.5 Real-World Analogy

A sparse table is a **well-thumbed cookbook with pre-baked size labels**. For any recipe window (a range of ingredient amounts), you look up the largest pre-baked table of that size (power of two) that fits, take two overlapping copies, and combine - no cooking needed at query time.

30.6 Common Mistakes

- Using a sparse table for range sums (overlap breaks it).
- Forgetting the $+ 1$ in $r - (1 \ll k) + 1$.
- Not precomputing $\lg$ (or computing it per query $\rightarrow O(\log n)$ anyway).
- Off-by-one with half-open vs closed ranges.
- Building $st[k]$ where $i + (1 \ll k) > n$ writes out of bounds.

30.7 Applications

- Static RMQ (min/max/gcd), LCA via Euler tour + RMQ (with depth), LCP of suffixes, disjoint-set-like union in offline processing, any idempotent query.

30.8 Interview Tips

- "Static array, many min/max queries" $\rightarrow$ sparse table; quote $O(1)$ query.
- " $O(1)$ LCA" = Euler tour + sparse table over depths.
- Know when NOT to use it: any updates $\rightarrow$ BIT/segment tree.
- Practise: range min query, LCA, GCD range queries.

30.9 Summary

- Sparse table: $O(n \log n)$ build, $O(1)$ idempotent queries, immutable.
- Overlapping blocks are fine for min/max/gcd - the whole trick.
- Choose the right static/dynamic tool from the comparison table.

Practice Set: Segment Tree, Fenwick, Sparse Table

Beginner Problems

1. Build a segment tree and answer point-update + range-sum queries.
2. Implement the iterative Fenwick tree (add + prefix sum).
3. Build a sparse table for RMQ and answer min queries.
4. Point update + range minimum with a segment tree.
5. Range sum with the difference-trick Fenwick (range update, point query).

Intermediate Problems

1. Count inversions with a Fenwick tree.
2. Range sum query with point updates (LC 307, segment tree or BIT).
3. GCD range query (sparse table or segment tree).
4. k-th smallest in a dynamically changing multiset (BIT with binary lifting).
5. Lazy segment tree: range add + range sum.

Advanced Problems

1. Merge sort tree: count of numbers $< x$ in $[l, r]$.
2. Persistent segment tree: k-th smallest in a subarray (offline with updates).
3. Range max query with index retrieval (sparse table returns the position).
4. Segment tree beats style: range chmin + range sum (implement the simple version).
5. LCA with Euler tour + sparse table.

Interview-Style Problems

1. "Design a data structure supporting append + get sum of last k elements."
2. "Range sum query and range update in $O(\log n)$ each" (lazy segment tree).

Competitive Programming Problems

1. Count of numbers between L and R in a range (merge-sort tree).
2. Dynamic median with insert/delete using two heaps or a BIT.

Challenge Exercises

- Prove the BIT update/query chains are correct (why $\text{idx} \pm \text{lowbit}$).
 - Prove the sparse table query covers $[l, r]$ with two blocks.
 - Convert a lazy recursive segment tree into an iterative lazy one.
-

Solutions: Segment Tree, Fenwick, Sparse Table

Beginner Solutions

1-5. Direct implementations (Sections 28-30).

Intermediate Solutions

1. Compress ranks; scan right-to-left; query(rank-1) before add(rank,1).
2. Fenwick (sum) or segment tree - both $O(\log n)$.
3. Sparse table with gcd merge ($O(1)$ query) or segment tree.

1. Lazy segment tree (Section 28.4).

Advanced Solutions

1. Each node stores a sorted vector of its segment; query merges $O(\log n)$ vectors with binary searches - $O(\log^2 n)$ per query.
2. Persistent segment tree: roots[r] - roots[l-1] over values -> k-th order statistic.
3. Sparse table storing (value, index) pairs, comparing value first.
4. Segment tree beats: for chmin, keep max and second-max; if chmin $\geq$ max do nothing; if chmin $>$ secondMax, update only maxes and tag. (Sketch.)
5. Euler tour of the tree; RMQ over depth with the sparse table; the node with min depth is the LCA.

Interview-Style Solutions

1. A deque + sliding prefix structure, or a Fenwick over a large array.
2. Lazy segment tree (Section 28.4).

Competitive Programming Solutions

1. Merge sort tree with `lower_bound/upper_bound` on each node vector.
2. Two heaps (max for low half, min for high half) rebalanced on insert; BIT works with explicit delete.

Challenge Solutions

- BIT chain proof: `prefix(i)` needs the nodes whose covered ranges end at exactly i minus lowbit steps; the chain enumerates exactly the disjoint ranges.
- Sparse query: `st[k][l]` covers $[l, l+2^k)$, `st[k][r-2^k+1]` covers $[r-2^k+1, r+1]$; since $2^k \geq$ half the range, they overlap and cover all.
- Iterative lazy: more complex; keep lazy arrays indexed like the iterative tree and push only along the queried paths (advanced topic).

MODULE

String Algorithms

Pattern matching, hashing, Z-algorithm

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

31. String Searching: KMP, Rabin-Karp, Z-algorithm

31.1 Introduction

Finding a **pattern** inside a **text** is the most fundamental string problem. The naive approach is $O(nm)$. *This chapter covers the three classic optimizations:*

- **KMP:** $O(n+m)$ worst case using a failure (prefix) function.
- **Rabin-Karp:** $O(n+m)$ average using rolling hashes - and it generalises to searching for multiple patterns.
- **Z-algorithm:** $O(n+m)$, computes for every position the longest substring that matches the string's prefix - a super-powered KMP.

31.2 Theory: the failure function (KMP)

The prefix function $pi[i] = \text{length of the longest proper prefix of } pattern[0..i] \text{ that is also a suffix}$. When a match fails at position j , we can safely restart at $pi[j-1]$ instead of the beginning - no character is re-compared.

KMP never backtracks in the text

The text index only moves forward; j rewinds via pi to the longest already- matched prefix. Total work is $O(n + m)$ amortised.

31.3 Dry Run: prefix function of "ababc"

31.4 Rabin-Karp - rolling hash

Hash the pattern and a sliding window of the text; compare hashes. On collision, verify with a direct character compare.

Hash collisions

A single modulus can collide. Use two different (base, modulus) pairs, or a 64-bit unsigned hash, and always verify on a hit. Rabin-Karp's average is $O(n+m)$; worst case (pathological collisions) is $O(nm)$.

31.5 Z-algorithm

$z[i]$ = length of the longest substring starting at i that matches a prefix of the whole string.

Search pattern in text: build `pat + '$' + text`; every position where `z[i] == pat.size()` is a match. The `$` separator prevents matches spanning the boundary.

31.6 Dry Run: Z of "aaaaa"

31.7 What to use when

Task	Tool
find all occurrences of one pattern	KMP or Z
multiple patterns / 2D matching	Rabin-Karp
string equality of substrings (random access)	rolling hash
count palindromic substrings (even/odd radii)	Manacher (Ch 32)
suffix queries, distinct substrings count	suffix array / trie

31.8 Real-World Analogy

- **KMP** is a **librarian scanning a catalogue** who, when the book he expects at page 5 is missing, does not restart from page 1 - he remembers how much of the pattern was already matched and jumps to the right continuation point.
- **Rabin-Karp** is a **fast-food order checker** who compares the hash (order total) first and only opens the bags whose totals match.
- **Z** is a **cipher alignment tool**: for every offset, how many starting symbols agree with the first symbols - a quick way to spot periodic structure.

31.9 Common Mistakes

- KMP: off-by-one on the failure function; not handling `j == 0` correctly.
- Z: forgetting the `[l, r]` window optimization (without it, it is $O(n^2)$).
- Rabin-Karp: overflow in `h[l] * pw[...]` - take mod before/after.
- Using char arithmetic with signed types (cast to unsigned long long).
- Forgetting the separator when using Z for search (matches can straddle).

31.10 Applications

- Text editors (find), IDEs (search), plagiarism checkers, DNA/sequence search, spam filters (multi-pattern), file diff, packet inspection, autocomplete.

31.11 Interview Tips

- "Find all occurrences" -> mention KMP/Z, $O(n+m)$.
- "Multiple patterns / frequent re-hashing" -> Rabin-Karp rolling hash.
- "Count of distinct substrings" or "longest repeated substring" -> suffix array/trie (Chapter 17).
- Practise: strStr, repeated substring pattern, longest happy prefix (Z/pi), shortest palindrome (KMP on reversed).

31.12 Summary

- KMP: prefix function + forward scan, $O(n+m)$.
 - Rabin-Karp: rolling hash, $O(n+m)$ average, collision-safe with double hashing.
 - Z: one pass computing prefix matches, $O(n+m)$, easy pattern search.
 - Pick by need: single pattern -> KMP/Z; multi/2D -> Rabin-Karp.
-

32. Advanced String Techniques

32.1 Manacher's Algorithm - palindromes in $O(n)$

Compute for every center the palindrome radius in linear time using a mirror-index window.

Even-length palindromes

Insert a dummy character between all characters (e.g., #) so every palindrome becomes odd-length, run the odd algorithm, and translate indices back.

32.2 Suffix array and LCP (brief)

Sort all suffixes lexicographically in $O(n \log n)$ (prefix-doubling); LCP array gives the longest common prefix of adjacent suffixes. Applications: longest repeated substring, number of distinct substrings = $n(n+1)/2 - \text{sum}(\text{LCP})$.

32.3 String hashing for substring equality

The Rabin-Karp hash from Section 31.4 already gives $O(1)$ substring equality: $\text{hash}(l, r)$ from prefix hashes. Use it for binary-searching palindromes, checking "is substring A equal to substring B", and LCP queries.

32.4 Trie applications recap (Chapter 17)

- Autocomplete, word games, maximum XOR pair (insert all numbers, greedily take opposite bits), longest common prefix of a set.

32.5 Aho-Corasick (multi-pattern, brief)

Build a trie of all patterns plus failure links (like KMP's π on the trie); scan the text once, reporting every pattern that ends at each position - $O(n + \text{total pattern length})$ for all occurrences.

32.6 Real-World Analogy

Manacher is a **postman who, having delivered the longest parcel palindrome at a block center, reuses the known radii of mirrored centers** to skip re-inspecting symmetry already established - hence $O(n)$. Suffix arrays are a **phonebook of all suffixes sorted alphabetically** - the dictionary of a whole text.

32.7 Common Mistakes

- Manacher: index mirroring mistakes; forgetting the while-loop can still extend.
- Suffix array: $O(n \log^2 n)$ implementation forgetting stable sorts of ranks.
- Hashing: using base/mod pair that is weak; signed char indexing.
- Aho-Corasick: forgetting to propagate failure links into children.

32.8 Applications

- Palindrome mining (solutions, longest palindromic substring/subsequence), plagiarism and duplicate detection, text compression, bioinformatics (suffix arrays for pattern search in DNA), full-text search engines.

32.9 Interview Tips

- "Longest palindromic substring" -> expand-around-center $O(n^2)$ is usually enough; mention Manacher $O(n)$ for extra credit.
- "Distinct substrings" -> suffix array / trie; quote the formula.
- Practise: longest palindromic substring, distinct substrings, longest common prefix (trie), shortest unique prefix.

32.10 Summary

- Manacher: linear-time palindromes via mirror radii.
 - Suffix array + LCP: rich text analysis; distinct substring counts.
 - Rolling hash: $O(1)$ substring equality; double-hash to be safe.
 - Aho-Corasick: KMP on a trie for many patterns at once.
-

Practice Set: String Algorithms

Beginner Problems

1. Implement naive pattern search; count the comparisons.
2. Build and print the KMP prefix function.
3. Implement Rabin-Karp and test on a small text.
4. Implement the Z-function.
5. Find the first occurrence of a pattern (KMP and Z).

Intermediate Problems

1. Find all occurrences of a pattern (return starting indices).
2. Longest palindromic substring (expand around center; then Manacher).
3. Repeated substring pattern: is the string built from one smaller unit?
4. Longest happy prefix: longest proper prefix that is also a suffix (KMP).
5. Count occurrences of every pattern from a dictionary in the text (Aho-Corasick or per-pattern KMP).

Advanced Problems

1. Shortest palindrome: prepend characters so the result is a palindrome (KMP on reversed string).
2. Distinct substrings count (suffix array or trie).
3. Longest repeated substring (suffix array + LCP).
4. Rolling hash: find if any substring of length k repeats.
5. Maximum XOR of two numbers using a binary trie.

Interview-Style Problems

1. "Find the index of a pattern in a text; if absent, -1 (strStr)."
2. "Check if a string can become a palindrome by removing at most one character."

Competitive Programming Problems

1. Palindromic substring count (Manacher; count distinct palindromes).
2. String rotations: check if two strings are rotations ($s+s$, find pattern).

Challenge Exercises

- Prove KMP's $O(n+m)$ bound via amortized analysis of j .
 - Explain why Z works without the $[l, r]$ window it is $O(n^2)$.
 - Build the suffix array of a string by hand (prefix doubling).
-

Solutions: String Algorithms

Beginner Solutions

1-5. Sections 31.2-31.5.

Intermediate Solutions

1. KMP/Z return all indices (Sections 31.2, 31.5).
2. Expand around center; record max. Manacher optional.

1. Answer = $\text{pi}[n-1]$ of the string itself.
2. Aho-Corasick trie with failure links; report matches at each node.

Advanced Solutions

1. $s + \# + \text{reverse}(s)$; KMP pi ; the overlapping prefix length gives how much to prepend.

1. Suffix array: $n(n+1)/2 - \sum(\text{LCP}[i])$. Or insert suffixes into a trie and count nodes.
2. Max over LCP of adjacent suffixes.
3. Hash all length- k windows; on equal hash verify content; duplicate $\rightarrow$ repeat.
4. Insert each number's bits into a binary trie; for each number walk opposite bits greedily.

Interview-Style Solutions

1. KMP or Z; return -1 if empty.
2. Two pointers from both ends; on mismatch try skipping left or right once.

Competitive Programming Solutions

1. Manacher gives all radii $\rightarrow$ count = sum over odd + even lengths; distinct palindromes need a suffix structure (advanced).
2. $(s+s).\text{find}(p) \neq \text{npos} \ \&\& \ (s+s).\text{find}(p) < n$.

Challenge Solutions

- KMP bound: j only increases by 1 per step and falls back by pi , so total decreases bounded by total increases $\rightarrow O(n+m)$.

- Without $[l, r]$, the while-loop restarts comparisons $\rightarrow O(n^2)$ worst case.
- Prefix doubling: sort by first 2^k chars using previous ranks as a key pair.

MODULE

Geometry and Advanced Techniques

Computational geometry, offline algorithms

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

33. Computational Geometry Basics

33.1 Introduction

Computational geometry solves geometric problems with points, lines, segments, and polygons. In competitive programming and interviews the fundamentals are: points and vectors, the **cross product** (orientation and area), convex hulls, and polygon area. All arithmetic is done with integers or careful floating point.

33.2 Points and vectors

33.3 The cross product - the workhorse

What orientation tells you

$\text{orient}(o, a, b) > 0$: $a \rightarrow b$ is a **left (counter-clockwise) turn** at o . < 0 : right turn.
 $= 0$: collinear. This powers convex hulls, segment intersection, and polygon containment.

33.4 Convex hull - Andrew's monotone chain, $O(n \log n)$

Monotone chain intuition

Sort by x; sweep left-to-right keeping only left turns (lower hull); sweep right-to-left for the upper hull; concatenate. Every point above the current hull pops out - you never revisit rejected points.

33.5 Polygon area (shoelace formula)

Sign of the shoelace sum indicates clockwise vs counter-clockwise ordering.

33.6 Segment intersection

Two segments AB and CD intersect if they straddle each other: $\text{orient}(A,B,C) * \text{orient}(A,B,D) < 0$ and vice versa - plus collinear boundary checks. This is the classic "lines intersect" interview problem.

33.7 Closest pair of points - $O(n \log n)$

Sort by x; divide; recurse; check the strip of width $2d$ around the split with a y-sorted sweep (Section 25.7). Careful implementation is a classic divide-and-conquer exercise.

33.8 Real-World Analogy

The cross product is like **deciding whether you are turning left or right** when driving through three points on a map. The convex hull is the **rubber band stretched around all the nails** on a board - it snaps to the outermost points, skipping any nail inside the band. The shoelace formula measures the area you walk when you go around a polygon, summing the trapezoids between each edge and the x-axis.

33.9 Common Mistakes

- Integer overflow: cross products of 10^9 coordinates exceed int (use long long).
- Comparing area to 0 without epsilon when using doubles.
- Convex hull: forgetting the ≤ 0 pop removes collinear boundary points too.
- Not handling duplicate points or $n < 3$.
- Mixing 2D cross product with 3D intuition.

33.10 Applications

- GPS and mapping (convex hulls of coverage), collision detection, image processing (outline tracing), VLSI layout, robotics (visibility), graphics rasterization, and a huge set of competitive problems (fencing, guarding, polygon containment).

33.11 Interview Tips

- "Longest perimeter fence / smallest enclosing region" -> convex hull.
- "Do these line segments cross?" -> orientation tests.
- Quote $O(n \log n)$ for hull and closest pair.
- Practise: convex hull, points inside a polygon, segment intersection.

33.12 Summary

- Cross product: orientation (left/right/collinear) and double area.
 - Monotone chain: $O(n \log n)$ convex hull, two sweeps.
 - Shoelace: polygon area in $O(n)$ with long long.
 - Prefer integer arithmetic; only use doubles with epsilon when forced.
-

34. Advanced Techniques and Optimization

34.1 Introduction

Beyond the core algorithms, a toolbox of "meta" techniques turns hard problems into manageable ones. This chapter surveys the most reusable ones:

- **Two pointers and sliding window** (already used in arrays; formalised here).
- **Coordinate compression**: shrink huge value ranges to ranks.
- **Binary search on the answer**: convert "minimize the maximum" into a feasibility check.
- **Mo's algorithm**: offline range queries with $O((n+q) \sqrt{n})$.
- **Meet in the middle**: halve an exponential search space.
- **Incremental / offline tricks**: process events in the right order.

34.2 Coordinate compression

Map a large value set to dense ranks $\{0..k-1\}$ to use small-index structures (Fenwick, segment trees) and shrink the value axis.

34.3 Binary search on the answer

If the feasibility of a value is monotone ("can we do it within budget X?"), binary search the optimal value.

The pattern

Many "minimize the maximum / maximize the minimum / k-th something" problems are really "binary search the answer + $O(n)$ feasibility check". Quote $O(n \log(\text{MAX}))$ when explaining.

34.4 Mo's algorithm - offline range queries

For an array with no updates and many range queries, Mo's processes queries sorted by block of the left endpoint, moving pointers incrementally.

Mo's needs $O(1)$ add/remove

The trick only wins if you can slide one element in/out in $O(1)$. Frequency-count queries (how many distinct values in $[l, r]$) are the classic fit.

34.5 Meet in the middle

Split the input into two halves, enumerate subsets of each ($2^{(n/2)}$ each), and combine with sorting + two pointers or hash maps. Transforms 2^n into $\sim 2^{(n/2)}$.

34.6 The sliding window formalised

For "smallest window with property P" or "number of windows with sum $\leq k$ ", maintain two pointers so the window is always minimal/valid; each element enters and leaves once $\rightarrow O(n)$.

34.7 Offline processing and event sweep

- Sort events by time/coordinate and process incrementally (sweep line).
- Answers to "after k operations" often come by processing operations backward.
- Disjoint-set "remove edges" problems $\rightarrow$ reverse into edge additions.

34.8 Real-World Analogy

- **Coordinate compression:** replacing a world of house numbers 1, 7, 2001 with ranks 0,1,2 so the postman's book stays small.
- **Binary search on the answer:** "Can we deliver all parcels within 3 hours?" is easy to check; binary search finds the smallest feasible budget.
- **Mo's algorithm:** a delivery route that groups nearby destinations (same block) so the courier rarely zigzags across the city.
- **Meet in the middle:** splitting a 20-person guest list into two groups of 10 to brute-force each - far fewer combinations.

34.9 Common Mistakes

- Forgetting the compression is 0-indexed or losing the original values.
- Binary search on a non-monotone feasibility function (infinite loop).
- Mo's: wrong sorting comparator (must alternate r direction per block).
- Meet in the middle: forgetting to merge counts (not just existence).
- Sliding window: not maintaining the invariant when shrinking.

34.10 Applications

- Any contest problem with $n \sim 1e5$ and $q \sim 1e5$ range queries (Mo's).
- Large-value Fenwick/segment trees (compression).
- Minimize-maximize scheduling, allocation, partitioning (binary search).
- Subset problems with $n \leq 40$ (meet in the middle), range queries with updates handled by segment trees.

34.11 Interview Tips

- Recognize "minimize the maximum" instantly: say binary search on answer.
- For large-coordinate problems, mention compression before using a BIT.
- For many static range queries, mention Mo's ($O((n+q) \sqrt{n})$) if the platform expects offline answers.
- Practise: split array largest sum, Koko eating bananas, range frequency queries, subset sums near target.

34.12 Summary

- Compression: ranks for large values $\rightarrow$ small-index structures.
 - Binary search on the answer: monotone feasibility $\rightarrow O(\log \text{MAX})$ checks.
 - Mo's: offline range queries in $O((n+q) \sqrt{n})$ with $O(1)$ add/remove.
 - Meet in the middle: $2^{(n/2)}$ after splitting.
 - Sliding window and sweep-line: each element processed once.
-

Practice Set: Geometry and Advanced Techniques

Beginner Problems

1. Compute the cross product and orientation of three points.
2. Compute the area of a polygon with the shoelace formula.
3. Check if three points are collinear.
4. Coordinate-compress an array.
5. Binary search the smallest x such that $f(x)$ is feasible.

Intermediate Problems

1. Do two segments intersect?
2. Convex hull of a point set (monotone chain).
3. Is a point inside a convex polygon?
4. Split array largest sum: minimize the maximum subarray sum.
5. Koko eating bananas style "minimum rate" problems.

Advanced Problems

1. Number of distinct elements in ranges $[l, r]$ (Mo's algorithm).
2. Subset sums: how many subsets have sum $\leq S$ (meet in the middle).
3. Largest rectangle in a histogram variant using the stack.
4. Closest pair of points ($O(n \log n)$).
5. Smallest subarray with sum $\geq k$ (sliding window).

Interview-Style Problems

1. "Merge intervals so the max overlap is minimized" - or "minimum number of platforms" (greedy, Chapter 24).
2. "Given a list of points, find the rectangle with the maximum perimeter that bounds them" (convex hull).

Competitive Programming Problems

1. Range frequency query (how many x in $[l, r]$) offline with Mo's.
2. "Number of pairs with sum in a range" via meet in the middle or two pointers.

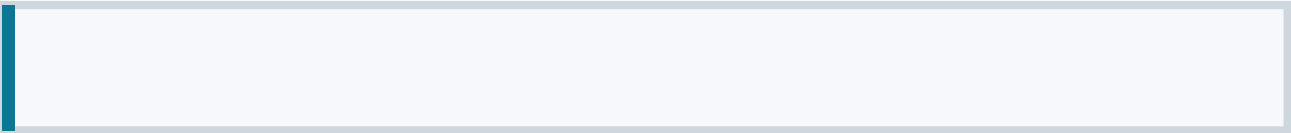
Challenge Exercises

- Prove the monotone chain produces a correct hull.
 - Analyse Mo's complexity exactly (pointer movement bound).
 - Implement meet in the middle for a subset-count problem by hand.
-

Solutions: Geometry and Advanced Techniques

Beginner Solutions

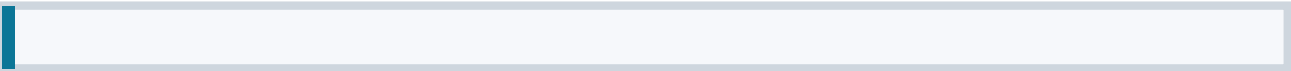
1-3. Section 33.3.



4-5. Sections 34.2-34.3.

Intermediate Solutions

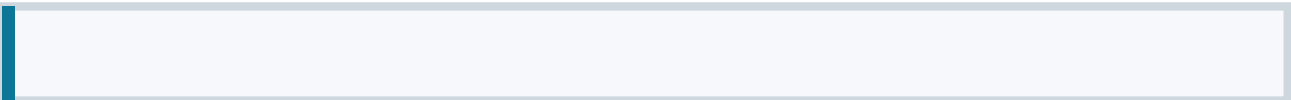
1. Straddle test: both pairs of orientations must have opposite signs (or be 0 with a boundary check).



1. For a convex polygon in order, check the point is on the same side of every edge (all orient signs equal).
2. Binary search on the max subarray sum; feasibility = can partition into k segments each $\leq \text{mid}$.
3. Same template: binary search the rate; check total time $\leq h$.

Advanced Solutions

1. Mo's with a frequency array; distinct count maintained with add/remove.



1. Stack of increasing heights (Chapter 16 style).
2. Divide and conquer closest pair (Section 25.7) with a strip sweep.
3. Two pointers; grow until $\text{sum} \geq k$, then shrink while still $\geq k$, track min length.

Interview-Style Solutions

1. Interval overlap counting (sort events) or greedy platforms.
2. Convex hull; the bounding rectangle is not the hull itself - "maximum perimeter rectangle" = hull-based rotating calipers (mention, don't code).

Competitive Programming Solutions

1. Mo's offline with frequency + the target x stored per query.
2. Sort one half; for each element of the other half, two binary searches count valid partners (or two pointers after sorting all).

Challenge Solutions

- Monotone chain proof: the sweep maintains a convex chain because every non-left turn pops; the upper sweep is symmetric; concatenation is the hull.
- Mo's bound: left pointer moves $O(q \cdot B)$ and right pointer $O(n^2 / B)$ across blocks; choosing $B = \sqrt{n}$ balances to $O((n+q) \sqrt{n})$.
- Meet in the middle: enumerate subsets of each half, count cross pairs by sorting one list.

MODULE

Revision 2

Consolidate: Recursion, Trees, Graphs, Greedy, DP, Strings, Advanced

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

Revision 2: Trees, Graphs, and Algorithm Design

R2.1 One-Page Revision Sheet

- **Trees:** n nodes $\rightarrow n-1$ edges; height/levels; full/complete/perfect; binary.
- **DFS on a tree:** in/pre/post-order; count subtree sizes via post-order.
- **BFS:** shortest path in unweighted graphs - $O(V+E)$; levels.
- **BST:** inorder gives sorted order; insert/search/delete $O(\text{height}) = O(\log n)$ balanced, $O(n)$ skewed. Successor = min of right subtree.
- **AVL:** balance factor $-1..1$; rotations LL, RR, LR, RL; height $O(\log n)$.
- **Heaps:** priority_queue; buildHeap $O(n)$; extract/insert $O(\log n)$; heapsort.
- **Tries:** insert/search/prefix $O(L)$; nodes for strings; count words.
- **Graphs:** adjacency list vs matrix vs edge list; dense vs sparse.
- **BFS/DFS:** $O(V+E)$. BFS = unweighted shortest paths. DFS = components, cycles, topo order (reverse post-order), SCC (Kosaraju/Tarjan), bridges/articulation (low-link), Euler (degree parity + Hierholzer).
- **Dijkstra:** $O((V+E) \log V)$, non-negative only; Bellman-Ford $O(V \cdot E)$ negatives; Floyd-Warshall $O(V^3)$ all-pairs; DAG topo-relax $O(V+E)$.
- **MST:** Kruskal (sort + DSU, $O(E \log E)$); Prim (heap, $O((V+E) \log V)$).
- **Union-Find:** find/unite with path compression + union by rank, $\sim O(\alpha(n))$.
- **Greedy:** locally optimal; prove with exchange argument; counterexample test.
- **D&C:** divide/conquer/combine; merge sort $O(n \log n)$; inversions in the merge.
- **DP:** overlapping subproblems + optimal substructure; state/base/recurrence/order/answer; top-down (memo) or bottom-up; knapsack descends the capacity loop.
- **Bit tricks:** $x \& (-x)$, $x \& (x-1)$, popcount, XOR cancels pairs, $1 \ll i$ masks.
- **Strings:** KMP (π , $O(n+m)$); Rabin-Karp (rolling hash); Z (prefix matches); Manacher (palindromes $O(n)$); suffix array + LCP.
- **Advanced structures:** segment tree ($O(\log n)$ query/update, lazy for range updates); Fenwick (prefix sum + point update, difference trick for ranges); sparse table ($O(1)$ idempotent queries, static).
- **Meta techniques:** coordinate compression; binary search on the answer; Mo's ($O((n+q) \sqrt{n})$); meet in the middle ($2^{n/2}$); sliding window $O(n)$.

R2.2 Complexity Cheat Sheet

Algorithm /	Build	Query / op	Notes
BST/AVL	$O(n)$	$O(\log n)$	AVL guarantees balance
Heap build / insert	$O(n)$ / -	$O(\log n)$	extract-min $O(\log n)$
Trie insert/search	$O(L)$	$O(L)$	L = word length
DFS/BFS	-	$O(V+E)$	components/cycles/shortest
Dijkstra	-	$O((V+E) \log V)$	non-negative

Algorithm /	Build	Query / op	Notes
Bellman-Ford	-	$O(V \cdot E)$	negative edges, cycle detect
Floyd-Warshall	-	$O(V^3)$	all pairs
Kruskal/Prim MST	$O(E \log E) / O((V+E) \log V)$	-	cut property
DSU find/unite	$O(n)$	$\sim O(1)$ amortized	$\alpha(n)$
Segment tree	$O(n)$	$O(\log n)$	lazy -> range updates
Fenwick	$O(n)$	$O(\log n)$	sums only
Sparse table	$O(n \log n)$	$O(1)$	idempotent, static
KMP / Z	$O(m)$	$O(n+m)$	pattern search
Manacher	$O(n)$	-	palindromes

R2.3 Key Patterns in Algorithm Design

- **Graph reachability** -> BFS/DFS; "shortest" -> BFS or Dijkstra.
- **Dependency order** -> topological sort (Kahn / reverse post-order DFS).
- **"Cheapest way to connect"** -> MST.
- **"Is u reachable from v / components under union"** -> DSU.
- **"Minimum steps with costs"** -> BFS/Dijkstra; "with constraints" -> state DP.
- **"All subsets / combinations"** -> bitmask or meet in the middle.
- **"Range query + update"** -> segment tree or Fenwick.
- **"Static range min/max/gcd"** -> sparse table.
- **"Count pairs with order property"** -> merge sort inversion / BIT.
- **"Longest X"** -> DP; **"number of ways"** -> DP sum recurrences.
- **"Minimize the maximum"** -> binary search on the answer + greedy check.

R2.4 Multiple-Choice Questions

1. In-order traversal of a BST yields: a) random order b) sorted order c) reverse sorted order.
2. AVL tree height for n nodes is: a) $O(n)$ b) $O(\log n)$ c) $O(\sqrt{n})$.
3. Dijkstra fails when: a) graph has negative edges b) graph is dense c) graph is unweighted.
4. Which structure supports range sums AND point updates in $O(\log n)$? a) sparse table b) Fenwick tree c) both b and segment tree d) hash table.
5. Kruskal sorts: a) vertices b) edges c) components.
6. BFS finds shortest paths when: a) weights equal b) weights non-negative c) always.
7. Topological order exists only for: a) any graph b) DAGs c) trees.
8. Fibonacci with memo is: a) $O(2^n)$ b) $O(n^2)$ c) $O(n)$.
9. The prefix function $\pi[5]$ of "aaaaaa" is: a) 0 b) 5 c) 1 d) 6 (proper prefix).
10. Path compression + union by rank give DSU find/unite: a) $O(\log n)$ b) $O(n)$ c) $\sim O(1)$ amortized.

Answers: 1-b, 2-b, 3-a, 4-c, 5-b, 6-a, 7-b, 8-c, 9-b, 10-c.

R2.5 True / False Questions

1. A tree with n nodes always has $n-1$ edges. (T)
2. AVL rotations happen only at the deepest imbalance. (T)
3. DFS can compute connected components in $O(V+E)$. (T)
4. Dijkstra works with negative weights. (F)
5. Bellman-Ford detects negative cycles reachable from the source. (T)
6. An MST is unique for every graph. (F - ties give multiple MSTs)
7. Greedy is always optimal for coin change. (F)
8. The sparse table supports point updates in $O(\log n)$. (F - it is static)
9. KMP runs in $O(n+m)$ even in the worst case. (T)
10. A segment tree needs $O(4n)$ nodes in the recursive array implementation. (T)

R2.6 Short-Answer Questions

1. Why does BFS (not DFS) compute shortest paths in unweighted graphs?
2. When would you pick Prim over Kruskal, and vice versa?
3. Explain the cut property and cycle property of MSTs.
4. What is the difference between the DP state "first i items" and "ending at i "?
5. Why is quicksort's worst case $O(n^2)$ and how do we mitigate it?

R2.7 Debugging Exercises

1. Dijkstra without the `if (d > dist[u]) continue;` stale-entry skip.
2. Union-Find unite that sets `parent[a] = b` without checking ranks.
3. BFS that checks the visited flag at *pop* time instead of *push* time.
4. Segment tree iterative query with `l += n, r += n` but using `[l, r]` closed.
5. DP knapsack with the inner loop ascending (unbounded reuse by accident).

R2.8 Code Tracing Exercises

1. What does Kruskal return for edges (0,1,1) (1,2,2) (0,2,3) (2,3,4)?
2. Trace Dijkstra from 0 on: 0-1(4), 0-2(1), 2-1(2), 1-3(1). Distances?
3. What is z for $s = \text{"ababab"}$?
4. $dp[i] = \max(dp[i-1], dp[i-2]+a[i])$ on $a = \{2,7,9,3,1\}$. What is the answer?
5. What does the XOR trick return for $\{1,2,1,3,2\}$?

R2.9 Answers to Short/Debug/Trace

Short answers: 1. BFS processes vertices in non-decreasing distance order (FIFO queue), so the first discovery of a vertex is via a shortest path; DFS explores one branch fully first and cannot guarantee minimality. 2. Prim when the adjacency list is given and the graph is dense ($O(V^2)$ matrix variant); Kruskal when edges are the natural input or the graph is sparse. Both are optimal - pick by input shape. 3. Cut property: the minimum-weight edge across any cut is in some MST. Cycle property: the maximum-weight edge in any cycle is never in an MST. 4. "First i items" = $dp[i]$ summarizes choices among items $0..i-1$ (order-free); "ending at i " = $dp[i]$ requires the sequence/action ends at element i (LIS). 5. A bad pivot (e.g., first element on sorted input) splits 1 and $n-1 \rightarrow O(n^2)$; mitigate by randomising the pivot or using median-of-three.

Debugging: 1. Without the skip, stale entries re-process finalized vertices; with non-negative weights the answer is still correct but slower - with negative weights it is wrong. 2. Skewed trees: $find()$ degrades toward $O(n)$ without rank/compression; use both optimizations. 3. Checking at pop time allows the same node to be pushed multiple times and processed repeatedly (still $O(V+E)$ in BFS but distance may be recorded wrong if using dist-on-first-push). 4. The half-open convention $[l, r)$ must be used consistently; closed intervals cause double counting at boundaries. 5. Ascending lets $dp[w - wt[i]]$ already include item $i \rightarrow$ unbounded knapsack. Descend to keep each item single-use.

Code tracing: 1. Sorted: (0,1,1) (1,2,2) (0,2,3) (2,3,4). Add (0,1), (1,2), skip (0,2) cycle, add (2,3) $\rightarrow$ weight $1+2+4 = 7$. 2. $dist = [0, 3, 1, 4]$ ($0 \rightarrow 2 \rightarrow 1$ is 3; $0 \rightarrow 2 \rightarrow 1 \rightarrow 3$ is 4). 3. $z = [0, 4, 2, 0, 2, 0]$? Recompute: "ababab" prefixes match at index 1 length 4, index 2 length 2, index 4 length 2 $\rightarrow z = [0, 4, 2, 0, 2, 0]$. 4. house robber: take $2+9+1=12 \rightarrow dp = [2, 7, 11, 11, 12] \rightarrow$ answer 12. 5. XOR of $\{1, 2, 1, 3, 2\} = 3$ (pairs 1 and 2 cancel, 3 remains).

R2.10 What to Revise Before Moving On

- Reproduce BFS, DFS, Dijkstra, Kruskal, and topological sort from scratch.
- Explain the difference between Dijkstra and Bellman-Ford in 30 seconds.
- Write a segment tree, Fenwick, and sparse table without looking.
- Solve any "longest/number-of-ways" problem by first naming the DP state aloud.
- Know exactly when greedy works (and when a counterexample kills it).

R2.11 Progress Check

If you can solve: Number of Islands (BFS/DFS), Course Schedule (topological sort), Network Delay Time (Dijkstra), Number of Provinces (DSU), Coin Change (DP), Single Number (XOR), Longest Palindromic Substring (Manacher/expand), and Range Sum Query with Mutable (segment tree) - you have solid command of modules 5-11. Move on to the back matter (STL reference, Big-O cheat sheet, FAQ, roadmap, recommended problems, glossary) and then start practising on real judges.

MODULE

Back Matter

Reference, roadmap, glossary

In this module you will master the data structures and algorithms above, with complete C++17 implementations, dry runs, complexity analysis and practice problems.

Appendix A: Complete STL Reference

A.1 Containers at a Glance

Container	Ordering	Access	Common ops
vector	insertion order	[] O(1)	push_back, pop_back, size, resize, at
deque	insertion order	[] O(1)	push_front/back, pop_front/back
list	insertion order	none	push/pop both ends, splice, merge
array	fixed order	[] O(1)	fill, size (compile-time)
string	insertion order	[] O(1)	find, substr, compare, +=, c_str
set	sorted	find O(log n)	insert, erase, lower_bound
multiset	sorted (dups ok)	find O(log n)	count, erase one/range
map	sorted by key	[] O(log n)	operator[], find, insert, erase
unordered_set	hash order	find O(1) avg	insert, erase, bucket_count
unordered_map	hash order	[] O(1) avg	emplace, at, find
stack	LIFO	top O(1)	push, pop, top, empty
queue	FIFO	front/back O(1)	push, pop, front, back
priority_queue	heap	top O(1)	push O(log n), pop O(log n)
bitset	fixed bits	[] O(1)	count, any, set, flip, &, ^, <<
tuple	fixed	get<I>	tie, structured bindings
pair	fixed	first, second	make_pair, comparison
span	view	[] O(1)	subspan, size (C++20)

A.2 Algorithms (all in <algorithm>)

A.3 Numerics and Utilities

A.4 String API

A.5 Custom Comparators

A.6 I/O Speedups

A.7 Common Pitfalls

- `v.size()` returns `size_t` (unsigned): `v.size() - 1` underflows when empty.
- `erase` on a `map/set` invalidates only that iterator (fine), but `vector` `erase` invalidates everything after it.
- `s.substr` copies: in hot loops use `string_view` or indices.
- `map::operator[]` inserts a default value; use `find/at` to avoid.
- `priority_queue` is a max-heap by default.
- `std::sort` requires random-access iterators (not for `list` - use `list::sort`).
- Comparing floats with `==` is dangerous; use an epsilon.

Appendix B: Big-O Cheat Sheet

B.1 Growth Rates (slow to fast)

Typical limits for 1 second of C++ (approx):

Complexity	Max n (~1 s)	Example
$O(n)$	$10^7 - 10^8$	linear scan
$O(n \log n)$	$\sim 10^6$	sort
$O(n \sqrt{n})$	$\sim 10^5 - 10^6$	Mo's
$O(n^2)$	$\sim 10^4$	double loop
$O(n^3)$	~ 500	Floyd-Warshall
$O(2^n)$	~ 20	bitmask subsets
$O(n!)$	~ 10	permutations
$O(2^n * n)$	$\sim 15-18$	bitmask DP

B.2 Operations Summary

Operation	Time
vector push_back (amortized)	$O(1)$
vector insert/erase middle	$O(n)$
unordered_map find/insert	$O(1)$ avg
map/set find/insert	$O(\log n)$
priority_queue push/pop	$O(\log n)$
string find (naive)	$O(n*m)$ worst
KMP / Z pattern search	$O(n+m)$
sort (introsort)	$O(n \log n)$
stable_sort	$O(n \log n)$
heapify an array	$O(n)$
segment tree query/update	$O(\log n)$
Fenwick add/sum	$O(\log n)$
sparse table query	$O(1)$
DFS/BFS	$O(V+E)$
Dijkstra	$O((V+E) \log V)$
Bellman-Ford	$O(V \cdot E)$
Floyd-Warshall	$O(V^3)$
Kruskal	$O(E \log E)$
Prim (heap)	$O((V+E) \log V)$
DSU find/unite	$\sim O(1)$ amortized
LCS / edit distance (DP)	$O(n \cdot m)$

Operation	Time
0/1 knapsack (DP)	$O(n \cdot W)$

B.3 Space Summary

- Arrays: $O(n)$. Matrices: $O(nm)$ or $O(n^2)$.
- Recursion: $O(\text{depth})$ stack - beware deep recursion on $n \sim 10^6$.
- Trees: $O(n)$. Adjacency list: $O(V+E)$. Matrix: $O(V^2)$.
- Segment tree (recursive array): $O(4n)$. Fenwick: $O(n)$. Sparse table: $O(n \log n)$.
- DP tables: $O(\text{number of states})$; often reducible to $O(k)$ by keeping only the last rows.

B.4 Time-limit mental math

- 10^8 simple operations ~ 1 second in C++ (rough; depends on the machine).
 - Always multiply inner loops by transition cost.
 - When in doubt, mention the bound and let the interviewer verify feasibility.
-

Appendix C: Frequently Asked Questions

C.1 Which is faster: `unordered_map` or `map`?

For most uses `unordered_map` (hash, $O(1)$ average) wins on speed but loses on determinism and worst case. Use `map` when you need sorted iteration or guaranteed $O(\log n)$. Beware adversarial inputs making the hash degrade - randomized hashing or a custom hash helps.

C.2 When is BFS better than DFS?

BFS for shortest paths (unweighted), level order, and anything requiring minimal steps. DFS for reachability, cycles, components, topological order, SCCs, bridges - and when the recursion depth is small. BFS uses a queue (more memory for wide graphs); DFS uses a stack (more memory for deep graphs).

C.3 What is the difference between top-down and bottom-up DP?

Both compute the same recurrence. Top-down = recursion + memo (easy to write, uses the call stack, natural for hard-to-order states). Bottom-up = iterative table fill (no recursion, control over memory, sometimes faster). Prefer the natural one; convert when the state order is confusing.

C.4 How do I know if greedy is safe?

Write the greedy rule, then actively hunt for a counterexample. If you cannot find one, sketch an exchange argument: take any optimal solution and show you can transform it into the greedy one without losing optimality. Coin change with $\{1,3,4\}$ and amount 6 is the canonical counterexample.

C.5 Why is the knapsack inner loop descending?

So that `dp[w - wt[i]]` refers to the previous item row - each item is used at most once. Ascending would allow reusing the same item (unbounded knapsack).

C.6 What are "idempotent" operations for sparse tables?

Operations where overlapping answers do not change the result: min, max, gcd, lcm, bitwise OR/AND, LCP. Sum and XOR are NOT idempotent - use a segment tree or Fenwick for those.

C.7 Should I use recursion or iteration for tree traversals?

Recursion is clear and standard for pre/in/post-order (depth $\sim \log n$ for balanced trees). For very deep trees (skewed, $n = 10^6$) the stack may overflow - switch to an explicit stack/iteration. BFS is always iterative (queue).

C.8 How do I handle 1-based vs 0-based indexing?

Pick one convention (0-based is standard in C++). For Fenwick and some segment trees, 1-based is natural; shift inputs by +1 when feeding them in. Be explicit in your head and in comments where the shift happens - most off-by-one bugs in graphs/trees come from mixing the two.

C.9 What does " $O(n \log n)$ " mean practically for sorting?

$\sim n \log_2 n$ comparisons. For $n = 10^6$ that is ~ 20 million operations - well within one second in C++. `std::sort` achieves this with quicksort + heapsort fallback; mergesort gives stability at the cost of $O(n)$ extra space.

C.10 How do I get faster at these problems?

Pattern recognition: after solving a problem, categorize it (which chapter's pattern?). Maintain a "one-liner" mental index: BFS for steps, DP for ways/max, greedy for intervals, BIT for prefix-update, topo for dependencies. Do spaced repetition of the Practice Sets in this book, then grind a curated problem list (Appendix E).

Appendix D: The DSA Roadmap

D.1 Phase 1 - Foundations (weeks 1-2)

C++ essentials and STL; time complexity; arrays, strings, searching, sorting. Finish: Chapters 1-7 and Revision 1. Target: Two Sum, Binary Search, Merge Sorted Arrays, Valid Anagram.

D.2 Phase 2 - Core structures (weeks 3-5)

Recursion, linked lists, stacks, queues, hashing. Finish: Chapters 8-12. Target: Reverse Linked List, Valid Parentheses, LRU Cache (later), Two Pointers on lists.

D.3 Phase 3 - Trees and advanced structures (weeks 6-8)

Trees, BST, AVL, heaps, tries. Finish: Chapters 13-17. Target: Inorder Traversal, Validate BST, Level Order, Kth Largest Element, Implement Trie.

D.4 Phase 4 - Graphs (weeks 9-11)

Representation, BFS/DFS, shortest paths, MST, topological sort, DSU, advanced graph algorithms. Finish: Chapters 18-23. Target: Number of Islands, Course Schedule, Network Delay Time, Number of Provinces, Critical Connections.

D.5 Phase 5 - Algorithm design (weeks 12-15)

Greedy, divide and conquer, DP, bit manipulation. Finish: Chapters 24-27 and Revision 2. Target: Jump Game, Count Inversions, Coin Change, House Robber, Single Number.

D.6 Phase 6 - Specialized topics (weeks 16-18)

Advanced structures, string algorithms, geometry, advanced techniques. Finish: Chapters 28-34 plus Appendices. Target: Range Sum with Updates, strStr, Longest Palindromic Substring, Split Array Largest Sum.

D.7 Phase 7 - Interview prep (weeks 19+)

Re-solve all Practice Sets; simulate mock interviews (10 minutes to design, 15 to code, 5 to test); review the FAQ and cheat sheets; do the Recommended Problem List (Appendix E); practise explaining complexity aloud.

D.8 Golden advice

- Solve, then re-solve a week later from memory - that is how retention works.
 - Always state complexity out loud when you present a solution.
 - If stuck 30+ minutes, peek at the pattern, not the full solution.
 - Compile and run every snippet in this book; verify by tracing small inputs.
-

Appendix E: Recommended Problem List

E.1 Must-do (in order)

1. Two Sum - hash map (Ch 12).
2. Contains Duplicate - set.
3. Best Time to Buy and Sell Stock - one pass.
4. Valid Parentheses - stack (Ch 10).
5. Merge Two Sorted Lists - pointers (Ch 9).
6. Linked List Cycle - Floyd's cycle detection (Ch 9).
7. Reverse Linked List - iterative/recursive (Ch 9).
8. Binary Search - boundary variants (Ch 6).
9. Search Insert Position - lower_bound (Ch 6).
10. First Bad Version - binary search on answer (Ch 6).
11. Valid Anagram - frequency array (Ch 5).
12. Longest Substring Without Repeating - sliding window (Ch 4).
13. Group Anagrams - hash of counters (Ch 12).
14. Maximum Subarray - Kadane (Ch 4).
15. Product of Array Except Self - prefix/suffix (Ch 4).
16. Merge Intervals - sort + scan (Ch 24 greedy).
17. Number of Islands - flood fill BFS/DFS (Ch 19).
18. Rotting Oranges - multi-source BFS (Ch 19).
19. Clone Graph - DFS/BFS with map (Ch 19).
20. Course Schedule I & II - topological sort (Ch 22).
21. Network Delay Time - Dijkstra (Ch 20).
22. Number of Provinces - DSU (Ch 21).
23. Kth Largest Element in an Array - heap/quickselect (Ch 16).
24. Implement Trie - trie (Ch 17).
25. Word Search - backtracking (Ch 8).
26. Coin Change - DP (Ch 26).
27. House Robber - DP (Ch 26).
28. Longest Increasing Subsequence - DP (Ch 26).
29. Edit Distance - 2D DP (Ch 26).
30. Longest Palindromic Substring - expand/Manacher (Ch 32).
31. Single Number - XOR (Ch 27).
32. Missing Number - XOR/sum (Ch 27).
33. Reverse Bits - bit tricks (Ch 27).
34. Count Bits - popcount (Ch 27).
35. Range Sum Query (Immutable) - prefix sums (Ch 3).

- 36. Range Sum Query (Mutable) - Fenwick/segment tree (Ch 28-29).
- 37. strStr - KMP (Ch 31).
- 38. Group Shifted Strings / Repeated Substring Pattern - KMP pi (Ch 31).
- 39. Split Array Largest Sum - binary search on answer (Ch 34).
- 40. Longest Consecutive Sequence - set/window (Ch 12).

E.2 Next tier (interviews)

LRU Cache, Serialize and Deserialize BST, Lowest Common Ancestor, Binary Tree Level Order, Validate BST, Diameter of a Tree (DP on tree), Top K Frequent Elements, Task Scheduler (greedy + heap), Word Ladder (BFS), Jump Game II, Gas Station, Min Stack, Evaluate Reverse Polish Notation, K Closest Points, Container With Most Water (two pointers), Trapping Rain Water.

E.3 Competitive programming tier

Segment tree beats, persistent segment tree, suffix array problems, heavy-light decomposition, flow (max-flow/min-cut), 2-SAT, string hashing duplicates, CDQ divide and conquer, tree DP + rerooting, Mo's with modifications.

E.4 How to use this list

- Solve in order; do not skip to hard ones early.
 - After solving, read the editorial and compare your complexity.
 - Re-solve each problem after 7 and 30 days (spaced repetition).
 - Track your time; interview readiness ~ "easy < 10 min, medium < 25 min".
-

Appendix F: Glossary

Term	Meaning
Algorithm	A finite sequence of well-defined steps solving a problem
Amortized cost	Average cost per operation over a sequence
Big-O (O)	Asymptotic upper bound on growth
Big-Omega (Ω)	Asymptotic lower bound
Big-Theta (Θ)	Tight asymptotic bound
BST	Binary search tree; in-order gives sorted output
BFS / DFS	Breadth-first / depth-first traversal
Complexity	Time/space growth as a function of input size n
DAG	Directed acyclic graph (has topological order)
DP	Dynamic programming: reuse overlapping subproblem answers
DSU / Union-Find	Disjoint-set structure for connectivity and merges
Edge	A connection between two vertices
Graph	Vertices + edges, possibly directed/weighted
Greedy	Local optimal choices aiming at a global optimum
Heap	Complete binary tree satisfying the heap property
Idempotent	Repeating/overlapping the operation does not change the result
Inversion	Pair $(i < j)$ with $a[i] > a[j]$
Lazy propagation	Deferred range updates in segment trees
LCA	Lowest common ancestor of two tree nodes
LIS / LCS	Longest increasing subsequence / common subsequence
Low-link	Smallest discovery time reachable from a DFS subtree
Memoization	Caching recursive subproblem answers (top-down DP)
MST	Minimum spanning tree (cheapest connected subgraph)
Prefix function	KMP's longest proper prefix-suffix lengths
Recurrence	Formula relating a DP state to smaller states
Relaxation	Improving $\text{dist}[v]$ via an edge from a closer node
Segment tree	Balanced tree answering range queries + updates in $O(\log n)$
Spanning tree	Connected subgraph with $V-1$ edges, no cycles
Sparse table	Static $O(1)$ range-query table for idempotent ops
State	The tuple defining a DP subproblem
Topological order	Ordering of a DAG with all edges forward
Vertex	A node of a graph (also called node)
Weighted graph	Graph with costs on edges

Closing Words

You made it. From a bare count to segment trees, from for loops to Dijkstra, and from guesswork to proofs. The real growth now comes from *using* all of it:

- Rebuild the cheat sheets from memory - if you can reproduce them, you know them.
- Solve the recommended problems, then re-solve them a month later.
- Explain an algorithm to a friend; the gaps you find are your next lesson.
- Every interview, contest, and side project is another rep.

Good luck - and keep a notepad of your favourite problems. Happy coding.